

Document Map and Executive Summary

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-00
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Orient readers, declare the binding product baseline, and map every source-of-truth document.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio,

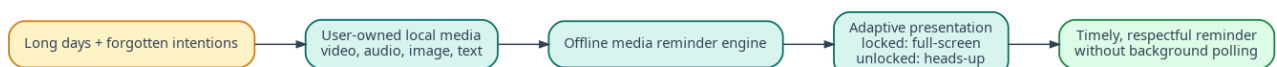
Term	Meaning
	image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Executive summary

Media Reminder is an Android-first, offline-only application that lets a person import local media and schedule it as a reminder. The app adapts the interruption to device state:

- a locked or non-interactive device may wake into a full-screen alarm activity;
- an unlocked device uses Android's system heads-up notification and never forces a full-screen takeover;
- while Media Reminder itself is foregrounded, an in-app strip provides the least disruptive interaction.

The reminder does not autoplay its attached video merely because it is due. Alert sound and vibration may begin according to the selected profile, but the reminder media begins only after the user selects **Play**. The application is local-only: no account, server, advertising, telemetry, or Internet permission is required. It is scheduled by Android rather than by an always-running loop.



2 Binding baseline

Area	Approved baseline
Product name	Media Reminder
Initial platform	Android phones and foldables; tablets are supported with responsive layouts but are not the primary alarm surface
App framework	React Native 0.86.x, TypeScript, New Architecture
Reliability core	Kotlin native module, Android components, Room, DataStore,

Area	Approved baseline
	AlarmManager, Media3
SDK baseline	minSdk 26, compileSdk 37, targetSdk 36 for the first public release; retest and move target to 37 when required by distribution policy
Connectivity	No backend and no INTERNET permission in the production manifest
Persistence	App-owned files plus Room; system settings are discovered, not treated as app-owned data
Scheduler	One next due system alarm, recalculated after every relevant state change; no second-by-second polling and no resident idle service
Locked-device presentation	Full-screen intent/activity only when the device is locked or non-interactive and Android permits it
Unlocked-device presentation	High-importance heads-up notification; exact visual height is controlled by Android and the OEM
Backup	Versioned logical ZIP containing JSON records and media with SHA-256 checksums; never a raw database copy
Distribution	Signed APK and checksums through GitHub Releases; Play distribution remains optional
License	Apache License 2.0 for application source, excluding user-provided media and brand assets identified separately

3 Product boundaries

Media Reminder is suitable for personal duas and adhkar, study clips, exercise demonstrations, routines, language practice, and other rich-media reminders. It is **not** certified as a medical device, emergency warning system, safety-of-life alarm, or guaranteed exact-timing service. Android permissions, Doze policy, OEM modifications, device shutdown, force-stop, and user-disabled notification channels can prevent or alter an alert. The product **MUST** explain those states rather than claiming impossible certainty.

The application **MUST NOT**:

1. upload or analyze a person's media;
2. request overlay permission to draw over other apps;
3. ask for unrestricted battery optimization exemption as a default setup step;
4. keep a foreground service running while idle;

5. silently enable a reminder whose exact scheduling prerequisites are unavailable;
6. restore operating-system permissions, notification channel choices, or ringtone URIs from a backup as though they were portable app data.

4 Source-of-truth map

ID	Document	Authoritative for
MR-00	Document Map and Executive Summary	Scope map, precedence, fixed baseline
MR-01	Vision and Product Charter	Purpose, principles, goals, non-goals
MR-02	Product Requirements Document	Users, outcomes, scope, requirements, success criteria
MR-03	User Experience and Interaction Specification	Screen flows, adaptive alarm interaction, copy, states
MR-04	Visual Design System	Tokens, components, responsive behavior, motion
MR-05	Functional Feature Specification	Feature-level behavior and acceptance rules
MR-06	Android Alarm, Notification and Battery Specification	Native scheduling, alerts, permissions, lifecycle
MR-07	Technical Architecture	Components, dependency direction, deployment architecture
MR-08	Internal Module and Data Contracts	Typed boundaries, commands, events, error contracts
MR-09	Database and Local Storage Specification	Tables, relationships, migrations, files and transactions
MR-10	Backup, Export and Import Specification	Portable archive format and restore algorithm
MR-11	Edge Cases, Failure Recovery and Conflict Handling	Failure taxonomy, degradation and recovery rules
MR-12	Security, Privacy and Threat Model	Assets, trust boundaries, misuse controls, secure defaults
MR-13	Accessibility, Localization and Inclusive Design	WCAG-informed behavior, TalkBack,

ID	Document	Authoritative for
		scaling, RTL and copy
MR-14	Testing, Quality and Device Matrix	Test strategy, devices, release evidence and defect gates
MR-15	Performance, Battery and Operational Budgets	Quantitative budgets and measurement procedure
MR-16	Roadmap, Release and Success Metrics	Release increments, metrics and change control
MR-17	Architecture Decision Records	Binding architectural choices and their rationale
MR-18	Contribution Guide and Engineering Standards	Repository rules, code review and implementation quality
MR-19	AI Agent Guide and Master Loop Prompt	Safe AI-assisted development loop and stop conditions
MR-20	Release, Distribution, Portfolio and Maintenance Guide	Signing, APK publication, support and portfolio evidence
MR-21	Requirements Traceability and Acceptance Catalog	Requirement-to-test mapping and release acceptance
MR-22	Research Baseline and Official Sources	Current official platform facts and refresh policy

5 Requirement taxonomy

Requirement identifiers use a prefix and a sequence. The catalog in MR-21 is authoritative.

Prefix	Domain	Example
PRD	Product outcome or scope	PRD-012
UX	Interaction or presentation	UX-041
FUN	Feature behavior	FUN-103
AND	Android/native behavior	AND-057
DAT	Persistence and integrity	DAT-026
BKP	Backup/restore	BKP-044
SEC	Security/privacy	SEC-018
ACC	Accessibility/localization	ACC-021
NFR	Performance/battery/	NFR-034

Prefix	Domain	Example
	reliability	
REL	Release/distribution	REL-015

Every implemented requirement **MUST** have at least one verification method: automated test, instrumented test, manual test, static inspection, or measured evidence. A requirement is not complete because a screen exists; the mapped acceptance evidence must pass.

6 Product state model

The app separates four kinds of state:

1. **Intent state** - reminders, schedules, profiles, categories and user settings.
2. **Derived state** - next occurrence, notification content and calculated recurrence.
3. **Runtime state** - active alarm session, bounded ringing service, wake state and action idempotency.
4. **Platform state** - notification permission, exact-alarm access, channel importance, full-screen eligibility, battery policy and device lock state.

Only the first three are persisted as app state. Platform state is probed and displayed. This distinction prevents backup restore, migrations, and UI assumptions from overwriting choices owned by Android.

7 Release definition

A public v1 release is acceptable only when:

- all P0 requirements in MR-21 pass;
- there are no open critical or high-severity security, data-loss, alarm-action, or accessibility defects;
- locked, unlocked, Doze, reboot, timezone-change, force-stop explanation, denied-permission and backup recovery flows have been tested on the required device matrix;
- idle operation shows no resident foreground service and no periodic wake loop;
- import/export round trips preserve every supported logical record and media checksum;
- the APK is reproducibly built from a tagged commit, signed, checksum-published and accompanied by a privacy statement and known-limitations page;
- included screenshots and demo media are original, licensed, or intentionally synthetic.

8 Change control

A proposed change that affects alarm timing, permission scope, backup compatibility, native module contracts, file ownership, or public security posture **MUST** update the ADR log before implementation. Schema and backup changes **MUST** include migration and compatibility tests in the same pull request. Product copy changes that alter a promise about reliability **MUST** be reviewed against MR-06 and MR-11.

9 Implementation reading order

A developer or coding agent starts with MR-00, MR-17, MR-06, MR-07 and MR-09, then reads the relevant feature and UX documents. A designer starts with MR-01 through MR-05 and MR-13. A tester starts with MR-21, MR-14, MR-11 and MR-06. A release manager starts with MR-20, MR-14, MR-15 and MR-22.

9.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Vision and Product Charter

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-01
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define why the product exists, who it serves, and the principles that constrain every design and engineering decision.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Vision

Media Reminder helps people remember meaningful actions by presenting the right local media at the right time without demanding cloud access, constant background execution, or an unnecessary interruption.

The originating use case is personal Islamic remembrance: a person saves useful videos about duas and adhkar, imports them, and schedules them as timely prompts. The product deliberately generalizes the engine so the same respectful interaction can support study, practice, routines and other personal goals without weakening the original need.

2 Problem statement

Conventional alarm applications are reliable but content-poor. Social feeds contain memorable audio-visual reminders but are unpredictable, distracting, network-dependent and optimized for continued consumption. Calendar notifications are compact but rarely provide the media context that helps a person remember what to do. A user therefore needs a private bridge between local content and system-grade reminder behavior.

The most important problem is not merely “play a video at a time.” It is to do so while respecting three competing needs:

- **reliability:** a due reminder should be scheduled by Android and survive normal app process death;
- **attention proportionality:** a locked phone may need an alarm surface, while an actively used phone should not be hijacked;
- **efficiency and privacy:** the app should sleep while idle, store everything locally and avoid broad permissions.

3 Target users

3.1 Primary persona: purposeful reminder user

A person who already has short local videos, audio or images that reinforce a personal practice. They want simple scheduling, custom snooze, clear completion, and confidence that the app is not uploading their library.

3.2 Secondary persona: routine learner

A student or language learner who attaches a short demonstration, pronunciation clip or explanation to a recurring reminder. They value categories, multiple schedules, quiet profiles and fast playback.

3.3 Secondary persona: privacy-conscious organizer

A person who avoids cloud accounts and wants a portable offline archive when changing phones. They need a transparent export, validation before restore and no vendor lock-in.

3.4 Portfolio evaluator

A recruiter, engineer or collaborator who evaluates the project as evidence of native Android integration, offline architecture, local data integrity, accessible mobile UX and battery-aware engineering. Portfolio presentation MUST remain secondary to user safety and product correctness.

4 Jobs to be done

Situation	Job	Desired outcome
I find a useful reminder video	Preserve it in a focused library	I can find and schedule it without returning to a social feed
I need a reminder at a meaningful time	Attach content and recurrence	Android presents it with appropriate urgency
I am actively using my phone	Avoid a forced screen takeover	I receive a compact actionable alert
My phone is locked	Make the due event unmistakable	The screen may wake into an alarm surface with clear actions
I change devices	Carry my configuration	A validated ZIP restores supported data and media offline
I am worried about battery	Avoid constant background work	The app remains dormant except for user actions and scheduled events

5 Product principles

5.1 1. Calm before clever

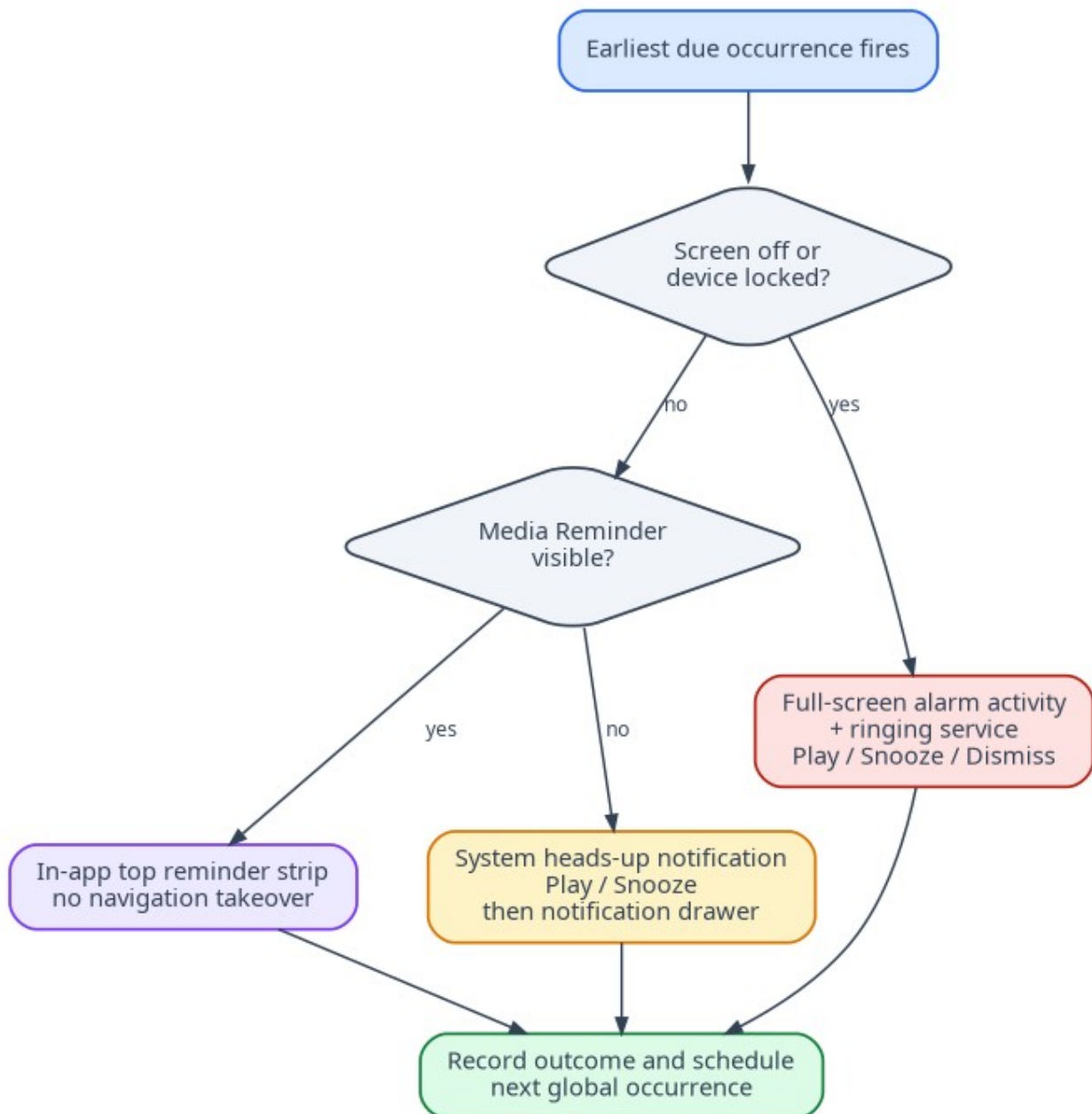
The interface MUST feel intentional, not gamified. It uses plain language, restrained motion and no guilt-inducing streaks. A missed reminder is information, not a moral judgment.

5.2 2. System-compliant reliability

The app uses Android's alarm and notification mechanisms rather than fighting the operating system. It MUST disclose when exact-alarm access, notification permission, channel importance, full-screen eligibility, device state or OEM restrictions reduce behavior.

5.3 3. Interruption follows context

Full-screen presentation is reserved for a locked or non-interactive device. An unlocked device receives a system heads-up notification. The app MUST NOT use `SYSTEM_ALERT_WINDOW` to simulate an arbitrary 20%-height overlay over other apps.



5.4 4. Local means local

Core functionality works in airplane mode. The production manifest contains no Internet permission. Media is copied into app-owned storage so later gallery deletion does not silently break a reminder.

5.5 5. Battery is a feature

No always-running JavaScript timer, periodic polling loop or idle foreground service. The scheduler holds one next due alarm, wakes briefly, reschedules and returns to idle. Battery claims are verified with measurements, not inferred from architecture alone.

5.6 6. Portability without pretending

Backups carry logical app data and media, not operating-system grants or device-specific paths. Import MUST preview what can and cannot be restored, validate every checksum and commit atomically.

5.7 7. User action remains sovereign

The user can snooze, dismiss or play. Video begins only after Play. Profiles may make alert sound persistent, but the app MUST always expose a clear stop path and a finite automatic timeout.

5.8 8. Inclusive by default

TalkBack labels, keyboard/switch access, 200% font scaling, high contrast, reduced motion and RTL-safe layout are product requirements, not later polish.

6 Goals

Goal ID	Goal	Indicator
G-01	Make local media schedulable as respectful reminders	A first-time user imports and schedules a reminder without documentation
G-02	Deliver state-adaptive alert presentation	Locked and unlocked test scenarios select different surfaces correctly
G-03	Remain offline and private	No network permission; no request leaves the app in static and runtime inspection
G-04	Remain battery-efficient while idle	No resident service or periodic wake loop; measured idle delta remains inside MR-15 budget
G-05	Make device migration trustworthy	Export/import round trip preserves supported records and file hashes
G-06	Demonstrate professional engineering	Tagged releases include documentation, tests, SBOM, signing and reproducible evidence

7 Non-goals

Version 1 does not provide:

- cloud accounts, cloud sync, remote backup or multi-device real-time state;
- iOS, desktop, Wear OS, Android Auto or web clients;
- downloaded social-media content acquisition or copyright circumvention;
- automatic prayer times, religious rulings, Quran content certification or built-in theological content;
- emergency alerts, medication adherence guarantees, life-critical scheduling or legal compliance claims;
- GPS, geofences, Bluetooth triggers, sensor-based habits or AI-generated reminder advice;
- public sharing, social feeds, leaderboards, streak pressure or advertising;
- arbitrary overlays over other apps;
- exact restoration of Android notification channels, permissions, alarm tones or OEM battery policy.

8 Ethical use statement

Users are responsible for the media they import and the contexts in which alerts play. The app **SHOULD** remind them to respect copyright, privacy, prayer, meetings, driving and shared spaces. It **MUST** avoid shipping copyrighted social clips as demo content. The reference starter library, if created, **MUST** use owned, licensed or public-domain media with documented provenance.

9 Success horizon

9.1 First-use success

A new user can understand the product, grant only the permissions needed for a chosen behavior, import media, create a daily reminder and test the alert. Permission education occurs at the moment of need, not as an undifferentiated startup wall.

9.2 Thirty-day success

The app remains installed, reminders continue after normal process death and reboot, the library stays intact, and the user can identify any degraded system permission from the Health screen.

9.3 Migration success

A person exports on one device, transfers the ZIP using any file-sharing method they choose, imports on another supported device, reviews conflicts and restores. The app then guides them through regranted device-owned permissions before enabling exact reminders.

10 Charter constraints

The team **MUST** treat the following as release blockers:

- a full-screen activity appears while the device is unlocked and another app is in active use;
- an alert cannot be silenced because the React Native runtime failed to initialize;
- imported media can escape app storage or be overwritten without a transaction record;
- a malformed ZIP can write outside the staging directory;
- the scheduler duplicates an occurrence after reboot or timezone change;

- the app is idle with a persistent foreground service;
- the product claims a fixed heads-up height or guaranteed exact delivery across Android devices;
- destructive import proceeds without a preview and explicit confirmation.

11 Ownership and decision rights

The product owner approves scope, brand and release priorities. The Android reliability owner has veto authority over changes that weaken alarm action availability, data integrity, permission compliance or battery behavior. UX can choose presentation within platform constraints but cannot replace a system-owned heads-up notification with overlay permission. Backup format changes require architecture, QA and security review.

12 Brand direction

The working identity is neutral and descriptive: **Media Reminder**. The product should convey **calm, intentional and dependable**. Islamic remembrance is the lead authentic use case in narrative and screenshots where appropriate, while the core product language remains inclusive of other lawful personal reminder uses.

12.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Product Requirements Document

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-02
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Specify product outcomes, release scope, user stories, functional and non-functional requirements, and acceptance criteria.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Product overview

Media Reminder is a local Android application that connects imported content to recurring or one-time schedules. It adapts presentation to device state, stores all content on-device, supports portable ZIP backup, and uses system scheduling rather than permanent background execution.

2 Release scope

2.1 P0 - public MVP

- import local video, audio and image files using Android Photo Picker or Storage Access Framework;
- create app-owned media records with title, optional notes, category and tags;
- create, edit, enable, disable and delete one-time, daily and weekly reminders;
- assign one of three reusable profiles: Gentle, Standard or Persistent;
- configurable snooze presets and a bounded custom snooze;
- locked/non-interactive full-screen alarm path where eligible;
- unlocked heads-up notification with Play, Snooze and Dismiss actions;
- foreground in-app reminder strip;
- local video/audio/image playback after Play;
- boot and app-update rescheduling;
- timezone and wall-clock change reconciliation;
- versioned ZIP export and import with preview, validation, merge or replace;
- Settings Health screen for notification permission, exact alarms, channels and full-screen eligibility;
- light, dark and system theme; TalkBack; dynamic text; reduced motion;

- signed APK release, privacy notice and diagnostic export without personal media.

2.2 P1 - post-MVP

- text cards as a first-class asset type;
- advanced recurrence including monthly day rules and interval rules;
- optional local completion history dashboard;
- duplicate detection by content hash before copying;
- user-created profiles beyond the three defaults;
- backup encryption with a user-supplied passphrase;
- home-screen widget and quick-add shortcut;
- localization for Tamil and Arabic after string externalization and RTL validation.

2.3 P2 - exploratory

- playlists or random media pools;
- Wear OS companion;
- optional user-operated cloud file target without an app backend;
- prayer-time plugin supplied as a separate, clearly scoped module;
- iOS feasibility study with documented scheduling differences.

3 Primary user journey

1. User opens the app and sees a concise explanation of local-only operation.
2. User imports a media file through a system picker.
3. App copies the selected stream to a pending file, hashes it, validates type/size, atomically promotes it and creates the database record.
4. User opens the asset, selects **Add reminder**, chooses a time, repeat rule, profile and snooze behavior.
5. App saves the intent, calculates the next occurrence and schedules the global next Android alarm.
6. At due time, native code decides the presentation from lock and interactive state.
7. User selects Play, Snooze or Dismiss without depending on JavaScript startup.
8. App records the outcome locally, calculates the next occurrence and returns to idle.
9. User may export a portable archive and restore it on a new device.

4 Product requirements

4.1 Core and privacy

- **PRD-001:** The production app MUST complete all core functions with airplane mode enabled.
- **PRD-002:** The production manifest MUST NOT declare `android.permission.INTERNET`.
- **PRD-003:** The app MUST NOT require an account, email address, phone number, analytics consent or cloud service.

- **PRD-004:** The app MUST explain that imported media is copied to private app storage and that uninstalling the app removes that app-owned copy unless backed up.
- **PRD-005:** User-facing reliability language MUST describe platform limitations and MUST NOT promise guaranteed delivery.

4.2 Media library

- **PRD-010:** A user MUST be able to import supported video, audio and image formats through system-owned pickers.
- **PRD-011:** A successful import MUST remain playable if the original gallery item is later deleted.
- **PRD-012:** A user MUST be able to rename, categorize, tag, preview and delete an asset.
- **PRD-013:** Deleting an asset used by reminders MUST require a choice to cancel, delete dependent reminders, or detach them into a disabled missing-media state.
- **PRD-014:** Unsupported, corrupt, oversized or insufficient-storage imports MUST fail safely without a partial library record.

4.3 Reminder authoring

- **PRD-020:** A reminder MUST combine one content item, one schedule rule, one profile and user-visible label information.
- **PRD-021:** MVP scheduling MUST support one-time, daily and selected weekdays.
- **PRD-022:** A user MUST be able to enable, disable, duplicate and delete a reminder.
- **PRD-023:** The editor MUST show the next calculated occurrence before Save.
- **PRD-024:** Invalid or nonexistent local times MUST be explained and resolved deterministically under MR-06.
- **PRD-025:** No reminder is treated as reliably active until required system capabilities are present or the user explicitly accepts Limited mode.

4.4 Alert behavior

- **PRD-030:** When the device is locked or non-interactive, an eligible due occurrence MAY wake a native full-screen alarm activity.
- **PRD-031:** When the device is unlocked and interactive, a due occurrence MUST NOT launch a full-screen activity over the current app; it MUST use a system heads-up notification when available.
- **PRD-032:** The app MUST NOT request overlay permission for reminder presentation.
- **PRD-033:** Play, Snooze and Dismiss MUST function from the notification or native alarm activity without waiting for React Native initialization.
- **PRD-034:** Attached media MUST begin only after the user selects Play.
- **PRD-035:** Every ringing path MUST have a finite timeout and an immediately reachable stop action.
- **PRD-036:** Android or OEM control of heads-up layout MUST be disclosed; the app MUST not promise exact screen-height coverage.

4.5 Battery and resilience

- **PRD-040:** Idle operation MUST use no resident foreground service, wake lock, JavaScript interval or periodic worker solely to monitor time.

- **PRD-041:** The scheduler MUST maintain one next due one-shot system alarm and recalculate after changes.
- **PRD-042:** Alarms MUST be reconciled after supported reboot, app update, timezone and clock-change broadcasts.
- **PRD-043:** An exact-alarm-denied state MUST be visible and actionable.
- **PRD-044:** Force-stop consequences MUST be explained in Health because Android prevents app wake until the user relaunches it.

4.6 Backup and migration

- **PRD-050:** Export MUST create a local, versioned ZIP with logical JSON data, media and checksums.
- **PRD-051:** Export MUST exclude raw database files, caches, thumbnails, device paths, permission grants and notification channel state.
- **PRD-052:** Import MUST validate archive structure, uncompressed limits, path safety, schema, references, hashes and available storage before commit.
- **PRD-053:** Import MUST support Inspect only, Merge and Replace modes.
- **PRD-054:** Replace MUST create a rollback snapshot or equivalent transactional recovery before destructive commit.
- **PRD-055:** Restored reminders MUST remain disabled until necessary device-owned capabilities are re-evaluated and the user confirms activation.

4.7 Accessibility

- **PRD-060:** All functions MUST be operable with TalkBack and switch access.
- **PRD-061:** Text MUST remain usable at 200% font scaling without clipping critical actions.
- **PRD-062:** Color MUST NOT be the only state indicator.
- **PRD-063:** Reduced-motion preference MUST remove nonessential movement.
- **PRD-064:** Alarm actions MUST use clear verbs, large targets and differentiated semantics.

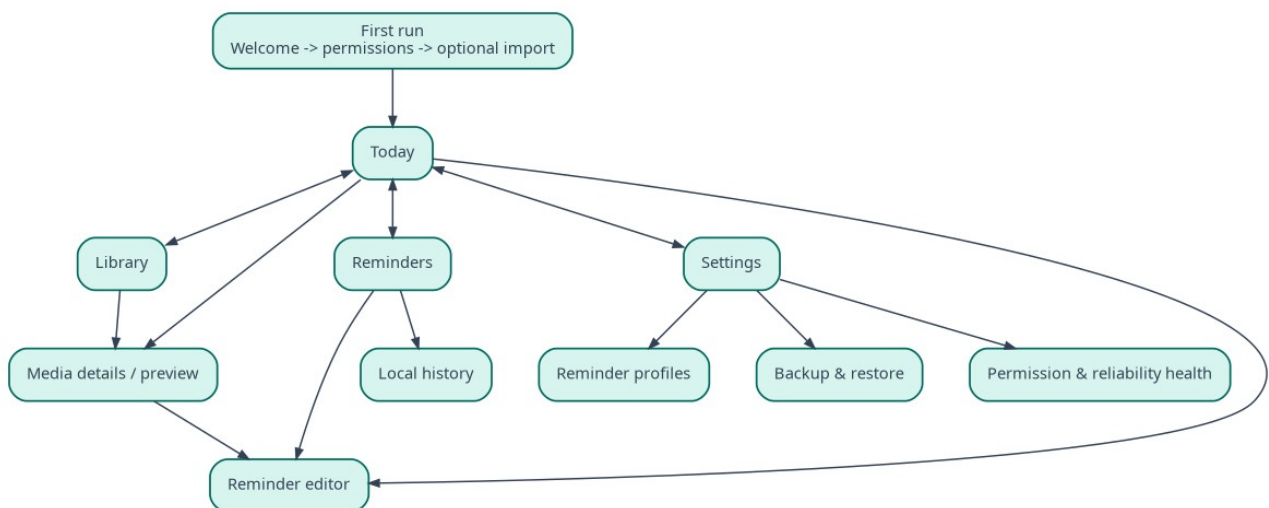
5 User stories and acceptance

Story	Acceptance summary
As a user, I import a downloaded dua video	Picker opens; selected bytes are copied, hashed and previewable offline; original deletion does not break playback
As a user, I schedule Morning Adhkar daily	Editor shows next time; reminder persists; one system alarm is recalculated; due event appears appropriately
As a user actively browsing another app	No full-screen takeover occurs; a heads-up notification offers Play, Snooze and Dismiss
As a user with a locked phone	Screen may wake to a native alarm surface; buttons work before RN is

Story	Acceptance summary
	ready; media waits for Play
As a user changing phones	Export creates a shareable ZIP; new device previews, validates and imports; permissions are re-requested separately
As a privacy-conscious user	Runtime inspection shows no network traffic and the manifest lacks Internet permission
As a battery-conscious user	Idle measurement shows no persistent service and no repetitive wake schedule

6 Information architecture

The primary navigation is **Today**, **Library**, **Reminders**, and **Settings**. The central creation action opens a sheet offering Import media or Create reminder. Health and Backup are first-level settings destinations because reliability and portability are primary product concepts, not hidden utilities.



7 Business and portfolio requirements

The project has no monetization requirement for v1. It MAY accept open-source contributions and sponsorship later, but no monetization path may introduce tracking or degrade offline operation without a new product decision.

The public portfolio page SHOULD include:

- a privacy-safe product narrative based on the genuine problem;
- adaptive alert behavior diagram;
- architecture diagram and battery strategy;
- APK download linked to a signed GitHub release;
- source repository, release notes and SHA-256 checksum;
- screenshots using synthetic or owned media;
- a limitations section that accurately states Android dependencies.

8 Product metrics

Metrics are local or release-process measurements; no telemetry is uploaded.

Metric	MVP target
First reminder task completion in moderated usability test	At least 5 of 6 participants without facilitator correction
Successful locked/unlocked presentation decision	100% across required scripted scenarios
Alarm action availability without RN startup	100% in native instrumentation tests
Export/import logical and hash round trip	100% for supported fixtures
Critical accessibility task completion	100% with TalkBack test script
Idle persistent services	0
Open P0 defects at release	0

9 Dependencies and risks

The product depends on Android permission and notification policy, OEM behavior, local file codecs, and the user's device settings. The largest product risks are expectation mismatch around “exact,” misuse of full-screen intents, notification channels changed by the user, force-stop behavior, large backup archives, and native/JavaScript state divergence. Each has a dedicated mitigation in MR-06, MR-10 and MR-11.

10 Exit criteria

MVP exits development only when all P0 requirements are traceably tested, no data-loss path remains unresolved, adaptive presentation is verified on physical devices, exported archives restore after clean install, and idle battery evidence meets MR-15. A visually polished demo without these conditions is not an acceptable release.

10.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

User Experience and Interaction Specification

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-03
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define every primary screen, state, flow, action, message, transition and device-adaptive alarm interaction.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created

Term	Meaning
	when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Experience goals

The experience should feel calm, predictable and respectful. The app must be understandable without technical Android knowledge while still exposing the health of platform capabilities. It separates **content**, **schedule**, and **alert behavior** so users can reuse the same profile and media without duplicating settings.

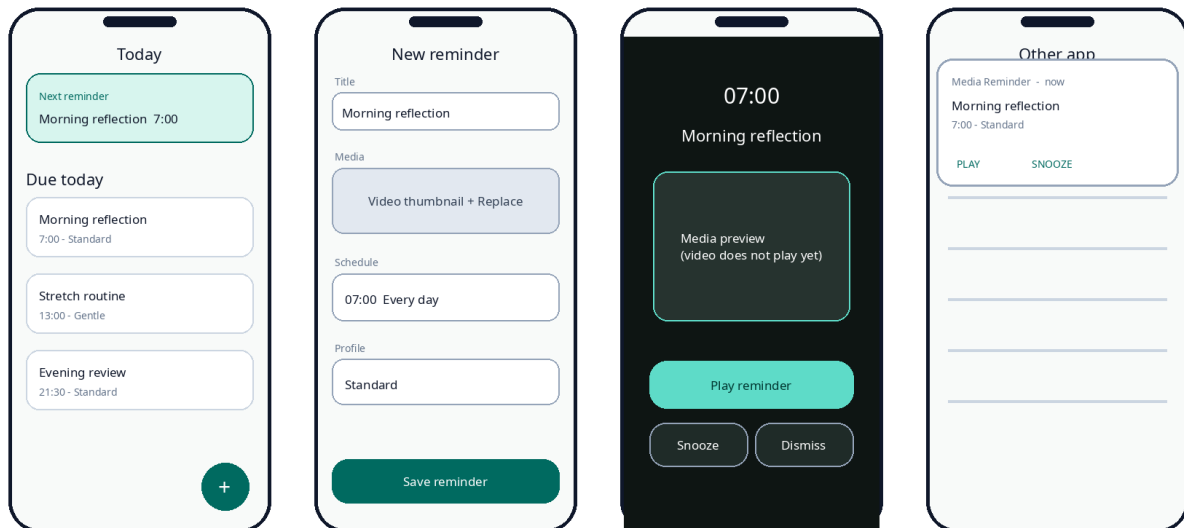
2 Navigation model

The bottom navigation contains four destinations:

- **Today** - next due reminders, recent outcomes and capability warnings that affect today;
- **Library** - imported media, categories, tags and storage use;
- **Reminders** - all schedules with enable state and next occurrence;
- **Settings** - Health, profiles, defaults, backup, appearance, accessibility, privacy and About.

A floating action button labeled **Add** opens a modal action sheet with **Import media**, **Create reminder**, and **Create text card** when text assets are enabled. Back navigation follows Android conventions; unsaved editors show a discard confirmation only when meaningful changes exist.

Media Reminder - Core Mobile Wireframes



System heads-up height is controlled by Android.

3 Onboarding

3.1 Page 1 - purpose

Title: **Make local media useful at the right moment**

Body: "Import videos, audio or images and schedule them as private reminders. Nothing is uploaded."

Primary: **Continue**

Secondary: **View privacy details**

3.2 Page 2 - adaptive behavior

Use a simple locked/unlocked illustration. Copy: "When your phone is locked, a reminder can open like an alarm. While you are using your phone, Android shows a compact notification instead." Avoid promising exact dimensions.

3.3 Page 3 - permissions by intent

Onboarding does not immediately request every permission. It explains that:

- notification permission is requested before creating the first active reminder;
- exact-alarm access is requested only after the user chooses exact timing;
- media access uses a system picker and does not grant the app the whole gallery;
- full-screen eligibility is checked when the user selects a profile that uses locked-screen alarm presentation.

Primary: **Start with my library.** The user can skip setup and explore with a demo text card.

4 Today screen

4.1 Normal state

Top area shows greeting-neutral copy: **Today** and date. A status chip shows **Ready**, **Action needed**, or **Limited timing**. The first card is **Next reminder** with content thumbnail, name, due time, repeat summary and actions: Play preview, Edit, More.

Below is a chronological list of today's occurrences. Each row has time, label, media icon, profile icon and state: Upcoming, Snoozed, Completed, Dismissed, Missed, Disabled or Needs setup. Completed and dismissed are factual, not celebratory or shaming.

4.2 Empty state

Title: **No reminders scheduled**

Body: "Import something meaningful, then choose when it should return."

Primary: **Import media**

Secondary: **Create reminder from a text card**

4.3 Capability warning

A single high-salience card appears only for a condition that affects active reminders. Example: **Exact timing is off**. Body explains the consequence. Primary opens Health. It never blocks browsing the library.

5 Library

The default presentation is a two-column grid on typical phones and adaptive columns on wider displays. Each card shows thumbnail, title, duration/type, number of active reminders and overflow menu.

Controls:

- search field with local title/category/tag search;
- filter chips for Videos, Audio, Images, Text and Missing;
- category chip row;
- sort by Recently added, Name, Most scheduled or File size;
- selection mode for batch category/tag/export/delete operations.

5.1 Import flow

1. User selects **Import media**.
2. App opens Photo Picker for supported visual media or the document picker for audio and general files.
3. A progress sheet reports Copying, Checking, Creating preview, and Ready.
4. The sheet can be dismissed only when background-safe copying is implemented; MVP keeps it modal with Cancel while the stream is open.
5. On success, the details editor opens with detected title and editable metadata.

Error copy states the cause and preserves user control:

- "This file type is not supported."
- "The file could not be read. It may have moved or be damaged."

- “Not enough free space. Free at least 420 MB and try again.”
- “Import was cancelled. No file was added.”

A duplicate hash prompts **Use existing**, **Import another copy**, or **Cancel** when P1 duplicate detection is active.

6 Media detail

The detail screen includes preview, title, type/duration/size, category, tags, notes, attached reminders and file integrity status. Primary action is **Add reminder**. Secondary actions: Play preview, Edit details, Export this item, Delete.

Deleting shows dependency-aware copy: “This media is used by 3 reminders.” Options are **Cancel**, **Keep reminders disabled**, and **Delete media and reminders**. The destructive button is last and colored as an error action.

7 Reminder editor

The editor is a single scrollable form with progressive disclosure:

1. **What** - selected media with Change action;
2. **When** - date/time and repeat choice;
3. **Alert style** - profile card with concise behavior summary;
4. **Snooze** - default duration and allowed custom range;
5. **Options** - label, notes, auto-timeout and history toggle;
6. **Preview** - “Next: Thursday, 6 August at 6:15 AM”; capability summary;
7. **Save reminder**.

The Save button is enabled only when data is structurally valid. A capability problem does not erase work. Instead Save creates the reminder disabled or Limited according to explicit user choice.

7.1 Repeat editor

MVP options: Once, Every day, Selected days. The weekday row uses localized first-day-of-week ordering. Date and time are separate controls. The screen reads the resulting rule in plain language.

For a local time that does not exist because of a daylight-saving transition, show: “2:30 AM does not occur on this date. The reminder will use 3:00 AM.” The user can accept or select another time. For an ambiguous repeated local time, default to the first occurrence and expose **Use second 1:30 AM** where the platform timezone supports it.

8 Profiles

8.1 Gentle

Best for low-urgency prompts. Unlocked: heads-up if Android permits, one short vibration, default notification sound off. Locked: notification on lock screen; no continuous ringing by default. Auto-timeout 60 seconds.

8.2 Standard

Unlocked: heads-up with sound and short vibration. Locked: full-screen eligible with repeating vibration and alarm sound. Auto-timeout 5 minutes.

8.3 Persistent

Unlocked: heads-up plus repeated notification escalation according to configured retry policy, but no full-screen takeover. Locked: full-screen eligible with continuous bounded alarm sound/vibration. Auto-timeout 10 minutes. The label explicitly says “Not for emergencies.”

Profile summaries state behavior rather than vague urgency words. Changes to a shared profile show the number of reminders affected.

9 Due reminder presentation

9.1 Decision

At trigger, native code samples current state once and chooses the least intrusive sufficient surface. Unknown state fails toward nonintrusive notification behavior.

State	Surface	Sound/vibration	Notes
Device locked or keyguard showing	Full-screen alarm activity when eligible	Profile-controlled bounded alarm	May wake screen; Android may still show a notification fallback
Screen non-interactive	Full-screen alarm activity when eligible	Profile-controlled bounded alarm	Same as locked path
Unlocked and interactive	System heads-up notification	Notification or profile-controlled short alert	Never launch full screen over current app
Media Reminder foreground	In-app top strip plus notification record	Profile-controlled; generally reduced duplication	Strip height is at most $\min(144\text{dp}, 20\% \text{ usable height})$
State cannot be determined	Heads-up/notification fallback	Standard channel behavior	Safer than surprise takeover

Android and OEMs control whether and how a heads-up notification appears. The design target is compact, but the app does not control a precise 0.2-screen height.

9.2 Locked-screen alarm surface

The native alarm activity uses an edge-to-edge darkened backdrop derived from a blurred/solid media theme, title, due time and three large actions:

- **Play** - stops alarm sound, marks accepted, opens native media player and begins attached media;
- **Snooze** - opens bottom sheet of configured durations; hardware volume keys may silence sound but do not dismiss;
- **Dismiss** - stops alert and records dismissed.

Media is not visible as an autoplaying video behind the controls. A static thumbnail may be shown. Controls remain reachable at 200% font scaling, in landscape and with TalkBack.

9.3 Unlocked heads-up

Notification title: reminder label. Text: due time and media name. Actions: Play, Snooze, Dismiss. A thumbnail MAY appear if it does not displace actions. The app never asks the user to “accept the alarm”; **Play** is clearer and describes the result.

Ignoring a heads-up leaves it in the notification shade. Gentle auto-times out. Standard records Missed after timeout. Persistent may issue bounded re-alerts according to profile, but never opens full screen while unlocked.

9.4 In-app strip

The top strip is nonmodal and pushes content only when necessary. It shows title, time, Play and Snooze. Dismiss is in overflow on small widths but remains a direct action for accessibility focus order. Swiping upward collapses it to a status chip; swiping sideways does not silently dismiss.

10 Snooze interaction

Default choices are 5, 10, 15, 30 and 60 minutes, customizable globally and per reminder. Custom input is constrained to 1 minute through 24 hours in MVP. The confirmation announces the absolute next time: **Snoozed until 6:25 AM**, not merely “10 minutes.” Multiple taps are idempotent.

A snoozed occurrence is a child occurrence tied to the original due event. It does not shift the base recurring schedule unless the user selects **Move schedule** from the reminder editor.

11 Missed and overlapping reminders

When the device becomes available after an occurrence was missed:

- occurrences less than the profile grace window old may alert once;
- older occurrences appear as Missed in Today without a surprise full-screen alert;
- if several are due together, one active alarm session is shown and others are queued in chronological order;
- after action, the next queued reminder is offered as a card, not stacked full-screen activities.

12 Health screen

Health is a dashboard, not a permission dump. Rows include Notifications, Exact timing, Locked-screen alarm, Notification channels, Battery/OEM notes and Last scheduler check. Each row shows Ready, Limited or Action needed, a plain-language effect and one action.

The app **MUST** not pretend it can read every OEM policy. Where behavior is unknowable, copy says: “Your device manufacturer may delay background alerts. Use Test reminder after changing battery settings.”

The **Test reminder** control schedules a native test 15 seconds ahead and asks the user to lock or keep the phone unlocked depending on selected scenario. It is visibly labeled as a test and does not write completion history.

13 Backup UX

13.1 Export

Screen summarizes asset count, reminders, estimated size, archive privacy and destination choice.

Warning: “This ZIP contains your media and reminder names. Anyone with the file can open it.” Primary: **Choose where to save**. Progress shows files completed and bytes written. On success: filename, size, hash, **Share**, **Done**.

13.2 Import

1. Choose ZIP using document picker.
2. **Inspecting backup** with cancel.
3. Preview version, export date, counts, size, checksum status and incompatibilities.
4. Choose Inspect only, Merge or Replace.
5. Review conflict plan.
6. Confirm. Replace requires typing REPLACE or a second explicit confirmation plus rollback explanation.
7. Commit, then show restored counts and required setup steps.

No reminders are silently activated until Android capabilities are checked on the new device.

14 Copy style

Use concise, respectful language. Avoid “failure” when a setting is simply disabled; say what happened and what the person can do. Avoid religious claims in generic product copy. Islamic example content may use accurate names supplied by the user, but the app itself does not evaluate religious correctness.

Preferred: “Exact timing is off. Android may deliver reminders later.”

Avoid: “Your alarm is broken.”

Preferred: “Dismiss”

Avoid: “Skip your dua.”

15 Motion and feedback

Navigation uses Material motion with 200-300 ms transitions. Alarm and notification actions respond immediately with haptic confirmation where allowed. Reduced-motion mode changes movement to crossfades or instant state changes. Import progress is determinate when byte length is known and otherwise uses a labeled indeterminate indicator.

16 Error and recovery pattern

Every error view contains:

1. a human title;
2. one-sentence effect;
3. preserved-state statement when relevant;
4. primary recovery action;
5. optional technical details behind **Details**;

6. diagnostic code that contains no file name or personal label.

Example: **Could not finish import.** "The selected file became unavailable while it was being copied. No library item was created." Actions: **Choose file again, Cancel.** Details: MR-IMP-READ-004.

16.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Visual Design System

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-04
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define brand attributes, color, type, spacing, shape, components, responsive behavior, iconography, motion and visual accessibility.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

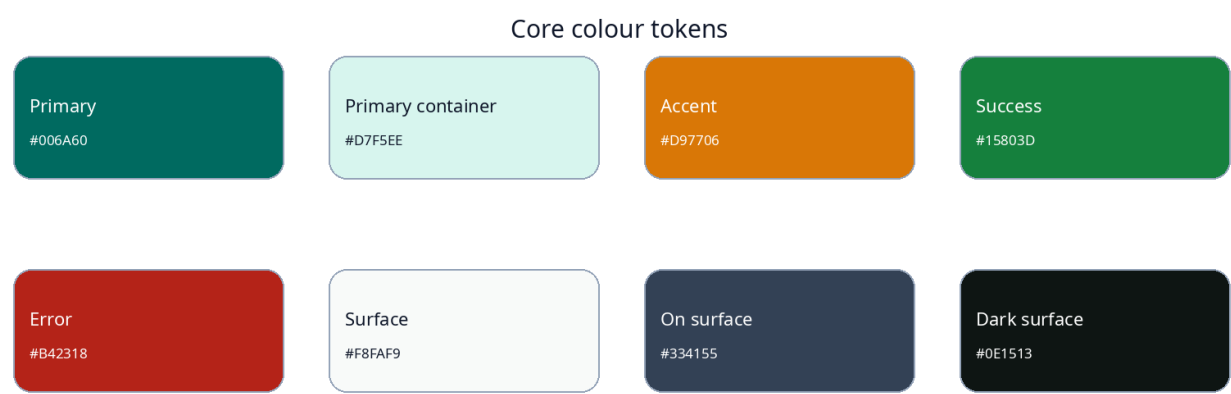
The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Design intent

Media Reminder should look like a quiet, modern utility rather than a loud alarm clock or generic social-media clone. The three brand attributes are **calm**, **intentional** and **dependable**. Material 3 provides the interaction foundation, while the product uses a restrained teal system, warm amber accents and generous spacing.



2 Color system

All color pairings **MUST** meet the contrast requirements in MR-13. Dynamic color **MAY** be offered later but the fixed brand palette remains the default for consistent screenshots and alarm recognition.

Token	Light	Dark	Use
Primary	#006A60	#5EDBC8	Primary actions, selected state, key status

Token	Light	Dark	Use
On primary	#FFFFFF	#003732	Content on primary
Primary container	#D7F5EE	#005047	Low-emphasis selected cards
On primary container	#00201C	#D7F5EE	Text/icons on container
Secondary	#D97706	#FFB951	Snooze, warm attention, due-soon indicator
Surface	#F8FAF9	#0E1513	App background
Surface container	#EEF2F0	#17201D	Cards, sheets, navigation
On surface	#171D1B	#DEE4E1	Primary content
On surface variant	#3F4946	#BEC9C5	Secondary content
Outline	#6F7975	#89938F	Dividers, controls
Error	#B42318	#FFB4AB	Destructive actions, blocking errors
Success	#15803D	#68D391	Completed/ healthy state, always with icon/text
Scrim	#000000 at 48%	#000000 at 64%	Modal and alarm backdrop

The alarm surface uses dark tonal surfaces even when the app theme is light, reducing glare on a woken screen. Controls retain full contrast. Red is reserved for destructive action or blocking fault, not normal urgency.

3 Typography

Use the Android system sans family or Noto Sans where bundled size and license are acceptable. Arabic localization uses Noto Sans Arabic or platform equivalent after glyph QA. Avoid custom display fonts that increase APK size or impair script support.

Role	Size/line height	Weight	Typical use
Display small	36/44 sp	500	Locked-screen due time on large devices
Headline large	32/40 sp	500	Alarm title
Headline medium	28/36 sp	500	Screen title on expanded layouts
Title large	22/28 sp	600	Top app bar, major card

Role	Size/line height	Weight	Typical use
Title medium	16/24 sp	600	List/card title
Body large	16/24 sp	400	Primary body copy
Body medium	14/20 sp	400	Supporting copy
Label large	14/20 sp	600	Buttons and tabs
Label medium	12/16 sp	600	Chips and metadata

Use tabular figures for times where available. Do not disable system font scaling. At 200%, action labels may wrap to two lines; critical controls MUST not truncate to ambiguous text.

4 Spacing and grid

The base unit is 4 dp. Common increments: 4, 8, 12, 16, 20, 24, 32 and 40 dp. Phone screen side padding is 16 dp; compact dialog padding is 24 dp; expanded layouts use a centered content max-width of 840 dp.

- Minimum touch target: 48 x 48 dp.
- Alarm primary actions: minimum 56 dp height and 64 dp preferred.
- List row: minimum 64 dp, expanding with font scale.
- Card internal padding: 16 dp.
- Section gap: 24 dp.
- Bottom navigation height: platform Material baseline plus system insets.

5 Shape and elevation

Component	Corner radius	Elevation
Small chip	8 dp	0
Text field / compact control	12 dp	0
Standard card	16 dp	0-1
Dialog	24 dp	3
Bottom sheet	28 dp top corners	3
In-app due strip	20 dp bottom corners	4
Alarm action button	20 dp	1

Do not use shadows as the only boundary in dark mode. Combine tonal surface, outline and elevation as needed.

6 Iconography

Use Material Symbols Rounded with a consistent optical size. Every icon-only button requires an accessibility label and tooltip where supported. Core concepts:

- Play: filled play arrow;
- Snooze: alarm with plus or schedule icon plus text;

- Dismiss: close icon only when paired with visible `Dismiss` on alarm surfaces;
- Health: shield/check;
- Backup: archive/download-upload directional icon;
- Profile: tune or notifications active;
- Missing media: broken image/file warning.

Religious iconography is optional and attached to user categories; the global brand does not assume a religion. Avoid decorative crescents as generic button symbols because they can be mistaken for dark mode or sleep.

7 Component catalog

7.1 Primary button

Filled primary color, verb-first label, optional leading icon. One primary action per surface. Loading replaces icon with progress but preserves width. Disabled state retains readable label and explains validation near the relevant field.

7.2 Secondary and tonal buttons

Outlined or tonal. Snooze on alarm surfaces uses warm secondary tonal treatment, while Play is primary and Dismiss is neutral/destructive depending on context.

7.3 Reminder card

Thumbnail 64 x 64 dp; title; next time; repeat summary; profile glyph; enabled switch with accessible state. Tapping body opens details; switch changes enable state but never propagates tap to the card.

7.4 Capability card

Status icon + Ready, Limited, OR Action needed; title; one-sentence consequence; trailing action. Avoid a dashboard filled with green checks; healthy items can collapse.

7.5 Media card

Aspect ratio 16:9 for video, square treatment for image/audio fallback. Duration/type label sits on a high-contrast scrim. Broken preview uses icon and text, never an empty gray rectangle.

7.6 In-app reminder strip

Maximum height is $\min(144dp, 20\% \text{ of usable viewport})$. Content scroll is not allowed within the strip. At high font scale it transitions to a compact card with Play and Snooze, while Dismiss moves to overflow but remains reachable in sequential focus.

7.7 Full-screen alarm

High-contrast static background, safe inset handling, time, label, optional thumbnail, profile status and three actions. No tiny swipe-only affordance. Time and title are announced once; continuous TalkBack focus stealing is forbidden.

8 Responsive behavior

Width class	Layout
Compact	Bottom navigation, single-column editors, two-column media grid when

Width class	Layout
	space permits
Medium	Navigation rail, two-pane Library detail, widened alarm controls
Expanded	Navigation rail/drawer, centered max-width content, three-pane optional management layout

Foldables MUST respond to hinge and posture without placing primary alarm controls across an occluding hinge. Rotation during an active alarm rebuilds the native activity from session state without restarting sound.

9 Insets and system UI

All screens honor status, navigation, cutout and gesture insets. Full-screen alarm MAY draw behind system bars but action controls remain inside safe areas. The app does not hide navigation gestures in a way that traps the user. System bar icon contrast follows the actual surface luminance.

10 Motion

Motion	Duration	Easing	Reduced motion
Navigation transition	250 ms	standard	Crossfade 100 ms or instant
Card expand	200 ms	emphasized decelerate	Instant size change
In-app strip enter	220 ms	emphasized decelerate	Fade only
Snackbar	Material default	standard	Fade only
Alarm button confirmation	120 ms	standard	Haptic/color only

No looping decorative animations. Vibration is functional feedback and follows profile and accessibility settings. Visual urgency MUST not rely on flashing.

11 States

Every interactive component specifies default, pressed, focused, hovered where applicable, disabled, loading and error states. Keyboard/switch focus uses a visible 2 dp primary outline with adequate contrast. Pressed-state overlays MUST not reduce text contrast below threshold.

12 Charts and history

Local history uses simple counts and accessible summaries rather than competitive streak visuals. A chart must have a textual equivalent. Completed, dismissed and missed use icon + label + color; no pie chart without values.

13 Brand assets

The initial icon direction is a rounded play triangle inside a subtle alarm-ring form, avoiding a literal bell-heavy or social-app appearance. Required exports: adaptive foreground/background, monochrome themed icon, 512 px store icon, SVG master and high-contrast small-size test. The logo MUST remain legible at 24 px and MUST not contain Arabic scripture or other sacred text that could appear in inappropriate system contexts.

14 Content density and writing

Screen titles use sentence case. Buttons use short verbs. Technical identifiers live behind Details. Times use the user's 12/24-hour preference. Dates use locale formatting. Avoid all-caps except generated archive codes. Keep snackbar text to one action and approximately two lines at normal scale.

15 Visual QA checklist

Before release, verify light/dark/system themes, contrast, 200% font scale, smallest supported width, landscape, cutouts, gesture navigation, RTL mirrored layouts, selected/unselected states, long labels and missing thumbnails. The locked-screen alarm must be reviewed in a dark room for glare and on OLED/LCD devices for readability.

15.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Functional Feature Specification

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-05
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define feature behavior, validation, state transitions and acceptance rules independent of UI implementation details.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Feature model

A reminder is a composition rather than a copied bundle:

Reminder = Media asset + Schedule rule + Reminder profile + Snooze policy + Label metadata

A media asset can have many reminders. A shared profile can have many reminders. An occurrence is derived from a reminder, and an active alarm session is derived from one occurrence at a time.

2 Media import and ownership

2.1 Supported MVP types

Kind	Accepted source	Validation	Playback
Video	Photo Picker or document picker	MIME sniff, extension advisory, size, readable stream, Media3 probe	Native Media3 player
Audio	Document picker	MIME sniff, size, readable stream, Media3 probe	Native Media3 player
Image	Photo Picker or document picker	Decode bounds and image header	Native image viewer
Text	App editor, P1 if deferred	UTF-8 length and safe rendering	Native/React Native text card

The allowlist is finalized by instrumented codec tests. File extensions MUST NOT be trusted as the sole type check.

2.2 Import transaction

1. Create a `pending_file_operations` record and random temporary filename.
2. Open the content URI through `ContentResolver`.
3. Stream into app-specific temporary storage with a bounded buffer while calculating SHA-256.
4. Abort if declared and actual size exceeds product limits or storage reserve.
5. Probe media and extract safe metadata.
6. `fsync` as appropriate, then atomically rename into `files/media/<uuid>.<ext>`.
7. Insert the media record and complete the pending operation in one logical commit.
8. Generate thumbnail as derived cache; thumbnail failure does not invalidate the source asset.

A crash leaves either no committed media row or a recoverable pending operation. Startup repair removes expired temporary files and reconciles operations.

2.3 Asset deletion

Deletion is two-phase: mark pending-delete and disable/deal with dependencies in a transaction, delete physical file, then finalize the row. If file deletion fails, keep a repair record and hide the asset from normal views only after the user-selected dependency policy is durable.

3 Library management

- **FUN-001:** Search MUST match normalized title, notes, category and tags locally.
- **FUN-002:** Filters and sort order MUST survive navigation and MAY persist as user preference.
- **FUN-003:** Categories are single-valued per asset in MVP; tags are many-to-many.
- **FUN-004:** Renaming an asset changes metadata only, never its internal filename.
- **FUN-005:** File integrity check recalculates hash on demand and marks Missing, Changed or Healthy.
- **FUN-006:** A missing asset disables attached reminders and offers Locate replacement, Delete, or Restore from backup.

4 Reminder lifecycle

States:

- Draft - editor not committed;
- Disabled - persisted but not considered by scheduler;
- Needs setup - enabled intent exists but a required platform capability is missing;
- Active - eligible and has a calculable next occurrence;
- Snoozed - one child occurrence is scheduled while base rule remains unchanged;
- Archived - hidden from default lists, retained for history;
- Deleted - tombstoned only during backup/import transaction, then removed under retention policy.

Enabling runs validation in this order: media availability, schedule validity, profile validity, notification capability, exact-alarm mode decision, full-screen eligibility if requested. The UI MUST report all applicable issues, not stop at the first.

5 Schedule rules

5.1 Once

Stores a timezone-aware instant plus the originating zone and local representation for display. Once fired and resolved, it becomes inactive unless snoozed.

5.2 Daily

Stores local wall-clock time and time zone behavior. Default policy is **follow current device timezone**, suitable for a person traveling. An advanced fixed-zone option is P1. Calculation always returns the first valid future occurrence after the scheduling cursor.

5.3 Selected weekdays

Stores a 7-bit weekday mask and local wall-clock time. Locale affects display order only; persisted weekday identity is ISO Monday=1 through Sunday=7.

5.4 Recurrence invariants

- The next occurrence **MUST** be strictly later than the calculation cursor.
- A resolved occurrence **MUST NOT** be regenerated with the same occurrence key.
- Snooze **MUST NOT** mutate the recurrence rule.
- Clock rollback **MUST** not replay an already resolved occurrence inside the idempotency horizon.
- Timezone change recalculates wall-clock schedules; once schedules remain tied to their instant.

6 Profiles

A profile contains notification behavior, sound source reference, vibration pattern, locked-screen full-screen eligibility, timeout, retry policy, grace window and default snooze. Built-in profile identifiers are stable. Users can edit safe values but cannot delete a built-in profile; they may reset it.

Sound sources:

- built-in packaged alarm tone;
- Android default alarm tone resolved per device;
- user-selected document URI only when persistable access is valid;
- no media video's audio before Play.

A broken custom tone falls back to packaged tone and logs a local diagnostic event.

7 Alert actions

7.1 Play

Atomic effects:

1. acquire action idempotency key;
2. stop ringtone/vibration and release wake resources;
3. mark occurrence Accepted and active session TransitioningToMedia;
4. launch or reveal the native media player with asset UUID;
5. start media only after the activity is visible and audio focus is granted;

6. calculate next occurrence and reschedule global alarm;
7. mark session MediaPlaying.

If media is missing, stop alert first, show a clear error, mark outcome AcceptedMediaUnavailable and offer to open Library. Never continue ringing because playback failed.

7.2 Snooze

Stops active alert, validates duration, creates one snoozed occurrence with a new occurrence UUID and parent pointer, recalculates the globally earliest event, then confirms absolute time. Multiple identical intents with the same session/action nonce return the original result.

7.3 Dismiss

Stops alert, records Dismissed, clears active session and recalculates. Dismiss does not disable future recurring occurrences.

7.4 Timeout

Stops all active resources. The occurrence becomes Missed unless a profile explicitly records TimedOut. Persistent retry logic is bounded by retry count and absolute session lifetime.

8 Notification behavior

Notification channels are stable and user-owned once created:

- `reminder_gentle_v1` - default importance, no full-screen intent;
- `reminder_standard_v1` - high importance, sound/vibration defaults;
- `reminder_persistent_v1` - high importance, alarm-oriented defaults;
- `reminder_status_v1` - low importance for background operation status, used only when legally required;
- `reminder_errors_v1` - default importance for import/export completion or repair, not alarm delivery.

Because channel sound/importance cannot be silently overridden after creation, profile screens must distinguish app preference from channel-effective behavior and link to Android channel settings.

9 Foreground in-app behavior

When the app is foregrounded, the same native due event is persisted and posted as a notification for system history/action resilience. React Native receives an event and shows an in-app strip. If the event bridge fails, the notification remains sufficient. The strip and notification share an action nonce so double taps cannot produce duplicate snoozes.

10 Reminder collision policy

Only one active alarm session owns sound/vibration. Due occurrences are placed in a native queue ordered by scheduled instant then stable UUID. The visible activity shows “1 of N” when more exist. Dismissing one advances to the next; **Dismiss all due** is available behind an explicit overflow action. No overlapping services play competing audio.

11 History

MVP history is optional and local. It stores scheduled time, actual trigger time, action and completion time; it does not store media frames or content. Default retention is 90 days. The user can clear history without deleting reminders. No streak language, rankings or remote analytics.

12 Categories and tags

Category names are unique under Unicode case-folded comparison within a user profile. Deleting a category moves assets to Uncategorized. Tags are reusable and deletion removes relationships only. Import conflict handling is specified in MR-10.

13 Settings

Settings groups:

- Reminder defaults: default profile, snooze choices, timeout;
- Appearance: theme, thumbnail autoplay off, 12/24-hour follows system;
- Accessibility: reduce motion follows system, stronger haptics toggle where supported, announce due time;
- Storage: usage, orphan repair, clear derived cache;
- Health: permissions and test reminders;
- Backup: export/import and backup warning;
- Privacy: local-only statement, history retention, diagnostic export;
- About: version, licenses, source, limitations.

14 Diagnostics

A local diagnostic ring buffer stores technical event codes, timestamps, app version, OS version, device manufacturer/model, permission state snapshots and timing deltas. It MUST NOT include media bytes, titles, notes, category names, file names, exact reminder labels or selected paths. Export is user-initiated JSON and previewable before sharing.

15 Test reminder

The Health screen can schedule a one-shot test 15 seconds ahead. It uses a synthetic label, packaged tone and no personal asset. The user selects Locked test or Unlocked test and receives instructions. Test events never alter history, streaks or recurring schedules and are clearly distinguishable in notification IDs.

16 Import/export features

Export supports all data or selected assets/reminders. The all-data path is P0; selection export MAY be P1. Share is invoked only after archive closure and checksum calculation using FileProvider read-only URI grants. Import never trusts internal absolute paths or filenames.

17 Feature acceptance invariants

- Every user action that stops an alert is native-first and idempotent.
 - Every file operation is staged and recoverable.
 - Every schedule change triggers deterministic next-event recalculation.
 - Every destructive action identifies dependent data.
 - Every system-owned capability is displayed as observed state, not silently assumed.
 - Every background component has a bounded lifetime and documented reason.
 - Every archive is fully validated before modifying current data.
-

17.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Android Alarm, Notification and Battery Specification

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-06
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define the native Android execution model, permissions, scheduling, adaptive presentation, ringing lifecycle, reboot recovery and battery controls.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.

Term	Meaning
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Scope and safety position

This specification governs all behavior that must remain reliable when the React Native process is absent, slow or crashed. Alarm dispatch, action handling, alert stopping, next-event scheduling and active-session recovery are native Kotlin responsibilities.

Media Reminder is an alarm/reminder application, but it is not a safety-of-life service. Exact timing means the strongest user-authorized Android mechanism available; it does not mean delivery through device shutdown, force-stop, disabled notifications, revoked access or every OEM modification.

2 Android baseline

Item	v1 baseline
Minimum Android	Android 8.0 / API 26
Compile SDK	API 37 / Android 17
Initial target SDK	API 36 / Android 16
UI framework	React Native 0.86.x
Native language	Kotlin
Database	Room
Media	AndroidX Media3
Alarm scheduler	AlarmManager one-shot exact alarm when authorized
General background work	WorkManager only for deferrable repair/maintenance, never for exact due delivery

The target SDK decision is revisited before every release against current Play and direct-distribution requirements. Compilation against API 37 permits current compatibility testing without claiming all API 37 behavior is targeted in v1.

3 Permission policy

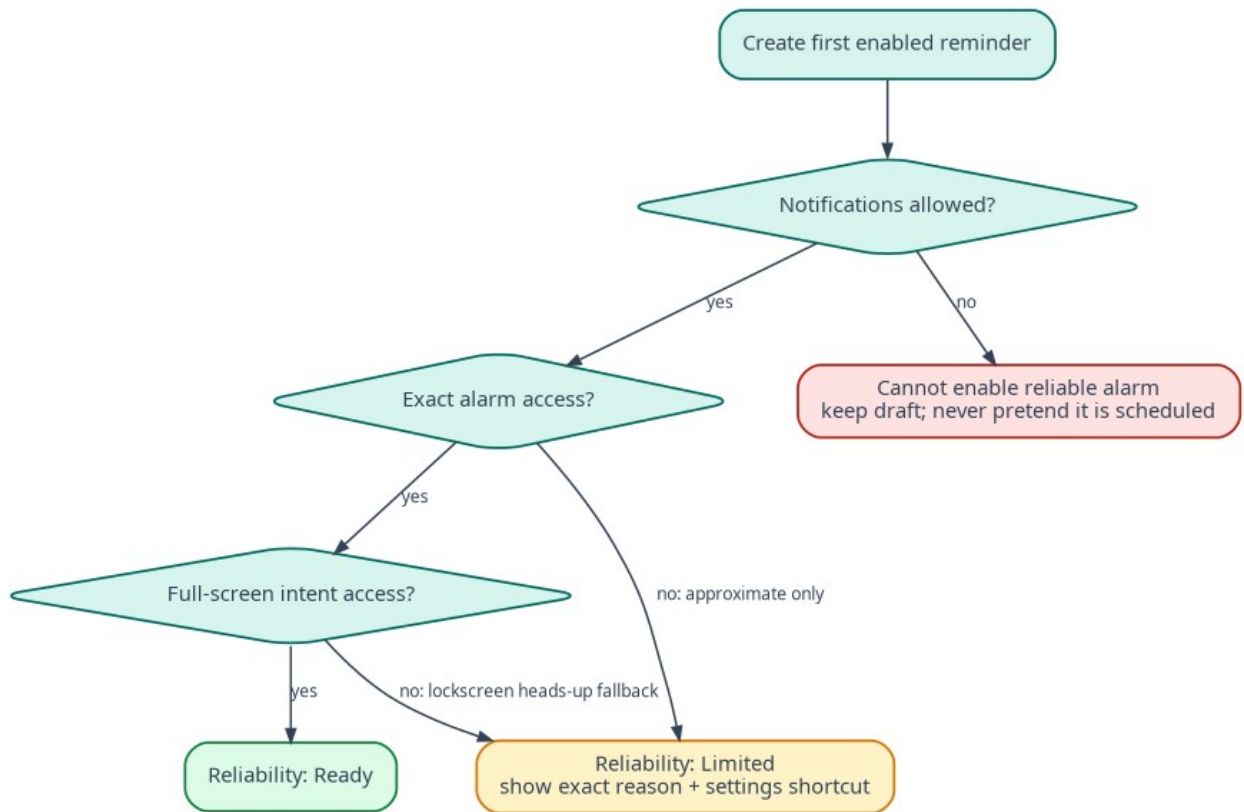
3.1 Manifest declarations

Permission/capability	Reason	Request policy
POST_NOTIFICATIONS	Show due alerts on Android 13+	Runtime request immediately before first active reminder or Test reminder
SCHEDULE_EXACT_ALARM	User-authorized exact alarms	Special-access education after user selects exact timing; never at cold launch
USE_FULL_SCREEN_INTENT	Locked/non-interactive alarm surface where eligible	Manifest declaration; check effective eligibility on API 34+
RECEIVE_BOOT_COMPLETED	Reconcile schedules after reboot	Manifest; no user dialog
VIBRATE	Profile vibration	Manifest; user profile controls behavior
FOREGROUND_SERVICE	Bounded active ringing service	Manifest; service exists only during active alert
FOREGROUND_SERVICE_MEDIA_PLAYBACK	Active audible alarm/media playback on modern Android	Manifest and <code>mediaPlayback</code> service type
WAKE_LOCK	Bounded wake bridge/active alert only	Manifest; every acquisition has explicit timeout

3.2 Explicitly excluded

The production manifest **MUST NOT** declare `INTERNET`, `SYSTEM_ALERT_WINDOW`, broad `READ_MEDIA_*` gallery access merely for import, `REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`, `ACCESS_NOTIFICATION_POLICY`, `READ_PHONE_STATE`, location, contacts, microphone or camera.

`USE_EXACT_ALARM` is intentionally not used in v1 because it is intended for narrowly qualifying app categories and carries distribution implications. The user-controlled `SCHEDULE_EXACT_ALARM` path is more transparent for an open-source general media reminder. This decision is reviewed if store policy establishes a better qualified route.



4 Capability state machine

The app maintains observed capability states, never a single “permissions granted” Boolean.

Capability	Ready	Limited	Blocked
Notifications	Runtime permission granted and channel not blocked	Permission granted but effective channel below high importance	Runtime permission denied or all relevant channel blocked
Exact timing	<code>canScheduleExactAlarms()</code> true	User selected inexact Limited mode	Access false and user declined Limited mode
Full-screen locked alert	Manifest permission effective and platform permits FSI	Notification fallback available	Notifications blocked
Media	App-owned asset readable	Asset missing but reminder retained disabled	Storage corruption or unresolved operation
Scheduler	Next event persisted and OS alarm registered	Reconciliation pending	No eligible reminder or unrecoverable

Capability	Ready	Limited	Blocked
			database fault

Health MUST show both app preference and observed Android behavior. Notification channel settings are user-owned after creation; app profile edits cannot silently raise a channel the user lowered.

5 Scheduling architecture

5.1 One global next alarm

At any stable point, the application schedules at most one ordinary next-due `PendingIntent` for reminders, plus explicitly bounded test or maintenance alarms. It does not call `setRepeating()`.

Algorithm:

1. In one database transaction, resolve stale active sessions and query the earliest eligible occurrence.
2. Persist `scheduler_state` with occurrence UUID, due instant, generation and pending-intent request code.
3. Cancel the previously registered pending intent when identity changed.
4. If exact access is available, call `AlarmManager.setAlarmClock()` with the occurrence trigger and a show-intent to the Reminders screen. This makes alarm intent visible to system surfaces and is appropriate for user-visible alarm functionality.
5. If exact access is unavailable and the user accepted Limited mode, call `setAndAllowWhileIdle()` or the best compliant inexact mechanism, label the reminder Limited, and do not claim exactness.
6. If no eligible event exists, cancel the previous alarm and clear scheduler state.

Every operation uses immutable/update-current `PendingIntent` flags appropriate to API level. Identity is based on a stable scheduler slot, while occurrence identity is authenticated in extras and verified against Room.

5.2 Why not WorkManager

WorkManager is used only for deferrable tasks such as orphan-cache cleanup, optional integrity scanning and retrying a nonurgent export finalization. It MUST NOT deliver scheduled reminders because its execution window is not an exact alarm contract.

5.3 Recalculation triggers

Recalculate after reminder create/edit/enable/disable/delete, profile change that affects eligibility, snooze, dismiss, play, timeout, asset availability change, import commit, exact-alarm access change, notification permission change, boot, package replacement, timezone change, significant clock change and startup reconciliation.

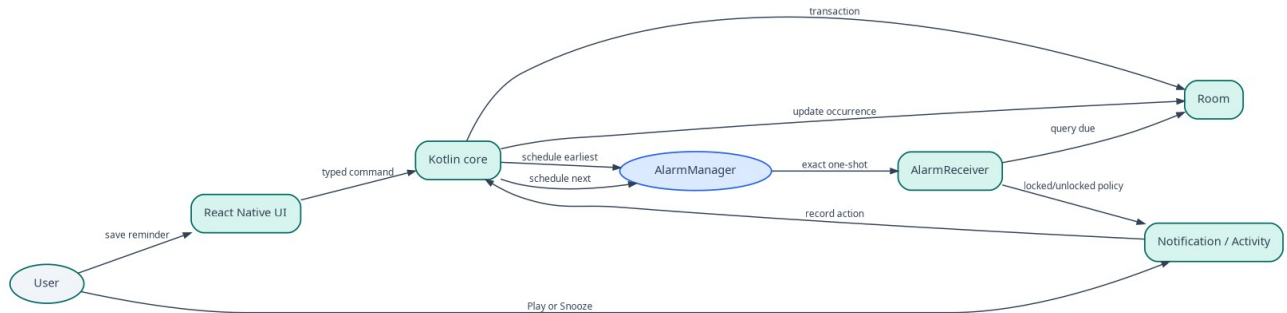
6 Alarm dispatch

`AlarmDispatchReceiver` is a small manifest receiver implemented in Kotlin. It MUST:

1. call `goAsync()` and finish quickly;
2. load scheduler state and occurrence in Room;
3. reject stale generation, mismatched UUID or already-resolved occurrence;
4. atomically claim the occurrence using an idempotency row;

5. create or recover one `active_alarm_session`;
6. calculate device presentation state;
7. post the notification and, if appropriate, start the bounded ringing service and full-screen path;
8. schedule the next occurrence before completing, or queue reconciliation;
9. release its short wake bridge and finish.

Target receiver-to-visible-alert latency is under 500 ms p95 on reference devices and must never intentionally wait for the React Native bundle.



7 Adaptive presentation decision

Native code samples:

- `PowerManager.isInteractive`;
- `KeyguardManager.isDeviceLocked` and keyguard visibility;
- whether Media Reminder has a resumed foreground activity;
- notification permission/channel status;
- full-screen eligibility on supported APIs.

Decision rules:

1. If the device is locked or non-interactive, profile permits locked alarm, notifications are usable and full-screen intent is eligible, post a high-importance alarm notification with a full-screen intent to `AlarmActivity` and start bounded ringing.
2. If the device is locked/non-interactive but full-screen intent is not eligible, post the highest compliant lock-screen notification, start only behavior allowed by the effective channel/service policy, and record Limited FSI state.
3. If the device is unlocked and interactive, post a high-importance notification **without** a full-screen intent. Android may present it as heads-up. Never launch `AlarmActivity` directly.
4. If the app is foreground, additionally emit an event to show the in-app strip. The system notification remains the fallback.
5. If state is uncertain, choose the unlocked notification path.

Android controls heads-up dimensions, duration, ranking and whether it appears. The app can design notification content and channel importance but cannot guarantee a 20%-height dropdown over another app.

8 Full-screen alarm activity

`AlarmActivity` is native, exported false, noHistory, excludeFromRecents and backed by active session ID. It uses `setShowWhenLocked(true)` and `setTurnScreenOn(true)` on supported APIs. It does not dismiss keyguard without explicit platform-authorized user flow.

The activity displays static reminder context and native Play, Snooze and Dismiss controls. It **MUST** handle configuration change, process recreation and duplicate intents. Closing the activity through Back is mapped to a profile-defined safe outcome; default is collapse to the notification while ringing continues for Standard/Persistent and stop for Gentle. The UI makes this explicit.

No media autoplays in the alarm activity. Play first stops alarm sound, then opens `MediaPlayerActivity`. This avoids competing alarm/media audio and respects the user's action.

9 Notification construction

Notification categories use `CATEGORY_ALARM` for Standard/Persistent due alerts and `CATEGORY_REMINDER` for Gentle. Visibility defaults to private; lock-screen content follows the user's **Show reminder names on lock screen** setting. Secret visibility is available.

Actions are native broadcast pending intents:

- `ACTION_PLAY(sessionId, nonce);`
- `ACTION_SNOOZE_DEFAULT(sessionId, nonce);`
- `ACTION_DISMISS(sessionId, nonce).`

A content intent opens the due item or media player without resolving it. Delete intent from notification shade maps to profile-defined Ignore/timeout, not Dismiss, and is recorded distinctly.

The notification is ongoing only during active ringing when necessary to prevent accidental clearing. It becomes removable after sound stops. Notification IDs are derived from stable session hashes with collision tests.

10 Notification channels

Channels are versioned because Android channel importance and sound become user-controlled after creation. New defaults require a new channel version and a transparent migration prompt; old channels are retained while referenced.

The Health screen opens the relevant system channel settings and displays:

- expected app profile;
- effective importance where readable;
- whether sound/vibration is controlled by Android;
- test action.

The app **MUST NOT** delete and recreate a channel to override a user's choice.

11 Ringing service

`AlarmRingingService` is a native foreground service started only for a claimed active alarm session that requires continuous sound or vibration. It declares media playback type and immediately posts the session notification before the platform deadline.

Responsibilities:

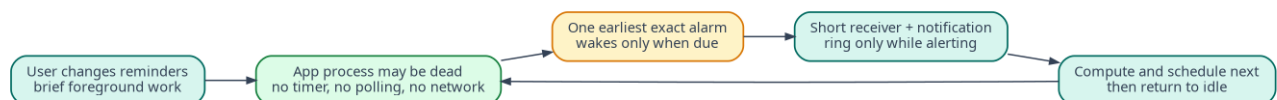
- play packaged/default/custom tone with `AudioAttributes.USAGE_ALARM`;
- apply a bounded vibration waveform;
- obtain audio focus where appropriate and respond to focus loss;
- maintain a session-scoped partial wake lock only while actively ringing, with an absolute timeout;
- stop within one second of Play, Snooze, Dismiss, timeout or invalidated session;
- recover its session after service recreation from Room, or stop if no valid session exists.

The service MUST NOT be sticky when no valid session remains. Maximum active lifetime is profile timeout, hard-capped at 10 minutes in v1. Persistent retry is implemented as future one-shot alarms, not an endlessly running service.

12 Audio behavior

- Alarm audio never uses the attached media file before Play.
- The app respects silent/DND/channel behavior as controlled by Android; it does not request DND policy access in v1.
- During a detected communication audio mode, default behavior is to reduce to notification sound/vibration rather than overpower a call. No phone-state permission is requested.
- Hardware volume keys alter the system stream according to Android; the alarm UI also has a visible **Silence sound** action in accessibility overflow.
- Bluetooth/headphone routing follows system alarm routing. The app does not secretly force speaker output.
- When Play begins, alarm sound stops before Media3 requests media audio focus.

13 Wake and battery behavior



Idle expectations:

- process may be dead;
- no service is resident;
- no wake lock is held;
- no JavaScript timer runs;
- one next alarm is registered.

Wake resources:

- receiver bridge wake lock: maximum 10 seconds and released in `finally`;
- active alert wake lock: session-scoped and hard-capped to alarm timeout;
- no wake lock during normal media browsing once activity lifecycle is sufficient;
- every acquisition is tagged and covered by unit/static tests.

No battery optimization exemption request is part of onboarding. OEM troubleshooting is optional, manufacturer-specific guidance is clearly labeled, and Test reminder is preferred over asking for unrestricted background operation.

14 Doze and app standby

Exact alarms authorized by the user are used for due events. Inexact Limited mode may be delayed in Doze and MUST be labeled accordingly. The app does not schedule frequent `allowWhileIdle` events to bypass system quotas. Snooze is a user-visible alarm and is scheduled through the same exact-access path.

15 Reboot and direct boot

The manifest receiver listens for `LOCKED_BOOT_COMPLETED`, `BOOT_COMPLETED`, `MY_PACKAGE_REPLACED`, `TIME_SET`, `TIMEZONE_CHANGED` and exact-alarm permission state changes where available.

A minimal next-alarm envelope containing no media path or sensitive label is mirrored into device-protected storage. Before first unlock after reboot, the app can reschedule a generic due alert. Media and full reminder details remain credential-protected; Play asks the user to unlock. After `BOOT_COMPLETED`, Room reconciliation replaces the envelope and normal behavior resumes.

If direct-boot support is omitted from an early build, the limitation MUST be stated and the release cannot claim pre-unlock reboot recovery. The approved v1 target includes the minimal envelope.

16 Time and timezone changes

Wall-clock recurring rules follow the device timezone by default. On `TIME_SET` or `TIMEZONE_CHANGED`, native code invalidates derived occurrences beyond the idempotency horizon, recalculates, and registers the next alarm. Already resolved occurrence keys prevent replay after clock rollback.

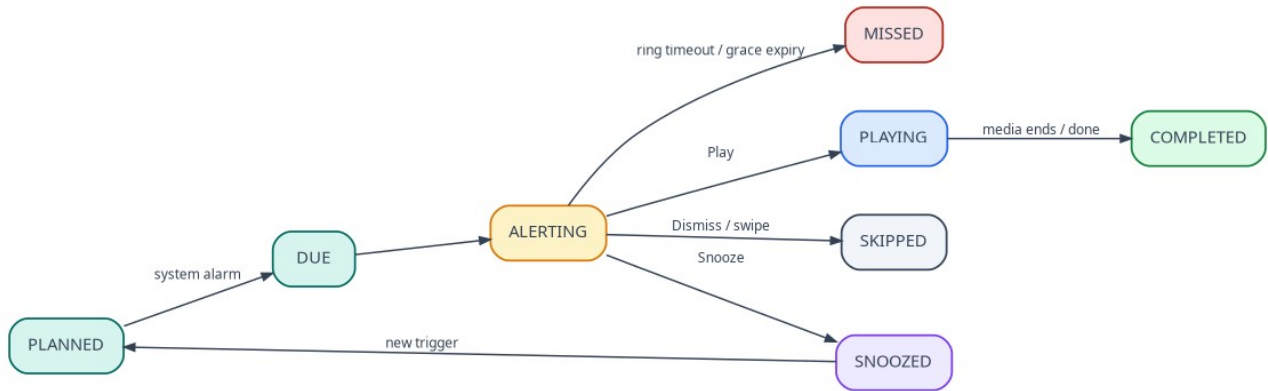
A once reminder stores an instant and does not move when timezone changes; its display updates to the new local equivalent. A daily reminder stores local time and follows the user's day.

17 Process death, crash and force-stop

Normal process death is fully supported. All actionable state exists in Room and native services/receivers can reconstruct it.

After user force-stop, Android suppresses alarms and broadcasts until the app is manually opened again. Media Reminder cannot override this. Health detects the next launch, explains the interruption, reconciles schedules and offers Test reminder. Copy MUST never imply that battery-efficient architecture bypasses force-stop.

18 State machine



Valid session transitions are:

Claimed -> Alerting -> {Snoozed, Dismissed, TransitioningToMedia, TimedOut, FailedSafe}
 TransitioningToMedia -> {MediaPlaying, AcceptedMediaUnavailable}

Terminal transitions are idempotent. Any invalid transition stops sound first, records a diagnostic and resolves to FailedSafe.

19 Android acceptance criteria

- **AND-001:** No unlocked/interactively used device test launches a full-screen activity.
- **AND-002:** Play/Snooze/Dismiss work with the React Native bridge intentionally disabled.
- **AND-003:** Only one ordinary next due alarm is registered after any stable operation.
- **AND-004:** No wake lock remains after the terminal session state.
- **AND-005:** A reboot restores the next event, including generic direct-boot behavior before unlock.
- **AND-006:** Revoking exact access cancels exact pending alarms, marks affected reminders and presents Limited choices.
- **AND-007:** Notification denial never starts an invisible continuous ringing service.
- **AND-008:** Full-screen ineligibility produces a transparent notification fallback.
- **AND-009:** Trigger-to-alert and action-stop budgets meet MR-15.
- **AND-010:** Manifest inspection confirms excluded permissions are absent.

19.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Technical Architecture

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-07
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define system components, dependency direction, runtime boundaries, deployment structure, failure isolation and implementation patterns.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created

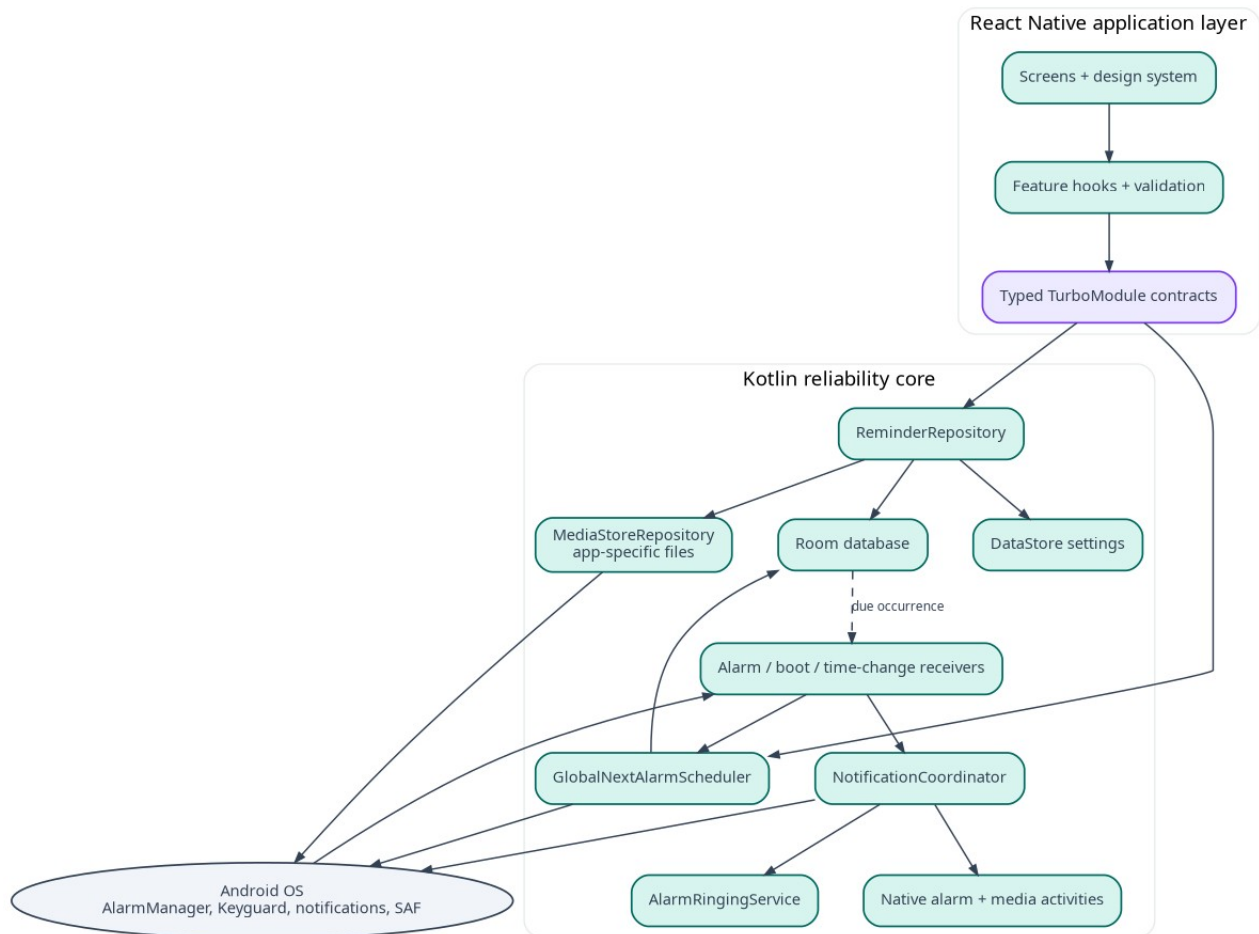
Term	Meaning
	when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Architectural style

Media Reminder uses a **hybrid offline mobile architecture**:

- React Native provides the normal application UI, navigation, forms and most presentation logic.
- Kotlin owns time-critical and background-safe behavior.
- Room is the authoritative application database and is accessed through native repositories.
- App-specific files hold imported media; cache holds derived thumbnails and temporary render artifacts.
- Android system services own scheduling, notification presentation, lock-screen policy and permission grants.

This is not a JavaScript app with a thin alarm plugin. The alarm core is a native subsystem with a typed React Native client.



2 Component map

2.1 React Native application

- app-shell: startup routing, theme, capability banner and global error boundary;
- features/today: occurrence timeline and next reminder;
- features/library: media list, metadata forms and preview orchestration;
- features/reminders: editor, recurrence form and profile selection;
- features/settings: Health, backup, profiles, privacy and About;
- design-system: tokens and accessible components;
- native-client: generated TypeScript interface to the TurboModule;
- state: view state and query cache only; no duplicate source-of-truth schedule database.

2.2 Kotlin domain and data

- ReminderRepository;
- MediaRepository;
- ProfileRepository;
- OccurrenceCalculator;
- SchedulerCoordinator;

- CapabilityRepository;
- AlarmSessionCoordinator;
- NotificationCoordinator;
- BackupRepository;
- DiagnosticRepository.

2.3 Android entry components

- MainActivity - React Native host;
- AlarmDispatchReceiver - due event claim and presentation decision;
- AlarmActionReceiver - Play/Snooze/Dismiss native commands;
- SystemEventReceiver - reboot/time/app-update/capability reconciliation;
- AlarmRingingService - bounded active sound/vibration;
- AlarmActivity - locked-screen native controls;
- MediaPlayerActivity - native resilient playback for accepted reminders;
- MediaReminderFileProvider - read-only sharing of completed exports.

3 Dependency rules

1. React Native features depend on the generated native client, never Android classes directly.
2. Android entry components depend on use cases/coordinators, not on UI or raw DAOs.
3. Repositories depend on Room DAOs and file gateways.
4. Domain calculation code has no Android UI dependency and is JVM-testable.
5. Backup DTOs are separate from Room entities and UI models.
6. Notification and scheduler identifiers are produced by dedicated factories with collision tests.
7. React state is disposable; killing the JS process must not lose user intent or active-session truth.

4 Source-of-truth ownership

Data	Owner	Read access
Reminders, profiles, schedules, occurrences	Room	Kotlin repositories; RN through queries
Imported media bytes	App-specific file storage	Media repository/player only
Derived thumbnails	Cache directory	UI/player; safely regenerable
Theme and lightweight preferences	DataStore	Settings repository
Active alarm session	Room plus bounded OS components	Alarm core and UI
Next registered alarm	Android AlarmManager plus mirrored scheduler	Scheduler coordinator

Data	Owner	Read access
	row	
Notification channel settings	Android OS	Capability repository, partial observability
View navigation/filter state	React state	React UI only

5 Repository layout

```
media-reminder/
  android/app/src/main/java/com/aslam/mediareminder/
    alarm/
    backup/
    capability/
    data/
    media/
    notifications/
    bridge/
    diagnostics/
  src/
    app/
    design-system/
    features/
    native-client/
    localization/
    testing/
  specs/
  docs/
  fixtures/
  scripts/
# Markdown source-of-truth pack
# portfolio and contributor docs
# synthetic media and backup fixtures
# build, schema, checksum and release tooling
```

Feature folders own screens, hooks, view models, tests and local types. Shared abstractions require two real consumers before promotion to a shared folder.

6 Startup sequence

1. Android creates `MainActivity` and initializes the RN host.
2. A native startup reconciler checks incomplete file/backup operations and active session consistency on an IO dispatcher.
3. React renders a fast shell using `DataStore` theme.
4. UI requests a `StartupSnapshot` from the native module: counts, next occurrence, capability summary, repair state and schema version.
5. Long repair operations surface as a dedicated state rather than blocking a blank splash screen.
6. The scheduler reconciles only if persisted generation or environment fingerprint requires it.

Cold startup does not enumerate or hash the full media library.

7 Native module

Use a Codegen/TurboModule interface so TypeScript and native signatures are generated/validated. The module exposes coarse-grained use cases rather than DAOs. Methods return serializable DTOs with ISO instants, stable UUIDs and typed error codes. Large media bytes never cross the bridge.

Events are limited to:

- `capabilityChanged;`

- `reminderDueWhileForeground;`
- `importProgress;`
- `exportProgress;`
- `repairStateChanged.`

Every event includes a monotonic sequence and correlation ID. UI can always refetch authoritative state if an event is lost.

8 Concurrency model

- Room writes run in transactions on `Dispatchers.IO`.
- Schedule calculation is pure and deterministic, called within a coordinator lock/mutex.
- File operations use a bounded IO pool and stream buffers; no whole-file byte arrays.
- Alarm action processing uses a database idempotency claim, not only an in-memory mutex.
- Only the active alarm session may own ringing resources.
- Backup import uses a single exclusive application mutation lock; read-only browsing may continue until commit, but reminder edits are temporarily disabled during final commit.

9 File architecture

```
files/
  media/<asset-uuid>.<normalized-extension>
  text/<asset-uuid>.json
  exports/                                # optional short-lived completed files before user save/share
  rollback/<operation-id>/ # bounded temporary rollback snapshot
cache/
  thumbnails/<asset-uuid>.webp
  import/<operation-id>/
  export/<operation-id>/
  playback/
no_backup/
  diagnostics/
device_protected/
  next_alarm_envelope.json
```

The database never stores absolute paths. It stores storage key + relative name. File path resolution is centralized and rejects traversal.

10 Scheduler transaction pattern

Because AlarmManager is outside Room, scheduling is an outbox-style operation:

1. transaction updates user intent, derived occurrence and `scheduler_state` desired generation;
2. coordinator applies desired state to AlarmManager;
3. on success it records applied generation;
4. on process death between steps, startup/system-event reconciliation sees desired != applied and retries;
5. receiver validates generation, so stale pending intents cannot act on new state.

This avoids pretending a database transaction can atomically include an operating-system alarm registration.

11 Alarm isolation

The native alarm core has no dependency on Metro, bundle loading, navigation readiness or React component state. It uses native resources for action labels and accessibility strings. The React UI may enhance the foreground presentation but can never be the only way to stop sound.

12 Media playback

`MediaPlayerActivity` uses `Media3 ExoPlayer` with local `file:///content` resolver abstraction internal to the app. It receives only an asset UUID and session token. It validates the token, resolves the app-owned file and starts after the Play action. It handles audio focus, lifecycle pause, rotation, PiP disabled in v1, and unsupported codec errors.

Playback completion can record `Completed` when the user watches/listens beyond a configurable threshold; MVP defaults to `Accepted` on Play and optionally `Completed` on natural end. The distinction is documented and not used to shame the user.

13 Backup architecture

Export creates immutable logical DTOs from a consistent database snapshot, streams JSON and media to the user-selected destination, then writes checksums and closes the archive. Import streams to private staging, validates, creates a conflict plan, and commits through repositories. Raw DAOs are never exposed to arbitrary archive fields.

14 Error architecture

Errors have four layers:

- domain code, e.g. `SCHEDULE_NONEXISTENT_LOCAL_TIME`;
- user-safe message key;
- diagnostic correlation ID;
- optional internal cause in local logs.

No user-visible stack trace. No file name in diagnostic export unless the user explicitly includes a redacted mapping. Components fail safe: alert errors stop sound, import errors leave current data unchanged, scheduler errors preserve desired state for reconciliation.

15 Build variants

- `debug`: developer tools, synthetic event injector, verbose local log;
- `qa`: release-like signing, test fixtures and controlled diagnostics, no remote analytics;
- `release`: no debug menus, no Internet permission, minified only after native action tests pass.

A build-time manifest test fails if excluded permissions or undeclared exported components appear.

16 Dependency policy

Prefer AndroidX/Jetpack and React Native core. Any third-party library must have active maintenance, compatible license, no hidden network requirement, modern Android support and a clear removal path. Native alarm scheduling should not depend on a generic notification package that obscures platform behavior; small platform wrappers are owned in-repo.

17 Observability

Local structured diagnostics use bounded files and privacy-safe event fields. Key events include scheduler desired/applied generation, trigger delta, presentation decision, action latency, permission snapshot code, service lifetime, wake-lock lifetime, import/export phase and repair result. No content titles or file paths.

18 Architectural acceptance

- Build succeeds with React Native runtime removed from an alarm instrumentation fixture.
 - Static dependency tests prevent Android entry components from importing React UI packages.
 - Room is the only reminder source of truth.
 - A stale PendingIntent cannot resolve a newer occurrence.
 - Crash injection at each import and restore phase results in either old or committed state, not mixed state.
 - Release manifest contains no Internet or overlay permission.
-

18.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Internal Module and Data Contracts

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-08
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define typed React Native/native boundaries, domain commands, data transfer objects, events, error codes and compatibility rules.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Contract principles

Contracts are versioned, narrow and use-case oriented. UI code does not receive Room entities, file paths, Android URIs, PendingIntents or platform objects. UUIDs are lowercase canonical strings. Instants are UTC ISO-8601 strings. Local times use HH:mm:ss plus IANA zone identifiers where required.

2 Native module surface

Conceptual Codegen interface:

```
export interface Spec extends TurboModule {
  getStartupSnapshot(): Promise<StartupSnapshot>;
  listMedia(query: MediaQuery): Promise<Page<MediaSummary>>;
  getMedia(id: UUID): Promise<MediaDetail>;
  beginMediaImport(request: ImportRequest): Promise<OperationRef>;
  updateMedia(request: UpdateMediaRequest): Promise<MediaDetail>;
  deleteMedia(request: DeleteMediaRequest): Promise<MutationResult>;

  listReminders(query: ReminderQuery): Promise<Page<ReminderSummary>>;
  getReminder(id: UUID): Promise<ReminderDetail>;
  saveReminder(request: SaveReminderRequest): Promise<SaveReminderResult>;
  setReminderEnabled(id: UUID, enabled: boolean): Promise<EnableResult>;
  deleteReminder(id: UUID): Promise<MutationResult>;

  listProfiles(): Promise<ReminderProfile[]>;
  saveProfile(request: SaveProfileRequest): Promise<ReminderProfile>;
  resetBuiltInProfile(id: UUID): Promise<ReminderProfile>;

  getCapabilitySnapshot(): Promise<CapabilitySnapshot>;
  openCapabilitySettings(kind: CapabilityKind): Promise<LaunchResult>;
  scheduleTestReminder(mode: TestMode): Promise<TestReminderResult>;

  beginExport(request: ExportRequest): Promise<OperationRef>;
  inspectBackup(uriToken: string): Promise<BackupInspection>;
  commitImport(request: CommitImportRequest): Promise<ImportResult>;
  cancelOperation(id: UUID): Promise<CancelResult>;

  playDueSession(sessionId: UUID, nonce: string): Promise<ActionResult>;
  snoozeDueSession(sessionId: UUID, minutes: number, nonce: string): Promise<ActionResult>;
}
```



```
dismissDueSession(sessionId: UUID, nonce: string): Promise<ActionResult>;
}
```

The exact generated syntax follows the installed React Native Codegen version. Semantic names and payload rules are binding.

3 Shared primitives

```
type UUID = string;
type Instant = string;           // UTC, e.g. 2026-08-05T22:15:00Z
type LocalDate = string;        // YYYY-MM-DD
type LocalTime = string;        // HH:mm:ss
type ZoneId = string;           // IANA identifier
type CorrelationId = string;
```

```
type ResultStatus = 'ok' | 'limited' | 'needs_action';
```

Numbers representing byte sizes are decimal strings in bridge DTOs when they may exceed JavaScript's safe integer range. Durations are integer milliseconds only where bounded below `Number.MAX_SAFE_INTEGER`; schedule snooze values use integer minutes.

4 Startup snapshot

```
interface StartupSnapshot {
  contractVersion: 1;
  schemaVersion: number;
  appVersion: string;
  mediaCount: number;
  activeReminderCount: number;
  nextOccurrence: OccurrenceSummary | null;
  capability: CapabilitySnapshot;
  repair: RepairSummary;
  activeSession: ActiveSessionSummary | null;
  sequence: string;
}
```

A contract version mismatch is a hard developer error in debug and a user-safe update-required screen in release. Additive optional fields are backward compatible; renamed or semantic changes require a new contract version.

5 Media DTOs

```
type MediaKind = 'video' | 'audio' | 'image' | 'text';
type IntegrityState = 'healthy' | 'unchecked' | 'missing' | 'changed' | 'unsupported';
```

```
interface MediaSummary {
  id: UUID;
  kind: MediaKind;
  title: string;
  durationMs?: number;
  sizeBytes: string;
  thumbnailToken?: string;
  category?: NamedRef;
  tags: NamedRef[];
  activeReminderCount: number;
  integrity: IntegrityState;
  createdAt: Instant;
}
```

`thumbnailToken` is an opaque app-local token consumed by an image provider; it is not an absolute path. UI sends an import intent request and native code launches the system picker so URI lifecycle stays native and typed.

6 Reminder DTOs

```
interface ReminderDraft {
  id?: UUID;
  mediaId: UUID;
  label: string;
  notes?: string;
  schedule: ScheduleRuleDto;
  profileId: UUID;
  snooze: SnoozePolicyDto;
  enabledIntent: boolean;
}

type ScheduleRuleDto =
  | { type: 'once'; instant: Instant; originZone: ZoneId }
  | { type: 'daily'; localTime: LocalTime; zonePolicy: 'follow_device' }
  | { type: 'weekdays'; localTime: LocalTime; isoWeekdays: number[]; zonePolicy: 'follow_device' };

interface SaveReminderResult {
  reminder: ReminderDetail;
  nextOccurrence: OccurrenceSummary | null;
  capabilityResult: CapabilityEvaluation;
  schedulerGeneration: string;
}
```

UI never calculates authoritative next occurrence. It may preview locally for responsiveness, but Save returns native-calculated truth and any DST resolution.

7 Capability contract

```
type CapabilityKind =
  | 'notifications'
  | 'exact_alarm'
  | 'full_screen_intent'
  | 'channels'
  | 'battery_environment'
  | 'scheduler';

interface CapabilityItem {
  kind: CapabilityKind;
  status: 'ready' | 'limited' | 'blocked' | 'unknown';
  effectKey: string;
  action: 'none' | 'request_runtime' | 'open_special_access' | 'open_channel' | 'run_test' | 'open_app';
  observedAt: Instant;
}
```

The bridge exposes semantic status, not raw permission constants. Raw platform values may be included in diagnostic-only fields unavailable to normal UI.

8 Operations and progress

Long-running imports/exports are represented by operation UUID. Progress events:

```
interface OperationProgressEvent {
  operationId: UUID;
  kind: 'import' | 'export' | 'backup_inspection' | 'backup_commit' | 'repair';
  phase: string;
  completedUnits?: string;
  totalUnits?: string;
  currentItemIndex?: number;
  totalItems?: number;
  cancellable: boolean;
  sequence: string;
  correlationId: CorrelationId;
}
```

Events are hints. A screen restored after process death calls `getOperation(id)` or startup snapshot; it does not assume it received every event.

9 Alarm action contract

Action nonce is generated by native notification/activity code. A result may be replayed for an identical nonce.

```
interface ActionResult {
  sessionId: UUID;
  outcome: 'playing' | 'snoozed' | 'dismissed' | 'already_resolved' | 'media_unavailable' |
'failed_safe';
  effectiveAt: Instant;
  snoozedUntil?: Instant;
  nextOccurrence?: OccurrenceSummary;
}
```

Native receivers call the same use case directly; the React bridge is not in that path.

10 Backup contract

```
interface BackupInspection {
  archiveVersion: string;
  createdAt: Instant;
  sourceAppVersion: string;
  mediaCount: number;
  reminderCount: number;
  compressedBytes: string;
  expectedUncompressedBytes: string;
  checksumStatus: 'valid' | 'invalid' | 'missing';
  compatibility: 'compatible' | 'migratable' | 'too_new' | 'unsupported';
  conflicts: ConflictSummary[];
  warnings: string[];
  importToken: string;
}
```

importToken references validated private staging and expires. CommitImportRequest cannot accept a raw URI; it must use the token from inspection so commit never bypasses validation.

11 Error envelope

Rejected promises use a normalized object:

```
interface NativeErrorEnvelope {
  code: string;
  messageKey: string;
  category: 'validation' | 'capability' | 'storage' | 'media' | 'schedule' | 'backup' | 'security' |
'internal';
  retryable: boolean;
  correlationId: CorrelationId;
  field?: string;
  safeDetails?: Record<string, string | number | boolean>;
}
```

Representative codes:

Code	Meaning	Retry
MR_MEDIA_UNSUPPORTED_TYPE	Header/probe not accepted	No, choose another file
MR_STORAGE_INSUFFICIENT	Free space below required reserve	After freeing space
MR_SCHEDULE_NONEXISTENT_TIME	DST gap requires resolution	After user choice
MR_EXACT_ACCESS_REQUIRED	Exact mode selected but access missing	After settings or Limited choice
MR_NOTIFICATION_BLOCKED	Alert cannot be shown	After permission/channel

Code	Meaning	Retry
		change
MR_BACKUP_CHECKSUM_INVALID	Archive content mismatch	No; use another archive
MR_BACKUP_TOO_NEW	Reader lacks schema migrator	After app update
MR_ACTION_ALREADY_RESOLVED	Duplicate action	Treated as success-like result
MR_INTERNAL_FAILED_SAFE	Invariant failed; resources stopped	Diagnostic/retry depending context

Messages are localized by key in React Native or native resources. Error envelopes MUST NOT contain file paths, labels or media titles.

12 Versioning rules

- Bridge contract version is integer and changes only for breaking semantics.
- Database schema version follows Room migration numbering.
- Backup archive version uses semantic `major.minor`.
- Domain event/diagnostic version is independent.
- Every release records all four versions in About and diagnostic export.

An older UI bundle must never run against a newer incompatible native contract. Standard packaged React Native builds prevent this, but OTA updates are not used in v1, eliminating split-version risk.

13 Event ordering and idempotency

Each event stream has a monotonically increasing sequence persisted per process/session as appropriate. UI compares sequence and refetches after gaps. Mutating requests may include a client operation ID; native stores recent IDs and returns the original result within the idempotency window.

Alarm actions require native-generated nonce and session. Backup commit requires validated import token. Delete operations require current entity version to prevent overwriting concurrent changes.

14 Security properties

- No method returns arbitrary filesystem paths.
- No method opens arbitrary intent URI supplied by JavaScript without validation.
- All share operations use explicit read-only grants.
- Sensitive media is not serialized across the bridge.
- Debug-only injection methods are absent from release Codegen spec.
- Every exported Android component validates action, package, token and record state.

15 Contract test requirements

Golden JSON fixtures exist for every DTO and error. TypeScript tests decode native fixtures; Kotlin tests encode the same semantic objects. Breaking changes fail CI unless contract version and migration guide

are updated. Fuzz tests cover unknown fields, missing optional fields, invalid enums, oversized strings and malicious backup DTO values.

15.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Database and Local Storage Specification

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-09
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define the Room schema, relationships, constraints, migrations, file layout, integrity model and transactional recovery.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

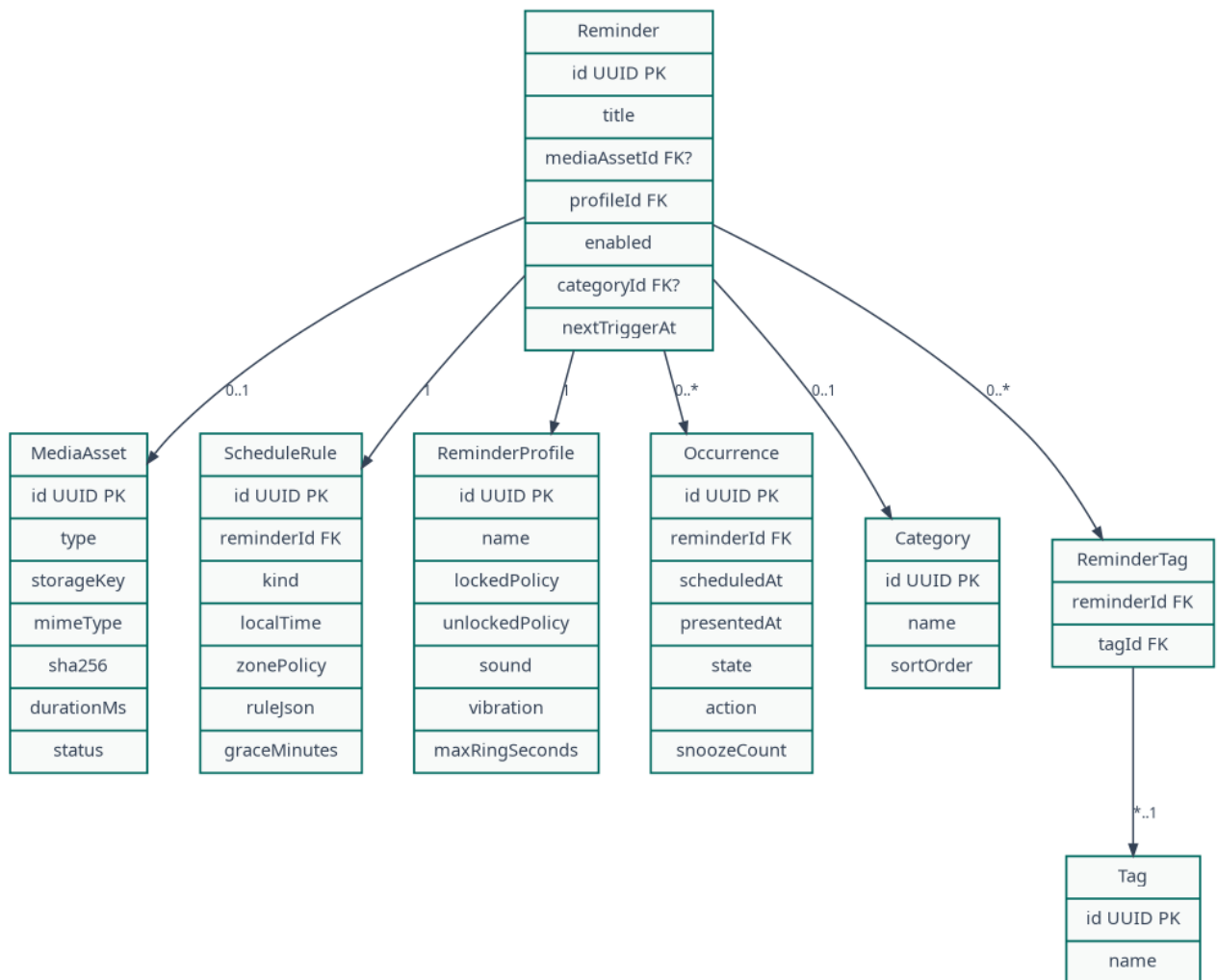
The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Persistence objectives

Persistence MUST remain correct through process death, reboot, partial file copy, notification action races, import cancellation and schema upgrades. Room is authoritative for logical data. The filesystem stores app-owned bytes. DataStore stores small preferences and a device-protected alarm envelope.



2 Database configuration

- Database filename is internal and never included in backups.
- Room uses WAL mode where supported.
- Foreign keys are enabled.
- Destructive migration is prohibited in release builds.
- Encryption-at-rest beyond Android sandbox/file-based encryption is a future option; v1 backups are explicitly unencrypted.
- Timestamps are UTC epoch milliseconds in Room; bridge/export serializes ISO-8601.
- Primary IDs are UUID strings generated by secure platform UUID.
- User-visible sort normalization is stored separately only when profiling demonstrates need.

3 Core entities

3.1 media_assets

Column	Type	Constraints/purpose
id	TEXT	PK UUID

Column	Type	Constraints/purpose
kind	TEXT	video/audio/image/text enum
title	TEXT	1-160 Unicode scalar length
notes	TEXT nullable	max 4000 characters
storage_key	TEXT	relative opaque filename, unique
mime_type	TEXT	normalized detected type
size_bytes	INTEGER	nonnegative
sha256	TEXT	64 lowercase hex, indexed
duration_ms	INTEGER nullable	nonnegative
width_px,height_px	INTEGER nullable	decoded metadata
category_id	TEXT nullable	FK categories, SET NULL
integrity_state	TEXT	healthy/unchecked/missing/changed/unsupported
created_at,updated_at	INTEGER	UTC millis
entity_version	INTEGER	optimistic concurrency

No source gallery URI is required after copy. Optional provenance such as original display name is excluded by default or stored as user-visible metadata only after privacy review.

3.2 reminder_profiles

Stable built-in UUIDs are seeded in migration. Columns include name, built-in flag, channel class/version, sound mode/reference, vibration JSON validated by converter, full-screen-on-locked Boolean, timeout seconds, retry count/interval, grace seconds, default snooze minutes, created/updated and entity version.

Profile timeout range: 15-600 seconds. Retry count: 0-3 in v1. Vibration total duration is capped.

3.3 reminders

Column	Purpose
id	PK UUID
media_id	FK media, restrict until dependency policy chosen
profile_id	FK profile, restrict
label	display label; default copied from title but independent
notes	reminder-specific notes
enabled_intent	user's desired enabled state
effective_state	disabled/needs_setup/active/archived
snooze_policy_json	validated bounded values

Column	Purpose
history_enabled	local recording preference
created_at,updated_at	timestamps
entity_version	optimistic concurrency

Effective state is derived but persisted for fast native queries and backup transparency; reconciliation can recompute it.

3.4 `schedule_rules`

One-to-one with reminder in v1. Fields include type, once instant, local time seconds-of-day, ISO weekday mask, zone policy, origin zone, DST gap policy, DST overlap policy and rule version. Check constraints ensure only relevant fields are set for each type.

3.5 `occurrences`

Column	Purpose
id	PK UUID
reminder_id	FK reminder cascade
kind	base/snooze/retry/test
parent_occurrence_id	nullable self FK
scheduled_at	due instant
occurrence_key	unique deterministic key for base occurrence
state	pending/claimed/accepted/snoozed/ dismissed/missed/timed_out/ failed_safe
triggered_at,resolved_at	nullable timestamps
action	terminal action code
created_at	timestamp

Unique (reminder_id, occurrence_key) prevents recurrence duplication. Only a bounded horizon of future derived occurrences is stored; the scheduler needs the next one, not years of rows.

3.6 `active_alarm_session`

At most one row is active, enforced by a singleton key or unique partial invariant in repository logic. Contains session UUID, occurrence UUID, state, started time, presentation decision, notification ID, service started flag, wake deadline, action nonce generation and last update. Terminal sessions are moved to diagnostics/history or deleted after reconciliation.

3.7 `scheduler_state`

Singleton row: desired occurrence UUID/time, desired generation, applied generation, pending-intent identity, last reconcile time, last reason, last error code. It is an outbox record for AlarmManager synchronization.

3.8 `pending_file_operations`

Tracks operation UUID, kind, temporary relative path, target relative path, associated entity UUID, phase, expected bytes/hash, created/updated and last error. Startup repair uses it to delete or finalize safe operations.

3.9 idempotency_records

Stores scope, key, request hash, result summary, created/expiry. Alarm action records persist long enough to cover repeated notification intents. General UI operation records have shorter retention.

3.10 categories and tags

categories(id,name,normalized_name,sort_order,created_at,updated_at) with unique normalized name.

tags(id,name,normalized_name,created_at,updated_at) with unique normalized name.

reminder_tags(reminder_id,tag_id) and optionally media_tags(media_id,tag_id) use composite primary keys.

3.11 schema_meta

Stores schema version, backup writer version, created app version, last migration version/time and integrity check state.

4 Indices

Required indices:

- media_assets(sha256);
- media_assets(category_id, updated_at);
- reminders(effective_state, updated_at);
- occurrences(state, scheduled_at);
- occurrences(reminder_id, occurrence_key) unique;
- pending_file_operations(phase, updated_at);
- idempotency_records(scope, key) unique and expires_at;
- category/tag normalized names unique.

Query plans for Today, next occurrence, active reminder count and media dependency count are captured in performance tests.

5 File storage

Imported bytes use an opaque UUID filename and normalized extension. The mapping is internal:

files/media/7f3...a9.mp4

No user-supplied path segment enters the file path. Text assets are JSON or UTF-8 in a separate directory. Derived thumbnails are WebP cache and may be cleared at any time. Export archives are written to a user-selected document URI where possible; a temporary private file is only used when the destination API requires it.

6 Storage limits

Defaults, configurable only through build constants:

- individual asset soft warning at 500 MB;
- individual asset hard limit at 2 GB for v1 unless large-file tests pass;
- backup expected uncompressed hard limit at 10 GB or 80% of available space, whichever is lower;
- maintain 250 MB or 5% free-storage reserve after import/restore, whichever is greater;

- text title 160 characters, notes 4000, category/tag 60;
- diagnostics 5 MB ring buffer;
- thumbnails cache 250 MB with LRU eviction.

These values are product protections, not filesystem capabilities. The UI displays estimates before copy.

7 Import atomicity

The database and filesystem cannot share a true atomic transaction. Use a journaled two-phase protocol:

1. persist pending operation;
2. stream and verify temporary file;
3. atomically rename on same filesystem;
4. transaction inserts record and marks operation committed;
5. cleanup removes journal.

On startup:

- temp exists, no target, phase Copying -> delete temp;
- target exists, no media row, phase Promoted -> verify hash then complete row only if full metadata exists, otherwise quarantine/delete;
- media row exists, target missing -> mark Missing and disable reminders;
- both exist and hash valid -> complete/clear operation.

8 Backup transaction support

Before Replace import, current logical records are exported to a private rollback package and current media map is journaled. Commit stages new media under an operation directory, inserts data in a Room transaction using conflict plan, atomically promotes files, applies scheduler desired state and then removes rollback after verification. A crash at any point is recovered by operation phase.

9 Database migrations

Every schema version has:

- forward Room migration;
- migration fixture from every supported prior public version or a tested chain;
- invariant checks;
- backup reader compatibility note;
- rollback strategy at release level (usually reinstall + restore, not database downgrade).

Migrations MUST be idempotent where retry could occur and MUST NOT hash entire large files on the main thread. Built-in profile changes use stable IDs and explicit version migration, preserving user edits unless reset.

10 Integrity checks

Startup performs lightweight checks only. Full `PRAGMA integrity_check`, media hash audit and orphan scan are user-invoked or deferrable maintenance. Health reports last full check. Any database corruption path first copies the damaged database to a private recovery area with bounded retention, then offers Restore backup or Reset. It never silently deletes data.

11 Data retention

Occurrence history defaults to 90 days. Diagnostic event retention is size/time bounded. Idempotency records are retained by scope: alarm action 7 days, UI mutation 24 hours, backup commit 30 days. Completed export temporary files expire after 24 hours unless the user saved them externally.

12 Uninstall and clearing data

The app explains that Android removes app-specific media and database on uninstall or Clear storage. External exported ZIPs are not deleted. Cache clearing does not remove source media or reminders. "Reset app" requires explicit typed/destructive confirmation and optional export prompt.

13 Database acceptance

- Foreign-key and check constraints pass on all fixtures.
 - Crash injection at each two-phase file step reaches a documented repaired state.
 - No database column contains an absolute filesystem path.
 - Next occurrence query remains within MR-15 budget at 10,000 reminders/50,000 retained occurrences.
 - Migration tests cover every public schema version.
 - Export/import uses DTO mapping, not database file copy.
-

13.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Backup, Export and Import Specification

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-10
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define the portable ZIP format, manifest, checksums, validation, conflict handling, transactional restore and compatibility policy.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

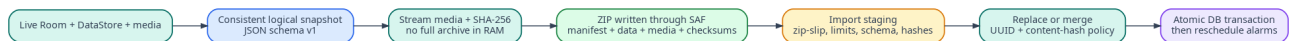
The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Goals

A backup must be portable between supported Android devices, inspectable, recoverable after interruption and independent of internal Room layout. It contains private user media and is therefore clearly labeled as sensitive. Version 1 archives are plain ZIP files; passphrase encryption is a later compatible format extension.



2 File naming

Default filename:

MediaReminder_Backup_2026-08-05_221500_v1.0.mrbackup.zip

The .mrbackup.zip suffix is descriptive only; the file remains a standard ZIP. User may rename it. Archive identity comes from manifest.json, not filename.

3 Archive layout

```

manifest.json
README.txt
checksums.sha256
data/
  media-assets.json
  reminder-profiles.json
  categories.json
  tags.json
  reminders.json
  schedule-rules.json
  reminder-tags.json
  settings.json
  history.json          # optional
media/
  <asset-uuid>.<extension>
  
```

Excluded:

- Room database/WAL/SHM files;
- DataStore binary files;
- cache thumbnails;
- absolute or content URI paths;
- notification channel IDs as an assumption of effective behavior;
- permissions, exact-alarm grants, full-screen eligibility, DND settings;
- active alarm session and pending operation state;
- diagnostic logs unless explicitly exported through a separate support action.

4 Manifest schema

```
{
  "format": "com.aslam.mediareminder.backup",
  "archiveVersion": "1.0",
  "createdAt": "2026-08-05T14:15:00Z",
  "sourceAppVersion": "1.0.0",
  "sourceSchemaVersion": 1,
  "minimumReaderArchiveVersion": "1.0",
  "exportId": "uuid",
  "scope": "all",
  "includesHistory": false,
  "counts": {
    "mediaAssets": 12,
    "reminders": 18,
    "profiles": 3,
    "categories": 6,
    "tags": 9
  },
  "totalMediaBytes": "486392014",
  "hashAlgorithm": "SHA-256",
  "recordsEncoding": "UTF-8 JSON",
  "privacy": "contains-private-media",
  "extensions": {}
}
```

Unknown minor-version extension keys are ignored only inside designated `extensions` objects. Unknown required fields or unsupported major version stop import.

5 JSON record rules

- UTF-8 without BOM.
- Top-level object includes `schema`, `records` array and optional `extensions`.
- Stable UUIDs are preserved.
- Instants are UTC ISO-8601.
- Enums use lowercase stable wire values.
- Strings retain Unicode; names are normalized only for conflict comparison, never rewritten silently.
- Numeric byte sizes use decimal strings where needed.
- JSON depth and string lengths are bounded during parse.
- Cross-file references must resolve before commit.

Example media record:

```
{
  "id": "d7c2c52f-5b5c-4ced-8531-f9efbc2a6d8f",
```



```

"kind": "video",
"title": "Morning remembrance",
"notes": null,
"archivePath": "media/d7c2c52f-5b5c-4ced-8531-f9efbc2a6d8f.mp4",
"mimeType": "video/mp4",
"sizeBytes": "28390110",
"sha256": "...64 lowercase hex...",
"durationMs": 93400,
"categoryId": "...",
"createdAt": "2026-07-31T10:00:00Z",
"updatedAt": "2026-08-04T12:12:00Z"
}

```

6 Checksums

checksums.sha256 contains SHA-256 for every file except itself, using a canonical two-space separator and relative POSIX path. It includes manifest.json, README, each data file and every media file. Export closes entries before writing the checksum file. Import recalculates while streaming and rejects mismatch.

A separate SHA-256 for the completed archive is displayed after export and may be published beside release fixtures; it is not stored within itself.

7 Export algorithm

1. Preflight: query counts and media sizes, estimate destination needs, display privacy warning.
2. User selects destination through ACTION_CREATE_DOCUMENT or compatible document-provider flow.
3. Create export operation row with cancellation token.
4. Open a consistent logical database snapshot.
5. Stream manifest placeholder/data JSON/media entries through ZIP output; never load full media into memory.
6. Calculate per-entry hashes and written bytes.
7. Write final manifest values, using a staging ZIP when the destination cannot replace an earlier manifest entry safely.
8. Write checksums, close ZIP, flush destination and calculate archive hash where readable.
9. Verify central directory by reopening or lightweight parser validation.
10. Mark complete and offer read-only FileProvider share when a private temporary file was used.

Cancellation deletes or truncates the partial destination where the provider permits; otherwise it clearly labels that the provider may retain an incomplete file.

8 Import phases

8.1 1. Acquisition

User chooses one ZIP through ACTION_OPEN_DOCUMENT. The app does not persist broad directory access. It streams/copies to a private staging location when random access or repeated validation is required.

8.2 2. Structural validation

Before extraction:

- confirm ZIP signature/central directory;
- cap entry count;

- reject absolute paths, drive prefixes, ..., NUL and normalized duplicates;
- reject symlink-like entries and unsupported compression methods;
- cap per-entry and total uncompressed sizes;
- enforce compression-ratio threshold to reduce ZIP bomb risk;
- require exactly one root `manifest.json` and checksum file;
- reject duplicate entry names and case-fold collisions on relevant filesystems.

8.3 3. Semantic validation

- parse manifest with streaming/bounded parser;
- confirm format and version compatibility;
- validate all JSON schemas, lengths and enums;
- validate UUID uniqueness;
- resolve every foreign key;
- compare declared counts/sizes to actual;
- verify every checksum and media header/probe;
- estimate final disk needs including staging and rollback;
- create conflict plan without modifying current data.

8.4 4. Preview

Display archive date/version/counts, checksum result, unsupported records, conflict summary, storage requirement and permissions that cannot be restored. Generate an expiring `importToken` tied to staged bytes and validation digest.

8.5 5. Commit

Only a valid token can commit. Acquire exclusive mutation lock, recheck free space and current entity versions, create rollback snapshot for Replace, then apply plan through repositories. Promote files atomically, commit Room changes, clear derived occurrences, reconcile capabilities and schedule. Verify record counts and sampled/full hashes as policy requires. Remove staging after success.

9 Import modes

9.1 Inspect only

No mutation. User may view compatibility and conflicts, then cancel or continue.

9.2 Merge

Existing data is retained. Rules:

- same UUID and same semantic record/hash: reuse existing;
- same media UUID and same hash but metadata differs: default keep local metadata, offer archive copy values in conflict review;
- same UUID with different media hash: conflict, import as new UUID with "Imported" suffix unless user selects local/archive;
- different UUID but same media hash: reuse existing bytes and create/import record mapping according to user choice;

- category/tag same normalized name: merge to local ID and remap references;
- reminder semantic duplicate: show conflict; default keep both disabled only when times/profiles differ, otherwise skip exact duplicate;
- built-in profile UUID: map to local built-in; archive customizations import as a new custom profile unless user explicitly applies them.

9.3 Replace

All user logical data and app-owned media are replaced by archive scope after rollback snapshot. Device-owned settings remain unchanged. On completion, reminders are recalculated and set to Needs setup/disabled until capability review. Replace confirmation states that current local-only data will be removed.

10 Conflict plan

Every conflict has stable identifier, local summary, archive summary, recommended action and consequences. Choices are deterministic and serializable so commit uses the reviewed plan. The plan expires if local entity versions change before commit.

11 Atomicity and rollback

Restore uses operation phases:

```
INSPECTED -> STAGED -> ROLLBACK_READY -> DB_PREPARED -> FILES_PROMOTED -> DB_COMMITTED -> VERIFIED -> SCHEDULED -> COMPLETE
```

Crash recovery examines phase and journal. Before `DB_COMMITTED`, restore current state and delete staged files. After commit but before verification, finish verification/scheduling or roll back using snapshot. The user sees **Finishing previous restore** on next launch.

12 Compatibility

Archive version uses major/minor:

- reader MUST reject a higher major version;
- reader MAY accept higher minor only when manifest declares current reader sufficient and unknown fields are extension-scoped;
- writer produces the oldest practical format compatible with current features;
- backup migrations convert DTOs in staging, never mutate original ZIP;
- at least the last three public major app releases' archives are supported where feasible;
- support policy is published in release notes.

13 Encryption roadmap

Plain ZIP v1 is intentional for portability and transparent recovery. The export screen warns that anyone with the file can read it. P1 encryption adds a new archive major/version using established authenticated encryption and password-based key derivation; it MUST not invent a custom cipher. Forgotten passphrases cannot be recovered, and metadata leakage is documented.

14 Sharing and privacy

Completed private exports shared through another app use FileProvider with read-only temporary URI permission. The app never grants a whole directory. Share chooser copy says that the selected destination controls the file after sharing. Export history stores no destination path by default.

15 Validation limits

Baseline configurable limits:

- 20,000 ZIP entries;
- 10 GB expected uncompressed or available-space policy, whichever is lower;
- 2 GB per media entry for v1;
- 100 MB maximum aggregate JSON (far above normal use but bounded);
- compression ratio warning at 100:1 and rejection at a security-reviewed threshold;
- nesting is irrelevant because entries are flat; nested ZIP media is treated as opaque unsupported media, not recursively extracted.

16 Backup acceptance

- **BKP-001:** A clean-device round trip preserves supported logical equality and media SHA-256.
 - **BKP-002:** Malicious traversal, duplicate, symlink, bomb and checksum fixtures are rejected before current data changes.
 - **BKP-003:** Cancellation/crash at every phase leaves either old or fully imported state after repair.
 - **BKP-004:** Higher unsupported major version produces a non-destructive update-required result.
 - **BKP-005:** Replace cannot proceed without a validated token, conflict plan and rollback-ready state.
 - **BKP-006:** Notification permissions, exact access and channel settings are never reported as restored.
 - **BKP-007:** Export peak memory remains within MR-15 regardless of media size.
-

16.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Edge Cases, Failure Recovery and Conflict Handling

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-11
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define expected behavior for platform, timing, media, storage, concurrency, backup, interaction and lifecycle edge cases.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.

Term	Meaning
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Failure philosophy

The preferred outcome is not “never fail”; it is **fail visibly, stop harmful side effects, preserve user data, and offer a deterministic next action**. Alert failures stop sound first. Import failures preserve current data. Scheduling failures preserve user intent and expose Needs setup. Unknown device state chooses the less intrusive presentation.

2 Severity model

Severity	Definition	Examples
Critical	Data loss, unbounded resource use, inaccessible stop action, security boundary escape	ZIP traversal writes outside staging; alarm cannot be silenced
High	Core reminder wrong or misleading, repeated duplicate, privacy promise broken	Full-screen over unlocked app; duplicate snooze events
Medium	Recoverable feature failure with workaround	Thumbnail missing; custom tone falls back
Low	Cosmetic or rare inconvenience	Sort order resets; harmless animation glitch

3 Device and lifecycle cases

3.1 App process killed normally

Expected: system alarm remains; native receiver starts; actions work; RN UI later reflects Room state. No user message needed.

3.2 User force-stops app

Expected: Android may suppress alarms and broadcasts until manual relaunch. On next launch, reconcile and show a one-time Health notice: "Android paused Media Reminder after it was force-stopped. Reminders are scheduled again now." Do not claim the app can self-recover before relaunch.

3.3 Device reboot

Expected: direct-boot envelope reschedules generic next alert; after first unlock, full Room reconciliation restores detail. If the app was installed on external/adoptable storage with delayed availability, show generic notification and retry after unlock/storage available.

3.4 App update

MY_PACKAGE_REPLACED triggers reconciliation. Database migration completes before active scheduling. If migration fails, no continuous alert starts; show recovery screen and retain old files/database copy.

3.5 App data cleared/uninstalled

All internal content is gone by Android design. There is no recovery without an external backup. About/Backup copy explains this before it happens; the app cannot intercept uninstall.

4 Time cases

4.1 Timezone changes while traveling

Daily/weekday schedules follow new device local time. Once reminders retain absolute instant. Today list refreshes. Avoid alerting twice for the same occurrence key.

4.2 DST spring gap

A nonexistent local time shifts to the first valid instant after the gap by default, with editor disclosure. Alternative Skip policy is P1. Tests cover zones with 30-minute and nonstandard transitions, not only one-hour changes.

4.3 DST fall overlap

Default uses the first occurrence. A persisted overlap policy prevents different recalculation after reboot. The same base occurrence key cannot fire twice.

4.4 Manual clock moved backward

Already resolved occurrences inside the idempotency horizon do not replay. Future wall-clock schedules recalculate. Diagnostics record time-change reconciliation, not personal labels.

4.5 Manual clock moved forward

Due items within grace may alert once; older ones become Missed. Never avalanche many full-screen activities.

4.6 Time changes while an alarm is active

Active session uses elapsed realtime for timeout and remains stable. Displayed clock updates, but the session does not restart or extend.

5 Alarm collision cases

5.1 Multiple reminders at same instant

One sound owner and one activity. Queue ordered by scheduled time and UUID. UI shows count. Each action resolves one item unless user explicitly chooses Dismiss all due.

5.2 Snooze collides with another reminder

Whichever is earlier becomes global next. At same instant, base occurrences precede retries/snoozes only if product policy says so; v1 orders by scheduled instant, then user-created base before retry, then UUID. No sound overlap.

5.3 Reminder due while another media item is playing

Show due notification. Locked full-screen behavior is evaluated from actual device state, but active Media Reminder playback counts as foreground; use in-app strip and notification. Do not stop accepted media automatically. User can pause and switch.

5.4 Reminder due during system alarm/call/navigation audio

The app requests appropriate audio focus and follows system/channel policy. It does not demand phone permissions or DND bypass. If communication mode is observed, reduce alert to noncompeting notification/vibration by default. Record Limited audio, not failure.

5.5 Rapid repeated actions

Native action nonce and database idempotency guarantee one result. All duplicate intents return resolved state and never create multiple snoozes.

6 Lock and presentation cases

6.1 Screen on but lock screen showing

Treat as locked. Full-screen eligible according to profile and platform.

6.2 Screen off but device not keyguard-secured

Treat non-interactive as locked-style presentation because the user is not actively using the phone.

6.3 Device is unlocked but user is idle

Still no forced full-screen. Android notification heads-up is the compliant surface. "Smart Presence" based on inferred idle time is deferred because it can surprise users and require unreliable heuristics.

6.4 Full-screen eligibility revoked

Post normal high-importance notification, start only compliant alert behavior, mark Health Limited and provide system-settings link/test. Do not repeatedly prompt at every occurrence.

6.5 Heads-up not shown

The notification remains in shade. Possible reasons include channel importance, DND, OEM ranking or user settings. Health test explains that app cannot control exact heads-up rendering. Persistent profile MAY re-alert within bounded policy but cannot launch full screen while unlocked.

6.6 Notification permission denied

Do not start an invisible ringing service. Reminder moves to Needs setup or, for a currently due previously active reminder, records BlockedByNotification and becomes visible next launch. No deceptive background audio.

7 Media cases

7.1 Original gallery file deleted

No effect after successful import because app owns a copy.

7.2 App-owned file missing

Integrity state Missing; disable attached reminders; due event stops alert and shows Media unavailable. Offer Locate replacement, Restore backup or Delete.

7.3 File changed/corrupted

Hash/probe mismatch marks Changed/Unsupported. Do not overwrite expected hash silently. Playback fails safe. User may replace and update expected hash through explicit repair.

7.4 Unsupported codec on new device

Import/restore can retain the file but mark Unsupported on this device. Reminder remains disabled until replacement or compatible player support. Export preserves original bytes.

7.5 Very long video

Playback is user-controlled. Alarm acceptance does not hold wake lock for entire video beyond normal media playback behavior. Completion semantics remain Accepted unless natural-end tracking is enabled.

7.6 Video contains loud audio

Alarm tone stops before media. Playback volume follows media stream. Preview and Play are explicit. No normalization in v1.

7.7 Media is private on lock screen

User setting can hide title/thumbnail and show "Media reminder." Full content appears only after unlock/Play. Direct-boot alert is generic.

8 Storage cases

8.1 Insufficient space during import

Stop streaming, delete temp, clear operation, preserve original library. Report estimated additional bytes. Do not partially insert media row.

8.2 Insufficient space during export

Destination may retain incomplete file depending provider. Mark operation incomplete, attempt cleanup, and clearly name the partial destination only when provider returns safe display name.

8.3 Insufficient space during restore

Reject before commit after considering staging, current data and rollback reserve. If space changes during commit, rollback. Never delete current media first to "make room" unless Replace plan and rollback are already safe.

8.4 SD card/document provider removed

App-owned media stays internal. Export/import stream fails with provider unavailable and current data remains intact. Custom tone URI fallback uses packaged tone.

8.5 Cache cleared

Regenerate thumbnails; no reminder or source media loss.

9 Backup cases

9.1 ZIP renamed or extension removed

Inspect content signature and manifest. Extension is not trusted.

9.2 Archive modified after inspection

Import token binds to staged archive digest. Commit rejects changed bytes/token expiry and requires reinspection.

9.3 Duplicate paths or case variants

Reject archive to avoid ambiguous extraction.

9.4 Checksum missing

v1 requires checksum file; reject as unsupported/incomplete, even when ZIP opens.

9.5 Some media missing

Reject before commit if manifest references absent files. A future partial-recovery tool may import valid independent records, but normal restore remains all-or-nothing.

9.6 Backup from newer version

Inspect and show counts only if safe manifest parse is possible, then block commit with update message. Never guess at unknown required semantics.

9.7 Merge conflict changed during review

Optimistic entity versions invalidate plan. Re-run inspection against current data.

9.8 Crash during replace

Startup sees operation journal, prevents normal mutation and completes rollback/forward recovery. Alarm scheduler is reconciled only after consistent data state.

10 Permission and settings cases

10.1 Exact access revoked after reminders scheduled

Permission-change receiver/startup check cancels exact pending alarm, marks affected reminders Limited/Needs setup, schedules an inexact fallback only if user preauthorized Limited mode, and shows one Health notice.

10.2 Notification channel sound changed by user

Treat system choice as effective. Profile page displays mismatch. Do not recreate channel.

10.3 App notification permission reset by hibernation

Health detects blocked notifications on next launch. No silent ringing. Active reminder intent remains preserved.

10.4 Battery saver/Doze

Exact path uses authorized system alarm. Limited path may delay. No battery exemption demand.

11 Interaction cases

11.1 Accidental Play then immediate Back

Alarm is already accepted and sound stopped. Returning does not restart alarm. History records Accepted; user can manually mark Dismissed only through details if desired.

11.2 Snooze duration crosses next base occurrence

Warn when custom snooze extends beyond next recurrence. Default behavior creates snooze and still preserves base occurrence, which may result in two future items; user can choose Skip next base for this cycle.

11.3 Delete reminder while due notification exists

Deletion transaction invalidates occurrence/session, stops service and cancels notification. Stale action returns already resolved.

11.4 Edit reminder while active

Active occurrence snapshot keeps its label/profile for current session; edits apply to future occurrences. Delete/disable explicitly resolves current session after confirmation.

11.5 System locale changes

Native alarm strings come from Android resources and update on next component creation. Stored user labels remain unchanged. Weekday display reorders; persisted ISO mask does not.

11.6 Font scale changes during active alarm

Activity recreates and restores session. Controls remain accessible; sound is not restarted.

12 Recovery screen

When an invariant prevents safe normal startup, show a native/React recovery screen with:

- what is affected;
- whether media/files remain present;
- Retry repair;
- Export recoverable backup where safe;
- Restore backup;
- Reset app as last resort;
- diagnostic code.

Reset is never the first action. Recovery export excludes known corrupt records and lists omissions.

13 Edge-case acceptance

Every scenario in this document has at least one automated or manual test ID in MR-21. Critical and high cases are release-blocking on all required API levels. OEM-only uncertainty is documented with observed devices and user-visible fallback rather than marked “not reproducible” without evidence.

13.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Security, Privacy and Threat Model

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-12
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define protected assets, trust boundaries, threats, controls, privacy behavior, secure development and incident response.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Security and privacy posture

Media Reminder is intentionally offline and account-free. This reduces remote attack surface but does not eliminate risk: the app processes untrusted media and ZIP archives, exposes notification actions, holds private personal content, and interacts with Android components while the process may be absent.

The production build MUST have no Internet permission. Android automatic cloud backup and device-transfer extraction of app data are explicitly excluded so imported media is not silently moved outside the user's manual export workflow.

2 Protected assets

Asset	Sensitivity	Protection objective
Imported media	Potentially highly private	Confidential inside Android sandbox; never upload; controlled sharing only
Reminder labels, notes and schedules	Private behavioral data	Least lock-screen disclosure; local storage; backup warning
Backup archives	Plaintext copy of private data	Explicit export, validation, scoped URI grants, clear custody warning
Alarm actions	Integrity-critical	Authenticated session/nonce, immutable PendingIntents, idempotent transitions

Asset	Sensitivity	Protection objective
Scheduler state	Integrity/availability-critical	Room + generation validation; no arbitrary external control
Signing key	Release-critical secret	Offline/secure storage outside repository and CI logs
Source and dependencies	Supply-chain asset	Review, pinning, SBOM, license and vulnerability process
Diagnostics	Potential metadata	Bounded, redacted and user-previewed before sharing

3 Trust boundaries

1. **User and Android system UI** - system picker/settings/notification surfaces are trusted to grant scoped choices.
2. **External content provider** - selected media/document URI is untrusted input even when chosen by the user.
3. **App sandbox** - Room and app-owned files are trusted only after integrity validation.
4. **Backup archive** - completely untrusted until staged and validated.
5. **Android IPC** - broadcasts, intents, PendingIntents and FileProvider URIs require explicit package/token checks.
6. **React Native bridge** - internal but treated as a typed boundary; JavaScript cannot request arbitrary file/system operations.
7. **Release pipeline** - build workers and dependency registries are supply-chain boundaries.

4 Threat actors and misuse

- another installed app sending crafted explicit/implicit intents;
- a malicious or malformed media file exploiting parser behavior;
- a malicious backup intended for traversal, resource exhaustion or record injection;
- accidental sharing of a plaintext archive;
- a person with unlocked physical device access;
- compromised dependency or build pipeline;
- developer mistakes such as exported receiver, mutable PendingIntent, sensitive log or broad URI grant;
- OEM/platform behavior that exposes more notification text than expected.

The product does not attempt to defend against a fully compromised operating system or an attacker who controls an unlocked device and can use the app normally. It does reduce accidental disclosure and cross-app manipulation.

5 Privacy requirements

- **SEC-001:** No production code path may send network traffic; manifest lacks `INTERNET` and `ACCESS_NETWORK_STATE` unless a future ADR changes scope.
- **SEC-002:** No account, advertising identifier, analytics SDK, crash-upload SDK or remote configuration.
- **SEC-003:** Media, labels, notes and schedule values **MUST NOT** appear in diagnostic export.
- **SEC-004:** Lock-screen notification visibility defaults to Private; user may choose generic Secret content.
- **SEC-005:** Automatic Android cloud backup and device transfer **MUST** exclude Room, DataStore, app media, diagnostics and device-protected schedule envelope.
- **SEC-006:** User-initiated ZIP export **MUST** warn that v1 archives are readable by anyone who obtains them.
- **SEC-007:** Sharing uses the Android chooser and read-only, temporary URI grants; no public filesystem directory is exposed.
- **SEC-008:** Clearing history must not delete reminder definitions, and deleting a reminder must not imply secure erasure from flash backups.

6 Android backup exclusion

Use both legacy and modern controls appropriate to supported APIs:

- `android:allowBackup="false"` as a conservative baseline;
- `android:dataExtractionRules` excluding `root`, `file`, `database`, `sharedpref`, `external` and device-protected domains from cloud backup and device transfer;
- legacy `fullBackupContent` exclusion file for older APIs;
- automated APK/manifest test and restore test to confirm no app-owned media appears through platform backup paths.

The manual `.mrbackup.zip` format is the supported migration mechanism. If a future release opts into operating-system transfer, it requires a privacy ADR and user-facing disclosure.

7 Exported component policy

All receivers, services, activities and providers are `android:exported="false"` unless platform integration strictly requires otherwise. `FileProvider` is exported false and grants URI permissions. Intent filters are minimized. Runtime handlers verify:

- action string is exact;
- package/component is expected;
- session/operation ID exists;
- nonce/token is valid and unexpired;
- current database state permits the transition;
- extras have type/length bounds.

Any impossible intent stops active sound where relevant, logs a safe diagnostic and returns without mutation.

8 PendingIntent security

PendingIntents are immutable by default. Use explicit package/component. Request codes are generated through a collision-tested factory, and semantic authorization still comes from session/nonce in Room, not obscurity of request code. FLAG_UPDATE_CURRENT is used only with immutable payload ownership; stale generations are rejected.

9 File and URI security

- System picker URIs are read only.
- Persistable grants are used only for custom user-selected tone where necessary; imported media is copied and the grant released.
- User display names are metadata, not paths.
- Internal filenames are UUIDs with a normalized allowlisted extension.
- Path resolver canonicalizes and confirms target remains inside expected root.
- FileProvider exposes only completed export paths declared in XML, never database/media roots broadly.
- Temporary grants are revoked or naturally expire after share; no write grants.

10 Media input handling

Media is untrusted. Controls:

- stream copy with size limits;
- MIME header sniff plus platform probe, not extension alone;
- image bounds decode before full decode;
- no embedded HTML/SVG/script execution in v1;
- Media3/platform codecs kept current through dependency updates;
- preview and thumbnail generation catch decoder failures and enforce pixel/duration limits;
- malformed content cannot change reminder database state until source copy and validation complete;
- fuzz/crash fixtures are retained in security tests when licenses permit.

A codec exploit can still exist in the platform/library. Dependency and Android security update guidance is included in release notes.

11 Backup attack controls

The archive reader treats every byte as hostile:

- no direct extraction before central-directory inspection;
- reject absolute paths, traversal, NUL, symlink, duplicate and case-collision entries;
- canonical destination check after normalization;
- entry count, size, ratio, JSON depth and string limits;
- stream hashing with bounded buffers;
- schema allowlists and enum validation;

- reference graph validation;
- staged token binds to archive digest;
- commit through repositories, never SQL supplied by archive;
- no recursive nested archive extraction;
- current data unchanged until complete validation and reviewed conflict plan.

12 Database integrity and injection

Room parameterized queries are mandatory. Search uses bound parameters and escaped wildcard semantics. There is no dynamic SQL from archive or user strings. Check constraints and domain validation defend against impossible values. Database files are never shared.

13 Lock-screen and screen capture

Default notification content is private. The user can set **Hide names on lock screen**, producing generic title and no thumbnail. The direct-boot envelope contains only opaque IDs and next due instant.

A privacy setting MAY enable `FLAG_SECURE` for media preview/player and blur sensitive content in Recents. It is not forced globally because users may intentionally capture their own content and portfolio screenshots. Alarm controls never expose notes on the lock screen.

14 Backup confidentiality

V1 ZIP is intentionally plaintext for reliability and portability. The export confirmation uses direct language and requires a user-selected destination. The app does not remember or auto-export to a cloud folder. Encryption is P1 and must use a reviewed format with authenticated encryption, memory-hard password derivation and no recoverable developer key.

15 Local encryption

Android file-based encryption and app sandbox are the v1 at-rest baseline. Database-level encryption is not required for MVP because it adds key management, migration and native risk while unlocked app content remains accessible. A future optional app lock/encrypted vault requires a separate ADR and must preserve alarm operation without exposing media before unlock.

16 Logging and diagnostics

Release logs use structured codes. Forbidden fields: asset title, file name/path, notes, category/tag names, ZIP destination, exact schedule label and raw URI. Allowed: UUID hashed/truncated for correlation, phase, duration, byte count, OS/API, device model, capability enum, error code and app version.

Logcat output is minimal in release. Diagnostic export is previewed, bounded and initiated by the user. Crash uploads are absent.

17 Supply-chain security

- lock JavaScript and Gradle dependency versions;
- verify checksums/signatures through supported package mechanisms;

- prefer primary AndroidX and React Native dependencies;
- generate an SBOM for each release;
- run dependency vulnerability and license scans in CI;
- remove unused transitive libraries and permissions;
- protect branches/tags and require review for workflow/signing changes;
- use reproducible build inputs where possible;
- release signing key never enters source control, chat prompts, screenshots or public CI logs.

18 Threat matrix

Threat	Control	Residual risk
ZIP traversal	canonical path validation, no direct extraction, malicious fixtures	parser/platform implementation flaw
ZIP bomb	entry/total/ratio limits, streaming, preflight space	crafted archive near limits consumes time
Intent replay	immutable explicit PendingIntent, nonce, Room idempotency	device compromise
Unlocked full-screen annoyance	trigger-time state check, no overlay, FSI omitted when unlocked	OEM notification behavior varies
Private lock-screen disclosure	private/secret visibility, generic direct boot	user/system settings can expose notification
Backup accidentally shared	clear warning, chooser, no auto-upload	user can still select public destination
Media parser exploit	allowlist, size/probe, updated Media3/platform	zero-day codec flaw
Data loss during restore	staging, checksums, rollback journal	device/storage failure during both data and rollback
Automatic cloud upload	backup/data extraction exclusions	OEM nonconformance must be tested
Dependency compromise	pinning, SBOM, review, scans	registry/build environment compromise

19 Privacy notice outline

The shipped privacy notice states:

- what is stored: imported content, reminder settings, optional history and local diagnostics;

- where: app storage on the device and user-selected backup destinations;
- what is transmitted: nothing by the app;
- permissions and their purpose;
- Android/OEM ownership of notification and alarm settings;
- deletion behavior, uninstall and backup custody;
- open-source repository and contact route without requiring personal data.

20 Security testing and response

Critical controls are covered by static manifest inspection, exported-component tests, PendingIntent replay tests, archive fuzz fixtures, path traversal tests, dependency scans and manual privacy review. A private vulnerability reporting method is documented in `SECURITY.md`. Security releases revoke compromised artifacts through release notes, publish new checksums, and never request users send private backup/media unless they have independently redacted it.

21 Security release gate

Release is blocked by any unreviewed exported component, Internet/overlay/broad storage permission, plaintext sensitive log, bypassable archive validation, missing rollback on Replace, mutable externally reachable PendingIntent, or inability to stop ringing after invariant failure.

21.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Accessibility, Localization and Inclusive Design

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-13
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define accessible interaction, assistive technology behavior, dynamic layout, language readiness, RTL support and inclusive content.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.

Term	Meaning
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Accessibility target

Media Reminder targets WCAG 2.2 AA principles adapted to native Android and follows Android accessibility guidance. The success condition is task completion, not merely automated scanner scores. Import, reminder creation, permission recovery, alarm action and backup restore must work with TalkBack and non-touch input.

2 Core requirements

- **ACC-001:** Every actionable element has a meaningful accessible name, role, state and enabled status.
- **ACC-002:** Minimum target size is 48 x 48 dp; full-screen alarm actions are at least 56 dp high.
- **ACC-003:** Text supports at least 200% system font scaling without loss of critical content or action.
- **ACC-004:** Color is never the sole indicator of enabled, due, completed, error or capability state.
- **ACC-005:** Normal text contrast is at least 4.5:1 and large text/UI boundaries meet applicable contrast requirements.
- **ACC-006:** Reduced-motion preference removes nonessential movement and looping animation.
- **ACC-007:** The application remains operable in portrait, landscape, split screen where permitted and responsive width classes.
- **ACC-008:** Notifications and native alarm controls use localized explicit action labels.
- **ACC-009:** Timeout behavior provides sufficient time and can be extended through profile configuration, while still retaining a hard safety cap.
- **ACC-010:** No content flashes in a seizure-risk pattern.

3 TalkBack semantics

3.1 Reminder row

Announce as one logical group: "Morning remembrance. Daily at 6:15 AM. Standard alert. Enabled. Next tomorrow." Provide separate actions for Open and Toggle. The switch must not be nested ambiguously in a fully clickable row.

3.2 Media card

"Video. Morning remembrance. 1 minute 33 seconds. Two active reminders." Thumbnail is decorative when the title conveys content; otherwise provide useful alt text entered by user. Do not announce filenames.

3.3 Capability card

"Exact timing. Action needed. Android may deliver reminders late. Button, Open settings." Status precedes action consequence.

3.4 Progress

Import/export progress announces phase changes and meaningful percentage thresholds, not every byte. Cancellation state and success are live-region announcements. Long operations remain discoverable after navigating away.

4 Alarm accessibility

The native alarm activity is the highest-priority accessibility surface.

- Initial focus lands on title/time summary, not Play automatically.
- Time and title are announced once; no repeated focus reset while sound vibrates.
- Focus order: summary, Play, Snooze, Dismiss, more options.
- Buttons have text labels, not icons alone.
- Snooze sheet announces absolute resulting time before confirmation.
- Hardware Back behavior is explained and does not trap the user.
- TalkBack gestures can reach a **Silence sound** action even when the user does not wish to resolve the reminder immediately.
- The screen survives 200% font and display size; if necessary, content scrolls while the three actions remain sticky and reachable.
- Vibration is not the only alert channel; visual and accessible notification content remain.

5 Notification accessibility

Notification title is concise and action labels are distinct. Avoid three actions beginning with the same word. When lock-screen privacy hides content, generic copy remains meaningful: "Media reminder due. Play after unlocking, Snooze, or Dismiss."

Android controls announcement timing and heads-up layout. The app tests with TalkBack but cannot force another app or OEM to announce in a specific order.

6 Dynamic type and layout

Text uses `sp`, layout grows vertically, and fixed-height containers are avoided for text. Truncation is allowed only for noncritical metadata and includes a detail view. Times may remain one line where the font role scales down within an accessible minimum; action labels never reduce below body-readable size to fit.

At large font scale:

- bottom navigation labels remain visible or switch to navigation rail/drawer on wider layouts;
- chip rows wrap/scroll with clear focus;
- editor labels remain above fields;
- in-app strip transforms into a compact card rather than clipping;
- dialogs may become full-screen sheets.

7 Motor and switch access

All functions have tap/button alternatives; no swipe-only delete, drag-only reorder or long-press-only command. Swipe gestures are enhancements. Focus indicators are visible. Keyboard Tab/arrow/Enter/Back behavior is tested on emulator and hardware keyboard. Reorder actions use Move up/Move down accessibility actions.

8 Color and visual perception

Status always combines icon, word and color. Error copy appears next to the field and in an accessible summary on submit. The design avoids low-opacity text for essential content. Dark mode is designed, not mechanically inverted. Thumbnail scrim maintain label contrast independent of image content.

A high-contrast mode may increase outlines and remove translucent surfaces. Android system contrast preference is followed where accessible through platform APIs; otherwise the app offers a setting.

9 Hearing and audio alternatives

An alarm can combine sound, vibration and visual presentation. Users may disable sound without disabling the reminder. Videos may not contain captions; the app allows notes/transcript text but does not claim automatic captioning in v1. Demo content used publicly includes captions/transcript where speech is important.

Custom vibration patterns are bounded and previewable. “Stronger haptics” is conditional on device capability and does not imply guaranteed intensity.

10 Cognitive accessibility

- One primary action per standard screen.
- Repeat rules are summarized in plain language.
- Permission screens say consequence and next action.
- No guilt, loss aversion, red failure marks for missed personal practices or competitive streaks.
- Destructive dialogs identify exact counts.
- Backup Replace uses two-step confirmation and rollback explanation.

- Technical codes are behind Details.
- Editor preserves entered values after a permission or validation issue.

11 Localization architecture

All user-visible strings, including native receivers/services/activities and notification channel names, live in localization resources. Concatenated sentences are prohibited. Use ICU/plural-aware formatting, locale date/time formatting, and translator comments for placeholders.

Persist stable semantic values, never localized display strings. Examples:

- weekday stored as ISO number, displayed locally;
- profile built-in ID stable, name localized;
- status stored `needs_setup`, displayed translated;
- archive wire values remain English stable tokens and are not localized.

12 Language roadmap

V1 ships English. The codebase is localization-ready from the first commit. Tamil and Arabic are the first planned additional languages because they align with the creator and originating use case. Release requires native-speaker review; machine translation alone is not accepted for alarm, permission, destructive or backup copy.

Religious terms may be user content. The generic app avoids asserting translations of duas or scripture. Any optional starter content has separate scholarly/content review and provenance.

13 RTL support

Arabic testing includes:

- mirrored navigation and directional layouts;
- nonmirrored media playback controls where platform convention requires;
- correct icon mirroring for arrows, not Play symbols;
- mixed Arabic/Latin filenames and times;
- numeral system behavior follows locale preference;
- thumbnail/text alignment and ellipsis;
- notification and native alarm resources;
- backup JSON remains direction-neutral and UTF-8.

Use start/end, not left/right, in layout code except intrinsically directional media controls.

14 Date, time and calendar

The app follows system 12/24-hour preference and locale date formatting. Recurrence calculations use Gregorian civil date/time in v1. Islamic calendar display is a future optional layer and MUST not alter schedule semantics without explicit design. The app avoids assuming Sunday or Monday as first day of week in display.

15 Content guidelines

Use simple verbs: Play, Snooze, Dismiss, Save, Export, Import. Avoid “Accept alarm.” Explain “exact timing” as a platform capability. Avoid idioms difficult to translate. Error messages state effect and recovery. Sentence case is standard.

16 Accessibility test matrix

Every release verifies:

- TalkBack latest on one reference Pixel and one Samsung;
- Switch Access or keyboard navigation for core flows;
- font scale 100%, 130%, 160%, 200%;
- display size default and largest practical;
- light/dark/high contrast;
- grayscale/color-blind simulation for status comprehension;
- reduced motion;
- portrait/landscape/fold width;
- English pseudo-localization with 40% string expansion;
- RTL pseudo-locale and Arabic sample;
- notification actions and full-screen alarm while TalkBack is running.

17 Accessibility acceptance

A release cannot waive a blocked Play/Snooze/Dismiss action, clipped alarm control, unlabeled permission action, color-only health state, focus trap or inaccessible destructive confirmation. Automated findings are triaged, but manual task testing is mandatory.

17.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Testing, Quality and Device Matrix

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-14
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define automated and manual quality strategy, device/API coverage, reliability simulations, security tests, evidence and release gates.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Quality objective

Testing must prove the app is safe under Android lifecycle pressure, not only that screens render. The highest-risk surfaces are native alarm actions, adaptive presentation, scheduling reconciliation, file transactions and backup restore. Test architecture mirrors those boundaries.

2 Test layers

Layer	Scope	Tools/approach
Pure unit	Recurrence, DST, conflict plan, validators, ID factories	Kotlin/JVM and TypeScript unit tests
Database	DAO constraints, transactions, migrations, query plans	Room in-memory/file tests and migration fixtures
Native component	receivers, services, activity, notification, permissions	Android instrumentation and fake system adapters where possible
React component	forms, a11y semantics, state rendering	React Native Testing Library
End-to-end	import-create-trigger-act-export-restore	Physical/emulated device automation plus manual checkpoints
Security/fuzz	ZIP/media/intent malformed inputs	Curated malicious fixtures, property tests, fuzz harnesses
Performance/battery	latency, wakeups, memory, storage	Android profiling/dumpsys,

Layer	Scope	Tools/approach
		scripted measurements
Visual/accessibility	themes, scale, TalkBack, rotation	Screenshot diff plus human review

Coverage percentage is informative. Release confidence comes from requirement traceability and risk-based evidence.

3 Required API matrix

API	Android generation	Primary purpose
26	Android 8.0	Minimum SDK, channels, background limits baseline
28	Android 9	legacy lifecycle/storage behavior and common older device
31	Android 12	exact-alarm special access, PendingIntent mutability
33	Android 13	notification runtime permission, modern media picker path
34	Android 14	exact access default changes, full-screen eligibility restrictions, FGS declarations
36	Android 16	initial target behavior and Play deadline baseline
37	Android 17	compile/current compatibility, background audio hardening verification

Emulators cover deterministic API permutations. Physical devices are mandatory for lock screen, audio routing, vibration, Doze, reboot, OEM notifications and battery.

4 Physical device matrix

Minimum release evidence:

- one current Pixel/reference Android device;
- one Samsung device with current One UI;
- one older API 26-28 physical device or verified lab device;
- one aggressive-background-management OEM family such as Xiaomi, Oppo, Vivo or equivalent available device;

- one foldable/tablet width test, physical or high-fidelity emulator;
- one low-memory 4 GB-class device.

Document exact model, OS build, security patch and channel settings in QA report. A device unavailable for a release is explicitly marked with risk acceptance; critical adaptive alarm tests cannot be waived on both reference and Samsung.

5 Test data

Use synthetic/original fixtures:

- short MP4/H.264/AAC;
- long and large sparse/test media within license;
- MP3/AAC/Opus where supported;
- JPEG/PNG/WebP with varied dimensions/rotation;
- corrupt headers, truncated files, extension/MIME mismatch;
- Unicode/RTL/Tamil/emoji titles;
- 10,000-reminder generated database;
- 5 GB streaming backup fixture;
- every prior public schema/archive version;
- malicious ZIP fixtures: traversal, absolute, duplicate, case collision, symlink, bomb ratio, bad checksum, deep JSON, unresolved references, higher major version.

No private personal video is committed to the repository.

6 Unit suites

6.1 Recurrence

Cover once, daily, weekday masks, leap day, month/year boundary, timezone travel, 30/45-minute offsets, DST gap/overlap, clock rollback cursor, grace window and snooze beyond base occurrence. Property: returned occurrence is strictly after cursor and unique per occurrence key.

6.2 Conflict planning

Combinations of same/different UUID, hash, normalized names, profile built-ins, semantic reminders and entity-version changes. Property: every archive record is mapped to import, reuse, skip, replace or explicit conflict; no unresolved reference reaches commit.

6.3 Validators

String length, UUID, MIME header, storage key, URI token, ZIP path normalization, ratio/size arithmetic overflow, JSON depth and action nonce.

7 Native alarm test plan

7.1 Presentation matrix

For Gentle/Standard/Persistent, test:

- screen off + secure lock;

- screen on + lock screen;
- unlocked home screen;
- unlocked while another app foreground;
- Media Reminder foreground;
- full-screen permission eligible/ineligible;
- notification permission granted/denied;
- channel high/default/blocked;
- exact access granted/denied;
- TalkBack on;
- portrait/landscape;
- Doze and battery saver.

Assertion: no unlocked case launches `AlarmActivity`; locked eligible cases do; all fallbacks are visible and actionable.

7.2 JS independence

Build/instrument a mode that prevents React Native initialization. Trigger an occurrence and verify native Play, Snooze, Dismiss, timeout, service stop and reschedule. This is P0 evidence.

7.3 Lifecycle

Kill process, swipe app away, reboot, package update, timezone change, manual clock change, rotate alarm, service recreation and low-memory pressure. Force-stop is tested for correct limitation copy after manual reopen, not impossible wake behavior.

7.4 Soak

Run at least 100 scheduled trigger/action cycles across profiles with random process kills and actions. Assert no duplicate occurrences, stale notifications, service leaks, wake-lock leaks or scheduler generations left unapplied.

8 Media and file tests

- Delete original after import; playback still works.
- Cancel at each copy phase.
- Fill storage during copy.
- Crash after temp complete, after rename and before/after DB commit.
- Change/delete internal file through debug harness and run repair.
- Decode thumbnail failure while source remains valid.
- Unsupported codec restore on another device.
- Two simultaneous import attempts are serialized/bounded.
- FileProvider share grants read only and expires.

9 Backup tests

9.1 Round trip

Export fixture, clean app data, import Replace, compare logical canonical JSON and every media SHA-256. Repeat across supported app versions and devices.

9.2 Merge

Run every conflict class and verify reviewed plan exactly matches committed mapping. Change local data after preview and assert token invalidation.

9.3 Fault injection

Terminate process at every restore phase, restart, and assert documented rollback/forward recovery. Simulate provider removal and disk full.

9.4 Security

No malicious fixture modifies current data or writes outside private staging. Peak memory remains bounded. Parsing time has a security timeout/complexity budget.

10 Accessibility and visual tests

Automated scanner/lint checks are supplemented by manual TalkBack scripts. Screenshot baselines cover key screens at light/dark, compact/medium, English/RTL pseudo-locale and 100/200% scale. Visual diffs are reviewed, not auto-approved solely by threshold.

Alarm action labels, focus order, announcement repetition and vibration/audio alternatives receive manual signoff.

11 Battery and performance tests

- 24-hour no-due-reminder idle comparison against clean baseline;
- 24-hour with representative 8 due reminders;
- inspect `dumpsys alarm`, service state, wake locks and battery attribution;
- receiver-to-alert timestamp instrumentation;
- action-to-sound-stop instrumentation;
- cold/warm startup on reference low/mid device;
- 2 GB import and 5 GB backup streaming memory;
- 10,000 reminders query/reconciliation.

Measurements follow MR-15 and include at least three runs after warm-up where applicable.

12 Release regression suite

P0 automated suite includes:

1. database migrations;
2. recurrence and DST properties;
3. alarm action idempotency;
4. manifest permission/exported-component checks;

5. backup malicious fixtures;
6. clean round trip;
7. native alarm JS-disabled instrumentation;
8. locked/unlocked presentation;
9. wake/service leak assertions;
10. accessibility smoke.

A tagged release cannot be built from a commit that did not pass the release workflow. Manual evidence is attached to the release QA record.

13 Defect policy

Severity	Release rule
Critical	Zero open; release artifact withdrawn if discovered
High	Zero open for data, alarm action, privacy, adaptive presentation or a11y; exceptional cosmetic high requires owner signoff
Medium	May ship only with documented workaround and no requirement breach
Low	Triaged into backlog

Flaky tests are defects. Quarantining a P0 alarm/backup/security test requires explicit issue, owner and replacement evidence; it cannot remain silently skipped.

14 Test environment controls

Record notification permission, exact access, FSI eligibility, channel importance, DND, battery saver, timezone, locale, lock type, charger state and manufacturer optimization settings. Reset these between cases. Test notification channels use versioned IDs to avoid previous user choices contaminating results.

15 QA evidence package

Each release produces:

- requirement traceability results;
- device matrix and OS builds;
- automated test summary;
- migration/archive compatibility matrix;
- battery/performance measurements;
- accessibility checklist;
- security scan/SBOM summary;
- known limitations;
- release APK/AAB hashes and source commit.

16 Exit criteria

P0 traceability is green, all critical/high blockers closed, physical locked/unlocked tests passed, 100-cycle soak passed, round-trip backup passed, no excluded permission present, idle architecture evidence passed and representative PDFs/docs correspond to the release design.

16.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Performance, Battery and Operational Budgets

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-15
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define measurable latency, battery, memory, storage, scalability and reliability budgets with evidence procedures.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.

Term	Meaning
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Budget philosophy

Battery efficiency is a product requirement. The strongest evidence is structural and measured: no resident idle service, no polling loop, one next system alarm and bounded work at trigger. Percentage battery readings alone are noisy, so release evaluation combines architecture assertions, system wake/service inspection and controlled comparisons.

2 Reference device classes

- **Low:** API 26-28, 4 GB RAM, midrange storage.
- **Reference:** current supported Pixel/Samsung-class device, 6-8 GB RAM.
- **High:** current flagship used only to identify headroom, not set minimums.

Budgets apply to release builds with synthetic local data. Thermal throttling, charging and external background activity are controlled or recorded.

3 Latency budgets

Operation	Target	Hard ceiling	Measurement
Alarm receiver entry to notification posted	≤ 500 ms p95	2,000 ms	Native monotonic timestamps
Play/Snooze/Dismiss tap to alarm sound stopped	≤ 150 ms p95	1,000 ms	Action receiver/service timestamp

Operation	Target	Hard ceiling	Measurement
Locked activity visible after receiver entry	<= 900 ms p95	2,500 ms	Instrumented frame/activity callback
Cold launch to usable shell	<= 2.0 s p95 reference	3.5 s low	Macrobenchmark/frame metrics
Warm launch to usable shell	<= 750 ms p95	1.5 s	Macrobenchmark
Today query, 1,000 reminders	<= 50 ms p95	150 ms	Repository benchmark
Earliest occurrence query, 10,000 reminders	<= 100 ms p95	250 ms	Room benchmark
Full scheduler reconciliation, 10,000 reminders	<= 250 ms p95	1 s	Native benchmark
Notification action state visible in UI after app open	<= 500 ms p95	1.5 s	End-to-end

The alert is posted before nonessential logging, thumbnail lookup or React events. Personal media metadata cannot block a stop action.

4 Idle battery budgets

4.1 Structural invariants

- zero foreground services when no alarm/media/explicit export is active;
- zero held wake locks when idle;
- zero JavaScript intervals used to observe time;
- zero periodic WorkManager jobs for due delivery;
- at most one ordinary next reminder AlarmManager registration;
- no network sockets because Internet permission is absent;
- no startup full-library hash scan.

Any violation is a release blocker regardless of battery percentage.

4.2 Measured target

In a controlled 24-hour test with active reminders configured but none due, Media Reminder SHOULD show no statistically meaningful additional battery drain compared with the same device/app-data state without scheduled reminders, and MUST remain below 0.5 percentage points of device battery capacity attributable to the app on reference devices. Because attribution is noisy, failure requires repeated runs and system wake evidence; pass still requires structural invariants.

With eight representative due reminders resolved within one minute, incremental energy is expected only around alert/playback. Media playback energy is reported separately from scheduler idle cost.

5 Wakeup budgets

- No periodic wakeups.
- One OS alarm wake per due occurrence, plus one per explicit snooze/retry/test.
- Boot/time/capability reconciliation occurs only on corresponding broadcasts/startup.
- Deferrable maintenance is opportunistic and constrained; default frequency no more than weekly if enabled.
- A single due event must not create repeated wake alarms unless its profile retry policy explicitly schedules them, capped at three.

`dumpsys alarm` evidence must show no accidental duplicate alarm identities after stable state.

6 Wake-lock and service budgets

Resource	Budget
Receiver bridge wake lock	≤ 10 seconds absolute; normally < 2 seconds
Active alarm wake lock	Only while ringing; $\leq$ profile timeout; hard cap 10 minutes
Ringing foreground service	Starts only for continuous alert; stops ≤ 1 second after terminal action
Import/export foreground service	Avoid in MVP by foreground modal operation; if later used for user-initiated long work, bounded to operation and cancelable
Media playback service	Not persistent in MVP; playback activity owns player lifecycle unless background playback is explicitly added later

Static/runtime tests verify acquisition and release in `try/finally` or session coordinator.

7 Memory budgets

Scenario	Budget
Usable app shell PSS on reference device	≤ 220 MB p95
Library with 100 visible thumbnails	≤ 260 MB p95; thumbnail cache bounded
Native alarm path without RN	≤ 90 MB incremental process PSS target
Media player 1080p local video	≤ 350 MB p95 excluding platform codec variation
Import/export additional heap	≤ 32 MB regardless of file size
Backup JSON parse	streaming or bounded; ≤ 64 MB additional at 20k records

Out-of-memory prevention is more important than exact PSS across OEMs. Large byte arrays of media/ZIP are forbidden by code review/lint patterns.

8 Storage budgets

- Signed arm64 APK target ≤ 35 MB; universal APK ≤ 60 MB, excluding user media.
- Installed code/native libraries target ≤ 120 MB.
- Database at 10,000 reminders and 90-day history target ≤ 50 MB.
- Thumbnail cache ≤ 250 MB and clearable.
- Diagnostics ≤ 5 MB.
- Pending/rollback storage is visible during backup and cleaned after completion/recovery.
- Imports maintain 250 MB or 5% free reserve after operation.

If React Native/Media3 version changes exceed package budget, release notes explain and optimization issue is opened; functionality/security updates can justify reviewed exceptions.

9 File throughput

No universal speed target is promised because document providers and storage vary. Requirements:

- streaming buffer 256 KB-1 MB per active stream, bounded;
- UI progress at least every 500 ms and not more than 10 updates/sec;
- cancellation checked between chunks;
- hashing occurs in the same streaming pass where possible;
- 2 GB import and 5 GB export/restore complete without heap growth proportional to file size;
- no main-thread file I/O.

10 UI rendering budgets

- scrolling library targets 60 Hz on reference device with no repeated full-size image decode;
- slow/frozen frames tracked in benchmark screens;
- initial Library loads first page, not all media;
- thumbnails use size-aware decoding and LRU cache;
- forms avoid unnecessary bridge round trips per keystroke;
- alarm activity is native and renders from a compact session snapshot.

11 Database scaling

Supported design target:

- 10,000 reminders;
- 20,000 media records subject to storage;
- 50,000 retained occurrences;
- 5,000 categories/tags combined;

- 20,000 ZIP entries.

Pagination is keyset/cursor-based where stable. Search may use Room FTS after P1 profiling; MVP LIKE must remain within tested budgets at target scale or FTS is pulled forward.

12 Reliability budgets

Measure	Target
Duplicate terminal action per occurrence	0
Unresolved active session after timeout/relaunch	0
Backup round-trip hash mismatch	0
Data-loss crash-injection cases	0
Open wake/service leaks in 100-cycle soak	0
Alarm action native availability	100% scripted tests
Unsupported archive modifying current data	0

13 Measurement procedure

1. Use release/QA build with fixed fixture and diagnostics enabled locally.
2. Reset device state, channel settings and app data as defined per test.
3. Run warm-up where relevant.
4. Capture at least three runs; latency reports p50/p95/max where enough samples exist.
5. Record model, OS build, charger, thermal state, battery saver, DND and network state.
6. Inspect AlarmManager registrations, service list and wake locks before/after.
7. Store raw reports in release evidence; publish summarized nonpersonal numbers.

14 Regression thresholds

A change fails performance CI/manual gate when a load benchmark regresses more than 15% and crosses a target, when any hard ceiling is exceeded, or when structural battery invariants break. Security/correctness fixes may receive a documented temporary exception with follow-up issue, but never for an unbounded service/wake lock or inaccessible stop action.

15 Optimization priorities

1. Stop/action latency and correctness.
2. Idle wake/service elimination.
3. File streaming and memory bounds.
4. Startup shell and capability snapshot.
5. Library scrolling and thumbnail cache.

6. APK size and noncritical animation polish.

This order prevents visible benchmark chasing from weakening alarm safety.

15.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Roadmap, Release and Success Metrics

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-16
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define implementation increments, exit gates, future scope, local success measurement and change management.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Delivery strategy

Build vertical, testable increments that establish the native reliability spine before visual breadth. Each milestone ends with a tagged internal build and evidence. A feature is not moved forward with unresolved data-integrity or alarm-action debt.

2 Milestone 0 - repository and decisions

Deliver:

- React Native 0.86.x Android project with New Architecture;
- Kotlin module skeleton and Codegen contract;
- Room/DataStore setup;
- CI for lint, type check, unit tests and manifest policy;
- source-of-truth specs, ADRs, contribution and security files;
- synthetic media fixtures and no Internet permission.

Exit: debug and release variants build; native contract smoke passes; repository rules prevent prohibited permissions.

3 Milestone 1 - app-owned media library

Deliver import through system picker, staged copy/hash/probe, Room asset records, thumbnail cache, Library and media detail, delete dependency policy and repair journal.

Exit: original source deletion does not break playback; crash injection passes each import phase; unsupported/low-space cases safe; TalkBack library flow usable.

4 Milestone 2 - reminder domain and scheduler

Deliver profiles, reminder editor, once/daily/weekdays recurrence, occurrence calculator, one-global-alarm coordinator, exact access/limited state and Today/Reminders screens.

Exit: DST/timezone unit suite passes; 10,000 reminder reconciliation budget met; stable OS alarm count is one; permission denial preserves intent.

5 Milestone 3 - native adaptive alarm

Deliver receiver, action receiver, notification channels, bounded ringing service, native alarm/player activities, device-state decision, foreground in-app strip, boot/time recovery and Test reminder.

Exit: React Native-disabled Play/Snooze/Dismiss passes; no unlocked full-screen launch; locked eligible full-screen works; service/wake lock stops; 100-cycle soak passes on reference and Samsung.

6 Milestone 4 - backup and recovery

Deliver logical ZIP export, checksums, inspection, malicious archive validation, Merge/Replace conflict plan, rollback journal, capability re-evaluation and user-selected save/share.

Exit: clean-device round trip equality/hash passes; all malicious fixtures reject; crash at every phase recovers; 5 GB stream stays in memory budget.

7 Milestone 5 - accessibility, privacy and polish

Deliver final design tokens/components, dark/system theme, 200% scaling, TalkBack, RTL readiness, local diagnostics, privacy notice, platform backup exclusions and complete Health flows.

Exit: accessibility manual script passes; privacy/manifest audit passes; screenshots contain only safe original/synthetic content; copy makes Android limits clear.

8 Milestone 6 - release candidate

Deliver signed candidate, SBOM, release QA package, migration/archive matrix, battery evidence, APK checksums, changelog, portfolio case study and known limitations.

Exit: MR-21 P0 green, no critical/high blockers, physical matrix complete, documentation and source tag aligned.



9 Version 1.0 scope lock

V1 contains video/audio/image import, once/daily/weekdays reminders, three profiles, adaptive locked/unlocked presentation, snooze/dismiss/play, offline storage, Health, logical ZIP backup, accessibility baseline and GitHub APK distribution. Text cards may ship if they do not delay the reliability gate; otherwise they move to 1.1.

No cloud sync, accounts, overlay permission, prayer-time calculation, automatic social media downloads, iOS, Wear OS or emergency claims are accepted into 1.0.

10 Post-release roadmap

10.1 1.1 - quality and expression

- text cards;
- custom user profiles;
- richer local search and duplicate hash handling;
- Tamil localization;
- optional local history summaries;
- widget feasibility.

10.2 1.2 - portability and language

- passphrase-encrypted archive v2 with migration;
- Arabic localization and full RTL signoff;
- selected-item export;
- archive recovery/repair viewer;
- profile export/import choices.

10.3 2.0 - extensibility review

- playlist/random pool model;
- user-operated cloud document target without app backend, only after privacy ADR;
- Wear OS companion feasibility;
- plugin boundary for prayer-time or external trigger calculators;
- iOS research with honest capability differences.

Major version is not promised until user evidence supports it.

11 Success measurement without telemetry

The app does not upload usage. Product success is measured through:

- opt-in moderated usability sessions using synthetic data;
- public issue reports and release downloads at repository level, with privacy limits acknowledged;
- local Health status shown to the user but not transmitted;
- test/QA metrics from controlled fixtures;
- optional anonymous feedback form hosted separately from the app, clearly not embedded core telemetry.

Local history is for the user, not product analytics.

12 V1 success criteria

Dimension	Target
Usability	At least 5/6 representative testers import and schedule without facilitator correction

Dimension	Target
Adaptive presentation	100% scripted locked/unlocked cases choose correct surface
Native resilience	100% action tests pass with RN disabled
Battery structure	No idle service/wake lock/polling; one next alarm
Backup	100% logical/hash round trip fixtures; zero malicious mutations
Accessibility	All critical task scripts pass TalkBack and 200% scale
Release quality	Zero critical/high blockers; signed checksum-published APK
Privacy	No Internet/overlay/broad gallery permission; automatic platform backup excluded

13 Feedback triage

Feedback categories:

- reliability/platform;
- data/backup;
- accessibility;
- media compatibility;
- UX/copy;
- feature request;
- security/private report.

Reliability and data-loss reports receive reproduction templates that request device/OS/settings and privacy-safe diagnostics, never private media by default. Platform-specific behavior is added to the observed OEM matrix.

14 Change management

A proposal follows:

1. problem and user evidence;
2. scope/non-goal check;
3. permission, battery, privacy and backup compatibility impact;
4. ADR if architectural;
5. spec/traceability update;
6. implementation and migration in same change;
7. release evidence.

“Small” features that add Internet, overlay, persistent service, broad storage, new backup fields or native background entry are architectural changes.

15 Deprecation policy

- Public backup major versions receive a stated reader support window.
- Notification channel versions are not removed while active profiles reference them.
- Database migrations are never dropped without an explicit minimum-upgrade path.
- Removed features preserve exported data where possible and state omissions.
- APK direct-download releases remain available unless revoked for security.

16 Release cadence and branching

Use trunk-based development with short-lived branches and tagged milestones. Release candidates are immutable tags; fixes produce a new candidate. Stable releases use semantic versions and signed tags where available. Changelog separates Added, Changed, Fixed, Security and Known limitations.

17 Stop conditions

Pause feature growth when:

- native alarm actions depend on JS;
- an unresolved data-loss path exists;
- idle battery invariants fail;
- backup reader/writer compatibility is untested;
- Android current policy materially changes full-screen/exact alarm eligibility;
- documentation promises more than test evidence.

18 Portfolio milestone

The portfolio case study is published only after a real installable release exists. It includes the originating Islamic reminder problem respectfully, explains generalization to a media reminder engine, shows adaptive presentation and backup architecture, and links source/APK/checksum/docs. It does not use downloaded Instagram reels without permission.

18.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Architecture Decision Records

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-17
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Record binding architectural decisions, their context, alternatives, consequences and reconsideration triggers.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 ADR policy

Each ADR is immutable after approval except for status and superseding references. New information produces a new ADR. Status values: Proposed, Accepted, Superseded, Rejected. All entries below are **Accepted** for the v1 baseline on 2026-08-05.

2 ADR-001 - Build an Android application, not a PWA

Context: The defining feature is a reliable scheduled alert that can wake a locked phone, provide native notification actions and recover after process death. Browser background execution and autoplay constraints cannot guarantee clock-like behavior.

Decision: Deliver a signed Android APK. Reuse web skills through React Native rather than treating a PWA as the production alarm surface.

Alternatives: PWA; Trusted Web Activity; Capacitor wrapper. These remain suitable for a lightweight companion but not the reliability core.

Consequences: Native Android permissions, lifecycle, testing and signing are required. Portfolio value increases through native integration. iOS is not automatically supported.

Reconsider when: Browser standards and OS policy provide a verified equivalent exact-alarm/full-screen mechanism across killed processes.

3 ADR-002 - Android-only version 1

Context: Android and iOS have materially different background scheduling and full-screen presentation capabilities. Designing both before proving the product would dilute reliability.

Decision: Scope v1 to Android phones/foldables, with responsive tablet UI. No shared promise of identical iOS behavior.

Consequences: Faster focused implementation and accurate platform claims. Architecture keeps domain logic portable where practical.

Reconsider when: Android v1 has real usage evidence and an iOS capability study defines honest fallbacks.

4 ADR-003 - React Native UI with a Kotlin reliability core

Context: React Native is efficient for forms, navigation and portfolio development, but due alarm actions cannot depend on JavaScript cold startup.

Decision: Use React Native 0.86.x/New Architecture for everyday UI and Kotlin for scheduling, receivers, services, Room repositories, notification actions, locked alarm activity and resilient player.

Alternatives: Pure Kotlin/Compose; all-JavaScript alarm plugin. Pure Kotlin maximizes uniformity but reduces the creator's preferred workflow. JS-only creates an unacceptable stop-action dependency.

Consequences: Two-language codebase and typed bridge tests. Alarm path survives RN failure.

Reconsider when: Project maintainers choose full Compose and can migrate without backup/behavior regression.

5 ADR-004 - Room is the logical source of truth

Context: Alarm receivers need data before React loads, and dual JavaScript/native stores create divergence.

Decision: Room stores reminders, schedules, profiles, occurrences, operations and active session. React receives DTOs through native use cases. DataStore holds only small preferences/device envelope.

Consequences: Native migration discipline is required. JS state is view/cache state, not durable truth.

6 ADR-005 - Schedule only the globally earliest due occurrence

Context: Registering every recurring occurrence increases wake/synchronization complexity and makes changes harder to reconcile.

Decision: Persist derived occurrences and register one ordinary next one-shot AlarmManager event. Recalculate after every relevant mutation/event. Do not use repeating alarms.

Consequences: Minimal OS alarm footprint and clear battery story. Scheduler correctness becomes central and receives extensive tests.

Reconsider when: Platform constraints demonstrate a reliability regression at supported scale that cannot be solved by reconciliation.

7 ADR-006 - Use user-authorized exact alarms with transparent Limited mode

Context: Exact reminders are core, but modern Android requires special access and exact alarms have battery cost.

Decision: Request `SCHEDULE_EXACT_ALARM` contextually. Prefer `setAlarmClock()` for enabled exact user-visible occurrences. When denied, offer explicit inexact Limited mode or keep reminder Needs setup. Do not silently degrade.

Alternatives: `USE_EXACT_ALARM`; inexact-only. `USE_EXACT_ALARM` is more restricted and distribution-sensitive; inexact-only breaks the intended alarm experience.

Consequences: Health and permission education are required. Access may be revoked and exact alarms canceled, requiring reconciliation.

8 ADR-007 - No persistent idle service or time polling

Context: Battery efficiency is a defining requirement.

Decision: No service remains active while idle; no JS interval observes the clock; no periodic worker delivers reminders. A bounded foreground service exists only while an audible alert is active.

Consequences: Android system scheduling carries idle timing. Service and wake-lock lifetime become measurable invariants.

9 ADR-008 - Locked full-screen; unlocked system notification; no overlay

Context: A full-screen alarm is useful when the user is not interacting with the device but irritating when they are. Android owns heads-up notifications and restricts full-screen intents.

Decision: At trigger, native code checks interactive/lock state. Locked or non-interactive may use FSI/native alarm if eligible. Unlocked uses a high-importance notification without FSI. Foreground app also shows an in-app strip. Never request `SYSTEM_ALERT_WINDOW`.

Consequences: The app cannot guarantee a precise 20% dropdown height. OEM/channel settings may suppress heads-up; notification shade is fallback.

10 ADR-009 - Media plays only after explicit Play

Context: The user's desired ritual is to acknowledge the reminder and then view/listen. Autoplay can be intrusive, disclose private content and conflict with platform audio rules.

Decision: Alarm tone/vibration may alert according to profile; attached media begins only after Play. Play stops the alarm before Media3 playback.

Consequences: Clear separation between alarm audio and reminder content; notification action must launch playback reliably.

11 ADR-010 - Copy imported media into app-specific storage

Context: Linking only to a gallery URI can break when permission expires, file moves or original is deleted.

Decision: Stream selected files into app-owned private storage, hash and validate, then release source access. Internal filenames are UUIDs.

Consequences: Uses additional device storage and uninstall removes the copy. Library is reliable and backup is deterministic.

12 ADR-011 - Use system pickers, not broad gallery permission

Context: User needs selected files, not persistent read access to the entire gallery.

Decision: Use Photo Picker for visual media and Storage Access Framework for documents/audio/export/import. Do not request broad media read permission in v1.

Consequences: Provider behavior varies and must be streamed safely. Privacy/permission surface is smaller.

13 ADR-012 - Use Media3 and native playback activity

Context: Accepted playback should remain functional if RN navigation is unavailable and should use supported Android media infrastructure.

Decision: Use AndroidX Media3 in a native activity for due playback; RN may use the same native component for previews through an abstraction.

Consequences: Codec behavior is device-dependent; unsupported media is handled explicitly. Adds native library size.

14 ADR-013 - Logical JSON/media backup, never raw database copy

Context: Raw SQLite couples archives to schema/filesystem and can include transient state.

Decision: Export versioned DTO JSON, media bytes, manifest and checksums. Import maps through repositories/migrations.

Consequences: More code but stable portability, conflict handling and validation. Database implementation can evolve.

15 ADR-014 - Plain ZIP in v1; encryption later

Context: A portable manual archive is required immediately. Correct encrypted archive design adds key derivation, recovery and compatibility complexity.

Decision: V1 writes a standard plaintext ZIP with explicit privacy warning. No developer-held key. P1 may introduce an authenticated passphrase format as a new archive version.

Consequences: User must protect the file. Portability and recoverability are high. Marketing must not call it encrypted.

16 ADR-015 - No Internet permission and exclude Android automatic backup

Context: "Local-only" can be undermined by accidental SDK traffic or operating-system cloud backup.

Decision: Production manifest has no Internet permission/analytics. Data extraction rules and legacy backup rules exclude all app data from system cloud backup and device transfer. Manual ZIP is the migration path.

Consequences: No in-app update check, remote crash reporting or cloud sync. Users transfer archives themselves.

17 ADR-016 - Room/file outbox and operation journals bridge atomicity gaps

Context: Room cannot atomically commit with AlarmManager or filesystem/document providers.

Decision: Persist desired scheduler generations and pending file/restore operation phases. Reconciliation completes or rolls back after crashes.

Consequences: More explicit state and repair code, but no false cross-system transaction assumption.

18 ADR-017 - Minimal direct-boot alarm envelope

Context: Credential-encrypted Room/media may be unavailable before first unlock after reboot.

Decision: Mirror only opaque next ID/time/profile class into device-protected storage. Reschedule a generic alert after locked boot; require unlock for personal details/media. Full reconciliation follows normal boot/unlock.

Consequences: Improved reboot reliability without leaking labels. Adds envelope consistency tests.

19 ADR-018 - Three reusable default profiles

Context: Configuring every sound/vibration/timeout repeatedly increases complexity.

Decision: Seed Gentle, Standard and Persistent profiles. Reminders reference profiles. Profile edits show affected count. Persistent is explicitly “not for emergencies.”

Consequences: Simpler authoring and consistent channels. Users need clear distinction between app preference and channel-effective behavior.

20 ADR-019 - Android SDK and React Native baseline

Context: The project starts on 2026-08-05 and should use an actively supported framework while meeting current distribution rules.

Decision: React Native 0.86.x stable line; minSdk 26; compileSdk 37; initial targetSdk 36; retest API 37 and move target when policy/dependency readiness requires.

Consequences: Covers modern Android while retaining API 26. Version facts are refreshed in MR-22 before implementation/release.

21 ADR-020 - Apache License 2.0

Context: The project is portfolio-worthy and may benefit from contributors while preserving patent/license clarity.

Decision: Application source uses Apache-2.0. Brand assets, screenshots and user media have separate provenance. Contributions require sign-off under repository policy.

Consequences: Commercial reuse is allowed under license terms; no warranty. Repository must include NOTICE/third-party license records as applicable.

22 ADR-021 - No medical, emergency or guaranteed timing claim

Context: Platform-controlled alarms can be disabled/delayed, and the app lacks regulated validation.

Decision: Product is a personal media reminder. Copy and portfolio materials explicitly exclude medical/emergency reliance and absolute delivery guarantees.

Consequences: Persistent profile remains bounded and cannot be marketed as critical alert infrastructure.

23 Supersession procedure

A new ADR states which prior ADR it supersedes, migration effect, backup/permission impact, release threshold and rollback. Code that contradicts an Accepted ADR is not merged merely because tests pass; the decision must be changed transparently first.

23.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Contribution Guide and Engineering Standards

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-18
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define repository workflow, code quality, architecture boundaries, tests, documentation, dependency review and definition of done.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.

Term	Meaning
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Contribution values

Correctness, privacy, accessibility and battery behavior outrank code novelty. Contributions should reduce ambiguity, preserve portable backups and keep native alarm controls available without JavaScript. The project welcomes focused changes with evidence.

2 Repository setup

Expected tools are recorded and pinned in the repository:

- current supported Node LTS compatible with React Native 0.86.x;
- npm lockfile with `npm ci`;
- JDK version required by the selected Android Gradle Plugin;
- Android SDK platforms 26, 36 and 37 plus emulator images used in CI;
- Ruby/CocoaPods are not required for Android-only v1;
- Git and platform signing tools.

Canonical commands:

```
npm ci
npm run typecheck
npm run lint
npm test
npm run android
cd android && ./gradlew test lintDebug connectedCheck
```

Release commands are scripted and never request signing secrets from source-controlled files.

3 Branch and change workflow

Use short-lived branches from `main`. `main` stays buildable. Pull requests must be focused, linked to requirement/issue and updated from current `main`. Direct pushes to protected release tags are forbidden.

Commit messages follow Conventional Commits, for example:

```
feat(alarm): add native snooze idempotency
```

A commit history may be squashed, but final release notes preserve meaningful changes.

4 Architecture boundaries

Contributors MUST follow MR-07:

- no durable reminder state in React-only storage;
- no `AlarmManager` calls from JavaScript packages;
- no raw DAO access from Android activities/receivers;
- no media byte arrays crossing the bridge;
- no arbitrary filesystem path exposed to UI;
- no RN dependency in Play/Snooze/Dismiss receiver path;
- no notification/full-screen behavior implemented with overlay permission;
- no direct archive-to-database mapping without validator/repository.

CI includes dependency/architecture tests where practical.

5 TypeScript standards

- `strict` enabled; avoid `any` and unsafe casts.
- Runtime-decode native/external payloads where Codegen cannot guarantee.
- Components are functional and accessible by default.
- Hooks do not hide persistent side effects.
- Feature modules own their screens/types/tests.
- Use design tokens; no isolated magic color/spacing values.
- User-visible strings come from localization resources.
- Errors use typed domain envelopes and safe message keys.
- React state does not duplicate Room truth beyond explicit query/view cache.

6 Kotlin standards

- Coroutines with structured concurrency; no `GlobalScope`.
- IO on `Dispatchers.IO`; no blocking main-thread database/file work.
- Receivers call `goAsync()` and finish within bounded work.
- Every wake lock/service has owner, timeout and terminal cleanup.
- Sealed classes/value objects represent states and errors.
- Room writes use explicit transactions.

- Use `java.time` APIs with desugaring as needed; no ad hoc Calendar recurrence logic.
- `PendingIntents` explicit and immutable.
- Android components exported false unless documented.
- Domain calculators are pure and JVM-testable.

Formatting is enforced by a pinned Kotlin formatter and static analysis; rule suppressions need comments and review.

7 Naming

- UUID entity variables: `reminderId`, `assetId`, `occurrenceId`, `sessionId`.
- Absolute timestamps: `scheduledAt`, `createdAt`; local values: `localTime`, `zoneId`.
- Booleans describe state: `isInteractive`, `enabledIntent`, `canScheduleExact`.
- Operations use verbs: `reconcileScheduler`, `claimOccurrence`, `commitImport`.
- Avoid generic `Manager` unless component coordinates multiple services; prefer `Repository`, `Calculator`, `Coordinator` or `Gateway`.

8 Database and backup changes

Any persistence change requires in the same pull request:

1. Room migration and migration test;
2. updated schema documentation/ERD if structural;
3. backup DTO effect and reader/writer compatibility decision;
4. round-trip fixture update;
5. rollback/recovery assessment;
6. MR-21 traceability update;
7. ADR if semantics or portability change.

Destructive migration flags are prohibited in release.

9 Permission and background changes

A new permission, receiver, service, foreground-service type, job, alarm or exported component requires:

- user problem and least-privilege rationale;
- Android version/policy research from official sources;
- battery impact;
- privacy notice impact;
- denial/fallback UX;
- manifest/security tests;
- ADR when it changes the approved permission set.

No dependency may introduce a permission unnoticed; merged manifest is inspected in CI.

10 Testing expectations

Change	Minimum evidence
Pure recurrence/domain	Unit/property tests
Room entity/query	DAO + migration/query-plan test
Alarm receiver/action	Instrumentation with RN disabled and duplicate intents
UI component	Component semantics + visual/manual state
Permission/notification	API matrix scenario and denial fallback
Import/file	crash/failure injection and cleanup test
Backup	round trip plus malicious/conflict fixtures
Accessibility	TalkBack/scale impact note and test
Performance-sensitive	before/after benchmark against MR-15

Bug fixes add a regression test where feasible.

11 Documentation standards

The Markdown pack is source; PDFs are generated artifacts. Requirements use stable IDs. No `TODO`, `TBD` or unresolved placeholder is accepted in an Approved document. Diagrams use editable source plus rendered PNG/SVG. Current platform facts are linked in MR-22 with access date and refresh trigger.

Code comments explain why/invariants, not restate syntax. Public APIs and unusual lifecycle choices receive KDoc/TSDoc.

12 Accessibility review

Every component PR confirms name/role/state, focus order, 48 dp target, font scale and non-color state. Alarm/notification changes require manual TalkBack evidence. String additions include translator context and pseudo-locale review for critical flows.

13 Security and privacy review

Never commit personal media, real backup files, signing keys, keystores, private diagnostics or secrets. Fixtures are synthetic and licensed. Security-sensitive changes follow MR-12. Vulnerability reports are handled privately; do not open a public proof-of-concept issue that exposes users before a fix.

14 Dependencies

A dependency proposal states:

- exact capability needed;
- why platform/current dependencies are insufficient;
- license;
- maintenance/release status;
- transitive size and permissions;

- native/React compatibility;
- removal plan;
- security history where available.

Lockfile changes are reviewed. Avoid “utility” packages for trivial code. Update dependencies in focused PRs with alarm/backup regression suite.

15 UI implementation

Use the documented Material 3-inspired design system. Screens must implement loading, empty, error, offline/local and high-font states. Do not hardcode notification heads-up geometry. Do not use sacred/religious symbols as universal controls. Public screenshots use safe media.

16 Pull request template

A PR answers:

- Problem and linked requirements.
- User-visible behavior.
- Architecture/ADR impact.
- Permission, privacy, battery, backup and accessibility impact.
- Tests and devices run.
- Screenshots/recordings with synthetic data when UI changes.
- Migration/rollback.
- Known limitations.

17 Review checklist

Reviewer verifies requirement correctness, native/JS ownership, failure paths, concurrency/idempotency, cleanup, permission least privilege, safe logs, accessibility semantics, test quality, documentation and release impact. “Works on my phone” is not sufficient for alarm code.

18 Definition of done

A change is done when:

- implementation follows approved specs/ADR;
- tests pass and new risk has evidence;
- no inaccessible or unsafe failure path;
- schema/backup compatibility handled;
- no battery invariant regression;
- strings/localization and design states complete;
- docs/traceability/changelog updated;
- release manifest remains compliant;
- code reviewed with no unresolved blocking comment.

19 AI-assisted contributions

AI-generated code is reviewed under identical standards. The contributor is responsible for licensing, correctness, security and tests. Generated changes must cite the requirement/ADR they implement and cannot invent APIs, permissions or dependencies. MR-19 governs the development loop.

20 Community conduct

Use a respectful, inclusive code of conduct. The originating Islamic use case should be treated accurately and without mockery, while the general engine supports diverse lawful personal practices. Harassment, scraped personal data and copyrighted demo media are not accepted.

20.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

AI Agent Guide and Master Loop Prompt

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-19
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Provide an authoritative prompt and closed-loop workflow for AI-assisted implementation without scope, security or architecture drift.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Purpose

This guide is designed for Claude Code, Codex, Cursor or another coding agent working in the repository. The agent operates as an implementation partner, not product owner. It must make small verifiable changes, read the source-of-truth pack, and stop when a decision is missing or evidence fails.

2 Non-negotiable constraints

1. Android-only v1; package `com.aslam.mediareminder`.
2. React Native 0.86.x/New Architecture for normal UI; Kotlin native reliability core.
3. No backend, account, analytics, advertising or production `INTERNET` permission.
4. No overlay permission, broad gallery permission or battery-optimization exemption request.
5. No idle persistent service, time polling or `WorkManager` reminder delivery.
6. One globally earliest ordinary `AlarmManager` event.
7. Full-screen only for locked/non-interactive state and only when platform permits; unlocked uses system notification without FSI.
8. Play/Snooze/Dismiss must work without React Native startup.
9. Media starts only after Play.
10. Room and app-owned files are source of truth; React state is disposable.
11. Imported media is copied, streamed, hashed and transaction-journaled.
12. Backups are logical versioned ZIPs; validate before mutation; never raw database copy.
13. Every background resource is bounded and cleaned up.
14. No medical/emergency guarantee.
15. Any permission, schema, backup or architectural change requires source-of-truth/ADR update.

3 Context loading order

Before coding, read:

1. AGENTS.md;
2. MR-00 Document Map;
3. MR-17 ADRs;
4. the requirement rows in MR-21 for the selected task;
5. the domain specification: MR-06 for alarms, MR-09 for persistence, MR-10 for backup, MR-03/MR-04 for UX;
6. existing code/tests in the exact module;
7. current official docs in MR-22 when platform behavior is involved.

Never rely only on a chat summary when repository documents exist.

4 Work-unit sizing

One loop should implement a coherent vertical slice small enough to review. Good examples:

- occurrence key calculator plus DST tests;
- pending file operation copy/repair state machine;
- exact-alarm capability snapshot and Health row;
- native Dismiss action with idempotency test;
- backup path validator with malicious fixtures;
- one accessible Reminder card component and states.

Bad work units: “build all alarms,” “finish the app,” or a refactor spanning unrelated domains.

5 Master loop prompt

Copy the block below into the coding agent at repository root.

You are the implementation agent for Media Reminder, an offline-first Android adaptive media alarm.

AUTHORITATIVE SOURCES

- Read AGENTS.md and specs/00_Document_Map_and_Executive_Summary.md first.
- Read specs/17_Architecture_Decision_Records.md and the requirement/test rows relevant to this task in specs/21_Requirements_Traceability_and_Acceptance_Catalog.md.
- Read the domain documents referenced by those rows.
- In a conflict, follow the precedence in MR-00. Do not silently choose a new architecture.

HARD CONSTRAINTS

- Android-only v1; React Native UI plus Kotlin native reliability core.
- No backend, account, analytics, ads, Internet permission, overlay permission, broad gallery permission, or default battery-exemption request.
- No persistent idle service, clock polling, repeating alarm, or WorkManager due delivery.
- Schedule one globally earliest one-shot system alarm.
- Full-screen alarm is permitted only when the device is locked or non-interactive and Android permits it. If unlocked, use a normal high-importance notification without a full-screen intent. Android owns heads-up size.
- Native Play, Snooze, Dismiss, timeout and sound stop cannot depend on React Native startup.
- Attached media starts only after Play.
- Room/app-owned files are authoritative. Imported media is streamed into private storage with hash, journal and recovery.
- Backup is a versioned logical ZIP with validation, checksums, staging, conflict plan and rollback. Never import a raw database.
- All file, wake-lock, service, notification and PendingIntent behavior is bounded, explicit and tested.

- Do not add a dependency, permission, exported component, schema field or archive field without explaining its impact and updating required docs/tests.

TASK

Implement exactly this work item: <INSERT ONE ISSUE OR REQUIREMENT SLICE>.

LOOP

1. ORIENT

- Summarize the relevant requirements, invariants and existing code paths.
- List assumptions. Resolve them from repository sources before asking questions.
- Inspect git status and do not overwrite unrelated work.

2. PLAN

- Propose the smallest vertical change.
- Identify files, migrations/contracts, tests, permission/privacy/battery/ally/backup impacts.
- State the observable acceptance result.
- If the plan conflicts with an ADR or lacks a required product decision, STOP and report the exact conflict instead of coding.

3. IMPLEMENT

- Make focused changes following architecture boundaries.
- Use typed domain states/errors, structured concurrency, streaming IO and idempotent native actions.
- Preserve backward compatibility or add explicit migration.
- Keep user-visible strings localized and accessible.
- Do not leave TODO/TBD placeholders in the completed slice.

4. VERIFY FAST

- Run the narrowest relevant formatter, static analysis and tests.
- Fix failures rather than weakening tests.
- Add regression tests for the behavior and failure path.

5. VERIFY SYSTEM

- Run the required broader suites from MR-14/MR-21 for the changed domain.
- For alarm code, include RN-disabled native action and locked/unlocked assertions.
- For file/backup code, include crash/malicious/cancellation cases and bounded-memory behavior.
- For UI, include semantics, large text, dark mode and error/empty/loading states.

6. REVIEW

- Inspect diff for accidental permissions, exported components, logs, paths, secrets, broad refactors and source-of-truth drift.
- Confirm wake locks/services terminate, actions are idempotent and current user data is preserved on failure.
- Confirm no generated/copyrighted private media entered the repository.

7. UPDATE SOURCES

- Update requirement traceability, changelog and specs/ADR only where behavior changed.
- Update fixtures/migration matrices as needed.
- Do not edit generated PDFs manually; regenerate them from Markdown.

8. REPORT

Return:

- implemented requirement IDs;
- concise behavior change;
- files changed;
- tests/commands and results;
- measured evidence when required;
- migrations/compatibility impact;
- privacy, permission, battery and accessibility review;
- residual risks or blockers.

STOP CONDITIONS

- Required decision is absent or conflicts with an Accepted ADR.
- A test exposes data loss, alarm action unavailability, unlocked full-screen takeover, unbounded background work, archive validation bypass, or privacy leak.
- Current official Android behavior differs materially from MR-22.
- The repository has unrelated uncommitted changes that would be overwritten.
- A requested change would require a prohibited permission or false reliability claim.

Never claim completion unless acceptance evidence passes. Never hide a failed test or replace it with a weaker assertion.

6 Agent output contract

Each completed loop reports a table:

Field	Required content
Requirements	IDs implemented/verified
Behavior	User-observable result
Files	Added/changed/deleted
Tests	Exact commands and pass/fail
Evidence	Screenshots, benchmark, device/API where relevant
Data	Schema/archive migration and rollback
Platform	Permission, notification, background impact
Quality	Accessibility/privacy/battery review
Residual	Known risk or None

Do not output unverifiable phrases such as “should work” as final evidence.

7 Task decomposition map

7.1 Foundation sequence

1. repository/manifest guard;
2. domain IDs/time primitives;
3. Room schema/migrations;
4. native Codegen snapshot;
5. media copy journal;
6. library UI;
7. recurrence calculator;
8. scheduler outbox;
9. capability Health;
10. native notification/actions;
11. locked activity/ringing;
12. player transition;
13. boot/time recovery;
14. backup export;
15. import validation/conflict/commit;
16. accessibility/performance/release.

Do not start polished alarm animation before native action state and scheduler idempotency exist.

8 Review heuristics for AI-generated code

Red flags:

- `setInterval`, background timer or repeating alarm;
- `service START_STICKY` without session validation;
- `SYSTEM_ALERT_WINDOW` or launch activity unconditionally from receiver;
- `READ_MEDIA_VIDEO` for a one-file picker use case;
- raw file path passed to React;
- `getBytes()` on `media/ZIP`;
- `fallbackToDestructiveMigration()`;
- mutable implicit `PendingIntent`;
- notification action routed only through JS deep link;
- database copied into ZIP;
- archive extraction before traversal/size validation;
- catch-all exception that continues ringing or deletes temp evidence;
- log statement containing title/path/URI;
- promise of exact 20% heads-up height;
- permission prompt on first launch with no user intent.

9 Prompt for design implementation

For a UI slice, append:

Use MR-03 and MR-04 exactly. Implement all states: normal, loading, empty, error, permission-limited, 200% font, TalkBack semantics, light/dark and compact/medium width. Use tokens rather than ad hoc values. Do not invent a generic SaaS dashboard. Keep copy calm and verb-first.

10 Prompt for adversarial review

After implementation, run a separate review agent with:

Review this diff as a hostile Android reliability, data-integrity and privacy reviewer. Find paths that can: show full screen while unlocked; leave sound/service/wake lock active; duplicate an action; lose media/data during crash; trust a ZIP/path/URI; add network/permission exposure; break backup compatibility; or block TalkBack. Cite files/lines and requirement/ADR. Do not rewrite code until findings are listed.

11 Human responsibility

The human owner reviews permissions, signing, release claims, religious/example content, license provenance and destructive migrations. An AI agent does not approve its own security finding, change ADR status or publish signed binaries without the release checklist.

11.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and

visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Release, Distribution, Portfolio and Maintenance Guide

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-20
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Define signing, APK publication, release evidence, optional store path, support, maintenance and portfolio presentation.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.

Term	Meaning
Alarm session	The bounded runtime state created when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Distribution strategy

The primary v1 channel is a signed APK published through GitHub Releases from a public source tag. The portfolio site links to that release rather than an unversioned file. Google Play distribution is optional and requires a separate policy/readiness review, but the app is built to meet current target and permission disclosure expectations.

There is no in-app updater because the production app has no Internet permission. Users obtain updates through the release page or an optional future store.

2 Application identity

Field	Value
App name	Media Reminder
Package	com.aslam.mediareminder
Versioning	Semantic user version plus monotonically increasing Android version code
Source license	Apache-2.0
Minimum SDK	26
Initial target	36
Compile SDK	37
Distribution artifact	Signed arm64/universal APK as chosen, plus optional AAB for Play

Changing package ID after public release breaks normal upgrade identity and is not permitted without a migration/distribution plan.

3 Signing key management

- Generate release key outside repository on a trusted device.
- Store primary and encrypted backup in separate secure locations controlled by owner.
- Never paste key password or keystore into an AI chat, issue, commit or public CI variable output.
- CI signing uses protected secret storage only if the threat model is accepted; local/offline signing is acceptable for early releases.
- Record certificate SHA-256 fingerprint publicly with releases.
- Test install/upgrade with the exact release signature.
- Key compromise triggers immediate incident process and clear user notice; direct APK users cannot be transparently migrated to a new signature.

4 Build pipeline



1. Protected tag points to reviewed commit.
2. Clean environment installs lockfile dependencies.
3. Run typecheck, lint, unit, migration, archive security and instrumentation suites.
4. Build release artifact with merged-manifest checks.
5. Generate SBOM, third-party notices and mapping/native symbol files for private retention.
6. Sign APK/AAB.
7. Verify signature and install/upgrade on reference device.
8. Calculate SHA-256 for every public artifact.
9. Attach QA evidence, changelog, privacy, known limitations and source tag.
10. Independently download and verify the published artifact before announcing.

5 Release assets

A release contains:

- MediaReminder-vX.Y.Z-arm64-v8a.apk and/or clearly labeled universal APK;
- SHA256SUMS.txt;
- source tag/archive;
- release notes/changelog;
- privacy notice;
- known limitations/device notes;
- documentation ZIP and combined specification PDF;
- SBOM and third-party notices, or links within source;
- optional demo video/screenshots using licensed content.

Do not distribute debug APKs or keystores.

6 Release notes template

6.1 Added

User-facing features.

6.2 Changed

Behavior, permissions, channels, schedules or backup compatibility.

6.3 Fixed

Bugs with requirement/test references when useful.

6.4 Security and privacy

Relevant changes without unsafe exploit detail before users can update.

6.5 Compatibility

Minimum/target SDK, database migration, archive reader/writer and notification-channel changes.

6.6 Known limitations

OEM/platform behavior, unsupported codecs and force-stop/full-screen/exact-access caveats.

6.7 Verification

APK SHA-256, signing certificate fingerprint, source commit and QA report.

7 Direct APK user journey

The portfolio/release page explains:

- this is an Android APK and must be obtained from the official release;
- verify filename/version/checksum where practical;
- Android may require allowing installation from the chosen browser/file manager;
- the app requests only documented permissions;
- updates are manual unless a store distribution is installed;
- exported backups should be created before major updates or device changes.

Do not encourage disabling broad device protections or installing from mirrors.

8 Google Play readiness

Before Play submission:

- confirm current target API requirement and deadline;
- confirm exact-alarm declaration/use case and Play policy;
- complete foreground service type declaration;
- validate full-screen intent eligibility as a genuine alarm app and provide requested evidence;
- complete Data safety consistently with no transmitted data and local handling;
- supply privacy policy, content rating, screenshots and app access declaration;
- test AAB splits and Play signing strategy;
- verify developer identity/registration requirements current at submission.

Play review may restrict full-screen/exact use. The app must retain transparent fallback and must not misrepresent the general media reminder as an emergency or calling app.

9 Developer verification and sideloading

Android distribution and developer verification rules are evolving. Before each public direct-download release, MR-22 must be refreshed and the release page updated with current official requirements. The project does not promise permanent unrestricted sideloading behavior. APK signing, source transparency and checksums remain required regardless.

10 Upgrade and rollback

Upgrade tests cover every supported public database version. App downgrade is not supported because Room schema and Android package version may be incompatible. Recovery path is: export with old version before upgrade where possible, retain external archive, and restore into a supported version. A bad release is withdrawn and replaced with a higher version code; users receive clear recovery instructions.

11 Notification channel migration

Channel defaults cannot be changed by silently deleting a user-owned channel. New behavior uses a new versioned channel and asks the user to review/migrate. Release notes state the effect. Existing channel settings are not overwritten.

12 Backup compatibility release gate

Each release publishes:

- archive writer version;
- oldest/newest supported reader format;
- database schema version;
- round-trip matrix with previous public versions;
- any omitted future-only fields.

A release cannot ship a backup writer change without reader fixtures and migration notes.

13 Maintenance schedule

At least before every feature release:

- refresh official Android/RN baseline;
- update dependencies and review advisories;
- run full physical alarm matrix;
- verify manifest/backup exclusions;
- run migration and archive compatibility;
- rerun battery/performance smoke;
- review open OEM issues;
- update third-party notices and SBOM.

Security fixes can ship sooner with focused evidence plus required core regression.

14 Support boundaries

Public issue templates request app version, Android/API, manufacturer/model, notification/exact/FSI status, reproduction steps and privacy-safe diagnostic export. They explicitly say not to upload personal media or full backup. Codec issues may use a minimal synthetic reproduction file.

No guaranteed response time is promised for an individual open-source project. Critical security reports use the private route in `SECURITY.md`.

15 Portfolio case study structure

15.1 Hero

Media Reminder - an offline-first Android app that turns local video, audio and images into state-aware reminders. Include install/source/document links and a clear Android badge.

15.2 Problem

Tell the authentic origin: useful dua/adhkar reminders were easy to find in short videos but easy to forget and social feeds were distracting. Avoid presenting downloaded Instagram content without permission.

15.3 Product insight

The interruption should adapt: locked phone can behave like an alarm; active use receives a compact system notification. Explain that Android controls heads-up geometry.

15.4 Engineering highlights

- native Kotlin reliability core behind React Native UI;
- one global AlarmManager event and zero idle service;
- native actions independent of JS;
- staged app-owned media;
- versioned checksummed ZIP restore;
- no Internet permission and no analytics;
- accessible adaptive design.

15.5 Evidence

Include architecture/sequence diagrams, battery/soak metrics, device matrix, backup malicious tests and screenshots. Use real measured results after implementation; do not publish planned numbers as achieved.

15.6 Limitations and learning

State force-stop, permission, OEM, codec and full-screen constraints. This increases credibility.

16 Screenshot and demo policy

All public media is original, licensed, public-domain or synthetic with provenance. Remove private reminder names, notification history, file paths and device identifiers. A demo Islamic reminder can use

creator-authored text/graphics and properly licensed recitation/audio; otherwise use a neutral synthetic clip.

Screenshots required:

- Today ready and capability-limited;
- Library and editor;
- unlocked notification simulation represented accurately;
- locked native alarm;
- media player after Play;
- Health;
- backup preview/conflict;
- dark mode and accessibility scale.

17 Release checklist

- Tag/commit reviewed and clean.
- Version/schema/archive/channel versions correct.
- P0 traceability green.
- No critical/high defects.
- Physical matrix, soak, backup, a11y and battery evidence complete.
- Release merged manifest reviewed; prohibited permissions absent.
- Automatic platform backup exclusion verified.
- APK signed and upgrade-tested.
- Checksums/certificate fingerprint generated.
- Privacy, known limitations, licenses, SBOM and changelog attached.
- Published artifact independently downloaded, signature/hash verified and installed.
- Portfolio links point to immutable release, not local development build.

18 End-of-life

If maintenance stops, publish a final notice, keep source and existing artifacts available where safe, recommend exporting data, document Android versions last tested, and avoid leaving a misleading “fully supported” download. A known severe vulnerability may require removing affected APKs while preserving source/history and explanation.

18.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Requirements Traceability and Acceptance Catalog

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-21
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Map normative requirements to priorities, verification evidence and critical release acceptance scenarios.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created

Term	Meaning
	when an occurrence is actively alerting.
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Purpose

This catalog is the release-facing index of normative requirements. It does not replace the detailed rationale and algorithms in MR-01 through MR-20. The verification ID points to a test, inspection or review artifact. P0 is required for public v1; P1 may be deferred only when the roadmap explicitly places it after v1.

2 Requirement summary

Domain	Total	P0	P1
Product and scope	39	39	0
Experience and design	14	12	2
Functional behavior	18	17	1
Android native and battery	18	18	0
Data and storage	12	12	0
Backup and migration	15	14	1
Security and privacy	12	11	1
Accessibility and localization	10	9	1
Non-functional budgets	12	12	0

Domain	Total	P0	P1
Release and maintenance	10	10	0

3 Requirement catalog

3.1 Product and scope

ID	Priority	Requirement	Verification
PRD-001	P0	Core import, scheduling, alert actions, playback and backup work in airplane mode.	E2E-OFFLINE-001; static manifest
PRD-002	P0	Production manifest contains no INTERNET permission.	SEC-MANIFEST-001
PRD-003	P0	No account, advertising, analytics or remote configuration is required.	SEC-STATIC-002; privacy review
PRD-004	P0	Onboarding explains that imported media is copied into app storage and removed on uninstall.	UX-ONBOARD-001
PRD-005	P0	User-facing copy never promises guaranteed delivery or exact heads-up size.	COPY-AUDIT-001
PRD-010	P0	Users can import supported video, audio and image through system pickers.	E2E-IMPORT-001..003
PRD-011	P0	Deleting the original gallery/provider file after import does not break app playback.	IT-FILE-004

ID	Priority	Requirement	Verification
PRD-012	P0	Users can rename, categorize, tag, preview and delete media.	E2E-LIB-001..005
PRD-013	P0	Deleting referenced media requires an explicit dependency policy.	E2E-LIB-DELETE-006
PRD-014	P0	Unsupported, corrupt, oversized and low-space imports leave no partial asset.	IT-FILE-010..014
PRD-020	P0	A reminder combines media, schedule, profile, snooze and label.	UT-DOMAIN-001; E2E-REM-001
PRD-021	P0	MVP supports once, daily and selected-weekday schedules.	UT-REC-001..040
PRD-022	P0	Users can create, edit, duplicate, enable, disable and delete reminders.	E2E-REM-002..008
PRD-023	P0	Editor displays the native-calculated next occurrence before final save result.	E2E-REM-009
PRD-024	P0	DST gaps and overlaps use documented deterministic policies with disclosure.	UT-REC-DST-001..020
PRD-025	P0	A reminder is not represented as reliably active when required capability is absent.	IT-CAP-001..008
PRD-030	P0	Locked or non-	IT-PRESENT-

ID	Priority	Requirement	Verification
		interactive devices may use eligible native full-screen alarm presentation.	LOCKED-001..006
PRD-031	P0	Unlocked interactive devices never receive a forced full-screen activity.	IT-PRESENT-UNLOCK-001..010
PRD-032	P0	The app does not request or use overlay permission.	SEC-MANIFEST-002; code search
PRD-033	P0	Play, Snooze and Dismiss work without React Native initialization.	IT-NATIVE-ACTION-001..009
PRD-034	P0	Attached media begins only after Play.	IT-PLAY-001
PRD-035	P0	Every ringing path has a visible stop action and finite timeout.	IT-RING-001..008
PRD-036	P0	Android/OEM ownership of heads-up presentation is explained.	COPY-AUDIT-002
PRD-040	P0	Idle operation has no resident service, wake lock, JS timer or periodic due worker.	PERF-IDLE-001..005
PRD-041	P0	Scheduler registers one ordinary globally earliest one-shot alarm.	IT-SCHED-001..012
PRD-042	P0	Boot, package update, timezone and clock changes reconcile	IT-RECON-001..012

ID	Priority	Requirement	Verification
		schedules.	
PRD-043	P0	Exact-alarm denial/revocation is visible and provides Limited or setup choices.	IT-CAP-EXACT-001..006
PRD-044	P0	Health accurately explains force-stop limitations after relaunch.	MAN-FORCESTOP-001
PRD-050	P0	Export creates a local versioned ZIP with logical JSON, media and checksums.	E2E-BKP-EXPORT-001
PRD-051	P0	Export excludes raw DB, cache, paths, permissions, channels and runtime sessions.	BKP-CONTENT-001
PRD-052	P0	Import validates path, limits, schema, references, checksums, media and space before mutation.	SEC-ZIP-001..030
PRD-053	P0	Import provides Inspect only, Merge and Replace.	E2E-BKP-MODE-001..003
PRD-054	P0	Replace has rollback-ready state and explicit destructive confirmation.	IT-BKP-ROLLBACK-001..010
PRD-055	P0	Restored reminders re-evaluate device permissions/capabilities before activation.	E2E-BKP-CAP-001
PRD-060	P0	Critical flows are operable with TalkBack and	A11Y-TASK-001..010

ID	Priority	Requirement	Verification
		switch/keyboard access.	
PRD-061	P0	Critical content/actions remain usable at 200% font scale.	A11Y-SCALE-001..008
PRD-062	P0	Color is not the sole state indicator.	A11Y-COLOR-001
PRD-063	P0	Reduced motion removes nonessential movement.	A11Y-MOTION-001
PRD-064	P0	Alarm actions use explicit verbs and large reachable targets.	A11Y-ALARM-001..004

3.2 Experience and design

ID	Priority	Requirement	Verification
UX-001	P0	Primary navigation is Today, Library, Reminders and Settings.	UI-NAV-001
UX-002	P0	Permission requests occur at the moment a selected feature needs them, not as a startup wall.	E2E-ONBOARD-002
UX-003	P0	Today shows next occurrence, today timeline and only capability warnings that affect active reminders.	UI-TODAY-001..004
UX-004	P0	Reminder editor preserves entered data when capability setup is incomplete.	E2E-REM-CAP-010
UX-005	P0	In-app due strip is nonmodal and at	UI-STRIP-001..004

ID	Priority	Requirement	Verification
		most min(144dp, 20% usable height).	
UX-006	P0	Unlocked due alert uses Play, Snooze and Dismiss actions in a system notification.	IT-NOTIF-001..006
UX-007	P0	Locked alarm displays static context and native Play, Snooze and Dismiss; no autoplay.	IT-ALARM-UI-001..006
UX-008	P0	Snooze confirmation states the absolute next time.	UI-SNOOZE-001
UX-009	P0	Multiple simultaneous due reminders are queued in one session, not stacked.	IT-COLLISION-001..005
UX-010	P0	Health distinguishes Ready, Limited and Action needed with consequence and action.	UI-HEALTH-001..006
UX-011	P0	Backup import shows inspection and conflict plan before commit.	E2E-BKP-PREVIEW-001
UX-012	P0	Copy is calm, factual and does not shame missed reminders.	COPY-AUDIT-003
UX-013	P1	Text cards may be added without changing the alarm core.	UT-TEXT-001
UX-014	P1	Local history uses factual outcomes	UI-HISTORY-001

ID	Priority	Requirement	Verification
		and no competitive streak pressure.	

3.3 Functional behavior

ID	Priority	Requirement	Verification
FUN-001	P0	Import streams to a pending file while calculating SHA-256.	IT-FILE-TXN-001
FUN-002	P0	Promote media atomically and journal every incomplete file operation.	IT-FILE-TXN-002..009
FUN-003	P0	Internal filenames use UUIDs and never user path segments.	SEC-PATH-001
FUN-004	P0	Search matches local title, notes, category and tags.	UT-SEARCH-001..006
FUN-005	P0	Missing media disables dependent reminders and offers repair choices.	E2E-REPAIR-001
FUN-006	P0	Next occurrence is strictly after cursor and occurrence key is unique.	PROP-REC-001..002
FUN-007	P0	Snooze creates a child occurrence and does not mutate base recurrence.	UT-SNOOZE-001..006
FUN-008	P0	Profiles hold bounded sound, vibration, timeout, retry and grace values.	UT-PROFILE-001..010
FUN-009	P0	Broken custom tone falls back to	IT-AUDIO-004

ID	Priority	Requirement	Verification
		packaged tone with safe diagnostic.	
FUN-010	P0	Only one active alarm session owns sound/vibration.	IT-COLLISION-006
FUN-011	P0	Alarm terminal actions are idempotent for duplicate intents.	IT-NATIVE-ACTION-010..020
FUN-012	P0	Play stops alarm before requesting media audio focus.	IT-AUDIO-PLAY-001
FUN-013	P0	History is optional, local and defaults to 90-day retention.	UT-RETENTION-001
FUN-014	P0	Diagnostics exclude content names, paths, notes and exact labels.	SEC-LOG-001..005
FUN-015	P0	Test reminders use synthetic content and do not write user history.	IT-TEST-REM-001..004
FUN-016	P0	Notification channels are versioned and never recreated to override user choices.	IT-CHANNEL-001..006
FUN-017	P0	Deleting/editing an active reminder resolves or snapshots current session safely.	IT-CONCURRENT-001..005
FUN-018	P1	Duplicate media hash flow can reuse existing bytes.	IT-DEDUP-001

3.4 Android native and battery

ID	Priority	Requirement	Verification
AND-001	P0	Use minSdk 26, compileSdk 37 and initial targetSdk 36, refreshed before release.	BUILD-SDK-001
AND-002	P0	Use SCHEDULE_EXACT_ALARM with contextual education and canScheduleExact Alarms checks.	IT-CAP-EXACT-007..012
AND-003	P0	Preferred exact scheduling uses setAlarmClock for user-visible due events.	IT-SCHED-API-001
AND-004	P0	Inexact Limited mode is explicitly selected and labeled as potentially delayed.	E2E-LIMITED-001
AND-005	P0	AlarmDispatchReceiver claims occurrence before presentation and rejects stale generation.	IT-DISPATCH-001..010
AND-006	P0	Unknown device state defaults to nonintrusive notification path.	UT-PRESENT-UNKNOWN-001
AND-007	P0	Full-screen intent is attached only for locked/non-interactive eligible path.	IT-FSI-001..010
AND-008	P0	Notification denial never starts invisible continuous ringing.	IT-NOTIF-DENIED-001
AND-009	P0	Native alarm	SEC-COMP-001; IT-

ID	Priority	Requirement	Verification
		activity is exported false and reconstructs from active session.	ACT-001
AND-010	P0	Ringing foreground service is session-bound, mediaPlayback typed and hard-capped.	IT-FGS-001..012
AND-011	P0	Wake locks have explicit timeouts and are released on every terminal/failure path.	IT-WAKE-001..015
AND-012	P0	Android 17 background audio rules are tested; alarm audio uses USAGE_ALARM/ex act exemption plus compliant FGS path.	IT-A17-AUDIO-001..006
AND-013	P0	Direct-boot envelope contains no labels/paths and supports generic pre-unlock alert.	IT-DIRECTBOOT-001..008
AND-014	P0	Wall-clock rules reconcile after TIME_SET/TIMEZONE_CHANGED without replay.	IT-TIME-001..010
AND-015	P0	Foreground app receives bridge event but notification remains sufficient fallback.	IT-FG-001..004
AND-016	P0	Notification visibility defaults private and supports generic	IT-LOCKPRIV-001..004

ID	Priority	Requirement	Verification
AND-017	P0	secret content. No DND-policy, phone-state or battery-exemption permission is requested.	SEC-MANIFEST-003
AND-018	P0	Stop service/audio within one second of terminal action.	PERF-ACTION-001

3.5 Data and storage

ID	Priority	Requirement	Verification
DAT-001	P0	Room is the sole durable logical source of truth.	ARCH-STATIC-001
DAT-002	P0	Foreign keys/check constraints and unique occurrence keys enforce invariants.	DB-CONSTRAINT-001..020
DAT-003	P0	Database stores no absolute filesystem paths.	DB-STATIC-001
DAT-004	P0	Scheduler desired/applied generations implement an outbox reconciliation pattern.	DB-SCHED-001..008
DAT-005	P0	At most one active_alarm_session exists.	DB-SESSION-001
DAT-006	P0	File/restore operations have durable phase journals and startup repair.	DB-OP-001..015
DAT-007	P0	Destructive Room migration is forbidden in release.	BUILD-DB-001
DAT-008	P0	Every public schema migration	DB-MIG-001..N

ID	Priority	Requirement	Verification
		has a fixture and invariant test.	
DAT-009	P0	Cache is regenerable and clearing it does not remove media/reminders.	IT-CACHE-001
DAT-010	P0	Storage reserve is checked before import/restore.	IT-SPACE-001..004
DAT-011	P0	History, diagnostics and idempotency have bounded retention.	UT-RETENTION-002..005
DAT-012	P0	Startup does not hash/enumerate the full library on the main path.	PERF-START-001

3.6 Backup and migration

ID	Priority	Requirement	Verification
BKP-001	P0	Archive root includes manifest, README, checksums, data JSON and media.	BKP-SCHEMA-001
BKP-002	P0	Archive identifiers and media UUIDs are stable across round trip.	E2E-BKP-ID-001
BKP-003	P0	SHA-256 covers every entry except the checksum file itself.	BKP-HASH-001
BKP-004	P0	Export uses streaming IO and bounded memory.	PERF-BKP-001..004
BKP-005	P0	Import rejects absolute, traversal, NUL, symlink, duplicate and case-collision entries.	SEC-ZIP-001..012
BKP-006	P0	Import enforces	SEC-ZIP-013..022

ID	Priority	Requirement	Verification
BKP-007	P0	entry count, per-entry, total-size, ratio, JSON-depth and string limits. Every reference, count, size, checksum and media header is validated before commit.	BKP-VALID-001..012
BKP-008	P0	Validated staging is represented by expiring digest-bound import token.	SEC-TOKEN-001..006
BKP-009	P0	Merge conflict choices are deterministic and serializable.	UT-CONFLICT-001..040
BKP-010	P0	Replace creates rollback-ready state before removing current data.	IT-BKP-ROLLBACK-011..020
BKP-011	P0	Crash at each restore phase repairs to old or fully committed state.	IT-BKP-CRASH-001..N
BKP-012	P0	Unsupported major archive version never modifies current data.	BKP-COMPAT-001
BKP-013	P0	Permissions, channels, ringtones and platform paths are not restored as app data.	BKP-EXCLUDE-001
BKP-014	P0	Plain ZIP privacy warning appears before export and import custody is clear.	UX-BKP-PRIV-001
BKP-015	P1	Encrypted archive	SEC-ENC-DESIGN-001

ID	Priority	Requirement	Verification
		uses reviewed authenticated format and new version, never custom crypto.	

3.7 Security and privacy

ID	Priority	Requirement	Verification
SEC-001	P0	Android automatic cloud backup and device transfer exclude all app data.	SEC-BACKUP-001..004
SEC-002	P0	Android components are exported false unless explicitly documented.	SEC-COMP-002
SEC-003	P0	PendingIntents are explicit and immutable; state/nonce is verified.	SEC-PI-001..008
SEC-004	P0	FileProvider exposes only completed export paths with read-only grants.	SEC-FP-001..006
SEC-005	P0	Selected URIs are treated as untrusted streams with bounded validation.	SEC-URI-001..006
SEC-006	P0	No archive field becomes SQL or a path without typed validation.	SEC-INJECT-001..006
SEC-007	P0	Release logs/diagnostics contain no private media metadata.	SEC-LOG-006..012
SEC-008	P0	Signing key and credentials never enter repository, logs or AI	REL-KEY-AUDIT-001

ID	Priority	Requirement	Verification
SEC-009	P0	prompts. Dependency changes are pinned, licensed, scanned and permission-reviewed.	SEC-SUPPLY-001..006
SEC-010	P0	Release includes SBOM and third-party notices.	REL-SBOM-001
SEC-011	P0	Security reporting route does not require users to upload private media/backups.	DOC-SECURITY-001
SEC-012	P1	Optional screen protection is user-controlled and does not block normal ownership use.	UI-PRIVACY-001

3.8 Accessibility and localization

ID	Priority	Requirement	Verification
ACC-001	P0	Interactive elements expose meaningful name, role, state and enabled status.	A11Y-SEM-001..010
ACC-002	P0	Touch targets are at least 48dp; alarm actions at least 56dp.	A11Y-TARGET-001
ACC-003	P0	Initial alarm focus and order are summary, Play, Snooze, Dismiss, more.	A11Y-FOCUS-001
ACC-004	P0	Alarm title/time is announced once without repeated focus stealing.	A11Y-ANNOUNCE-001
ACC-005	P0	Swipe-only or long-press-only critical actions	A11Y-MOTOR-001

ID	Priority	Requirement	Verification
ACC-006	P0	have direct alternatives. Progress announces phase/meaningful thresholds, not every byte.	A11Y-PROGRESS-001
ACC-007	P0	Strings are externalized, plural-aware and not concatenated.	L10N-STATIC-001
ACC-008	P0	Layouts use start/end and pass RTL pseudo-locale.	L10N-RTL-001..006
ACC-009	P0	Date/time follows locale and 12/24-hour system preference.	L10N-TIME-001
ACC-010	P1	Tamil/Arabic release requires native-speaker review of critical copy.	L10N-REVIEW-001

3.9 Non-functional budgets

ID	Priority	Requirement	Verification
NFR-001	P0	Receiver entry to notification is <=500ms p95 and <=2s hard on reference devices.	PERF-ALERT-001
NFR-002	P0	Action tap to sound stop is <=150ms p95 and <=1s hard.	PERF-ACTION-001
NFR-003	P0	Cold usable shell is <=2s p95 reference and <=3.5s low class.	PERF-START-002
NFR-004	P0	Earliest occurrence query at 10k reminders is <=100ms p95 and <=250ms hard.	PERF-DB-001

ID	Priority	Requirement	Verification
NFR-005	P0	Import/export heap growth is bounded and independent of file size.	PERF-IO-001..004
NFR-006	P0	No-due 24h test has no structural idle violation and <=0.5 percentage-point attributed drain target.	PERF-IDLE-006
NFR-007	P0	No wake/service leak remains after 100-cycle alarm soak.	SOAK-ALARM-001
NFR-008	P0	Scheduler supports 10k reminders and 50k retained occurrences.	PERF-SCALE-001
NFR-009	P0	Diagnostics <=5MB and thumbnail cache <=250MB with eviction.	PERF-STORAGE-001
NFR-010	P0	Universal APK target <=60MB and arm64 target <=35MB, reviewed exceptions allowed for security.	PERF-APK-001
NFR-011	P0	Critical/High data, security, alarm-action and accessibility defects are zero at release.	QA-GATE-001
NFR-012	P0	All operations preserve old or complete state under injected crash.	QA-ATOMIC-001

3.10 Release and maintenance

ID	Priority	Requirement	Verification
REL-001	P0	Public APK is signed and tied to an immutable source tag.	REL-SIGN-001
REL-002	P0	Release publishes artifact SHA-256 and signing certificate fingerprint.	REL-HASH-001
REL-003	P0	Release includes privacy, known limitations, changelog, licenses and documentation.	REL-ASSET-001
REL-004	P0	Published artifact is independently downloaded, verified and installed.	REL-PUBLISH-001
REL-005	P0	Upgrade tests cover every supported public database and archive version.	REL-UPGRADE-001
REL-006	P0	Manifest and current Android/RN/Play policy baseline are refreshed before release.	REL-RESEARCH-001
REL-007	P0	Portfolio claims use measured achieved evidence, not planned budgets.	PORT-AUDIT-001
REL-008	P0	Public screenshots/demo use owned, licensed, public-domain or synthetic media.	PORT-LICENSE-001
REL-009	P0	No debug APK, signing key or private fixture is	REL-SECRET-001

ID	Priority	Requirement	Verification
REL-010	P0	published. End-of-life or withdrawn release is clearly labeled with export/recovery guidance.	REL-EOL-001

4 Verification suite dictionary

Prefix	Evidence type
UT / PROP	Pure unit or property test
DB	Room constraint, DAO, migration or query test
IT	Android instrumented integration test
E2E	User-flow end-to-end test
SEC	Security/static/fuzz/manifest test
A11Y / L10N	Manual and automated accessibility/localization test
PERF / SOAK	Measured performance, battery or long-run reliability evidence
UI / COPY	Component, screenshot, interaction or content review
BUILD / ARCH	Build-time or dependency-boundary inspection
REL / PORT / DOC	Release, portfolio or documentation evidence
MAN	Controlled manual platform scenario

A range such as IT-NATIVE-ACTION-001..009 is a defined group in the test repository/report, not permission to claim unimplemented placeholders. Before release, the generated QA report expands every range into concrete passing cases.

5 Critical acceptance scenarios

5.1 AC-01 - First reminder offline

Given a clean install in airplane mode, the user imports a supported local video, creates a daily Standard reminder and sees a correct next occurrence. No network permission or request exists.

5.2 AC-02 - Locked presentation

Given notifications/exact/full-screen eligibility and a locked non-interactive phone, when the reminder is due, a native alarm surface appears, sound/vibration follows profile, and Play/Snooze/Dismiss work with RN disabled.

5.3 AC-03 - Unlocked noninterruption

Given the user is actively using another app, when a reminder is due, no full-screen activity launches. Android receives a high-importance notification without FSI and actions are available. Absence of a heads-up due to system settings is reported as platform behavior, not overridden with overlay.

5.4 AC-04 - Play transition

Given an active alert, when Play is selected, sound/vibration stops within the action budget, the occurrence resolves once, and attached media begins only after the player becomes visible. Missing media fails safe without continued ringing.

5.5 AC-05 - Snooze idempotency

Given duplicate Snooze PendingIntents for one nonce, exactly one child occurrence is created, one next alarm is registered, and every duplicate response returns the same absolute snooze time.

5.6 AC-06 - Process death and reboot

Normal process death does not prevent the due native alert. Reboot restores a generic pre-unlock envelope and full detail after unlock. No labels or paths exist in device-protected storage.

5.7 AC-07 - Exact access revoked

Future exact alarms are canceled/reconciled, active intent is retained, Health shows Limited/Action needed, and no silent inexact downgrade occurs unless preauthorized.

5.8 AC-08 - Notification blocked

No invisible continuous sound service starts. The occurrence is recorded blocked/needs setup and the next app launch explains recovery.

5.9 AC-09 - Import interruption

Crash/cancel/storage loss at each copy phase leaves no partial normal asset and startup repair reaches a documented state.

5.10 AC-10 - Export/import round trip

All supported logical records are canonically equal after clean-device Replace restore and every media hash matches.

5.11 AC-11 - Malicious archive

Traversal, absolute, duplicate, symlink, bomb, invalid checksum and higher-major fixtures are rejected before current data or scheduler changes.

5.12 AC-12 - Replace crash recovery

Termination at every restore phase recovers to the old state or complete new state, never mixed references or missing current data without rollback path.

5.13 AC-13 - Accessibility

A TalkBack user at 200% font completes import, reminder creation, locked alert action, Health recovery and backup inspection without focus trap, clipped action or color-only information.

5.14 AC-14 - Idle battery

With reminders configured and none due, no process/service/wake lock/timer is resident for time monitoring and one ordinary alarm registration exists.

5.15 AC-15 - Public release integrity

The downloaded APK matches published SHA-256, verifies under the published certificate fingerprint, upgrades from prior public version, and contains no prohibited permission/exported component.

6 Release acceptance decision

A release is **Accepted** only when every P0 row has passing evidence, all critical scenarios pass, and no Critical/High blocker remains in the domains defined by MR-14. `Not applicable` requires a written reason tied to product scope and reviewer approval; it cannot be used to hide an unimplemented P0 feature. Any platform limitation is accepted only when the specified fallback and user disclosure both pass.

6.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.

Research Baseline and Official Sources

Media Reminder - Offline-First Adaptive Media Alarm

Prepared for Mohamed Aslam Abdul

2026-08-05

Contents

0.1 Document control

Field	Value
Document ID	MR-22
Version	1.0
Status	Approved baseline
Last updated	2026-08-05
Product owner	Mohamed Aslam Abdul
Package identifier	com.aslam.mediareminder
Purpose	Record dated current platform facts, primary sources, design impacts, uncertainties and mandatory refresh triggers.

Reading rule: This pack specifies a production-oriented Android application, not a promise that third-party devices will behave identically. Where Android or an OEM controls presentation, timing, sound, or permissions, the app provides transparent status and the strongest compliant fallback.

0.2 Document conventions

The words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are normative. A requirement ID is stable once published. Requirement IDs may be retired but **MUST NOT** be reassigned to a different meaning.

Term	Meaning
Reminder	A user-authored instruction that connects content, a schedule, and a presentation profile.
Occurrence	One calculated due instance of a reminder.
Alarm session	The bounded runtime state created when an occurrence is actively alerting.

Term	Meaning
Media asset	An app-owned local video, audio, image, or text item.
Profile	Reusable alert behavior such as Gentle, Standard, or Persistent.
Exact-alarm access	Android special app access that allows exact scheduling where the platform requires it.
Full-screen intent	Android notification mechanism for urgent, time-sensitive activity presentation. It is not a general overlay.
Heads-up notification	System-rendered high-priority notification shown temporarily over the current app when Android permits it.
Source of truth	The authoritative specification or local persisted record for a decision or state.

1 Research snapshot

This document records current external platform facts used to choose the baseline. It is not a substitute for refreshing official documentation before implementation or release. Software versions, distribution requirements and permission policies are temporally unstable.

2 Baseline conclusions

- Android 17 is released and API 37 is the current compile/compatibility target.
- React Native 0.86.x is the current active stable production line on 2026-08-05; 0.87 is scheduled but not stable until 2026-08-10.
- Google Play's published 2026 requirement for new apps/updates is target API 36 or higher.
- Exact alarms are appropriate for core user-facing alarm functions, but `SCHEDULE_EXACT_ALARM` is user-granted and may be denied/revoked.
- Full-screen intent is restricted to genuine calling/alarm cases and is not an overlay substitute.
- Heads-up notification geometry/presentation is owned by Android/OEM and cannot be fixed to exactly 20% of the display.
- Android 17 background audio hardening reinforces a bounded foreground-service/visible-activity lifecycle; alarm audio with exact access and `USAGE_ALARM` receives a documented exemption from the additional target-37 WIU condition, but still needs a valid lifecycle.
- Photo Picker/SAF and app-specific storage satisfy selected-file import without broad gallery permission.
- Android Auto Backup can move app data unless configured; explicit data extraction exclusions are required for the strict manual local-only promise.

3 Official source register

ID	Official source	Baseline fact and design impact	Recheck
SRC-001	Android 17 is here - Android Developers, accessed 2026-08-05	Android 17 was released in June 2026 and is the current platform compatibility baseline. Impact: Compile and compatibility test against API 37; do not assume target policy without checking Play requirements.	Before every release or Android QPR compatibility cycle.
SRC-002	React Native Releases Overview - React Native, accessed 2026-08-05	React Native 0.86.x is an active stable series on 2026-08-05; 0.87 stable is scheduled for 2026-08-10 and is not yet the production baseline. Impact: Pin 0.86.x for initial implementation and reevaluate after 0.87 stabilizes with native dependency compatibility.	Before repository bootstrap and every RN upgrade.
SRC-003	Meet Google Play target API level requirement - Android Developers, accessed 2026-08-05	New apps and updates must target Android 16/API 36 or higher under the published 2026 requirement. Impact: Initial targetSdk 36; refresh before Play submission.	Before every store submission.
SRC-004	Schedule alarms - Android	AlarmManager can operate	When target SDK or alarm policy

ID	Official source	Baseline fact and design impact	Recheck
	Developers, accessed 2026-08-05	outside the app lifetime; exact alarms require appropriate access on modern Android and setAlarmClock is the most visible critical exact mechanism. Impact: Use one next one-shot alarm, request SCHEDULE_EXACT_ALARM contextually, check revocation and avoid polling.	changes.
SRC-005	Android 14 behavior changes - exact alarms - Android Developers, accessed 2026-08-05	SCHEDULE_EXACT_ALARM is denied by default for many fresh installs targeting Android 13+ on Android 14. Impact: Treat exact access as setup state and restore permissions separately from backup.	When minimum/target policy changes.
SRC-006	Android 14 behavior changes - full-screen intent - Android Developers, accessed 2026-08-05	For target 34+, full-screen intent eligibility is restricted to calling and alarm use cases and store policy may revoke default access. Impact: Use FSI only for genuine locked alarm behavior, probe eligibility and provide notification	Before Play submission and target SDK update.

ID	Official source	Baseline fact and design impact	Recheck
		fallback.	
SRC-007	Display time-sensitive notifications - Android Developers, accessed 2026-08-05	Full-screen intents are for urgent time-sensitive cases and are delivered through notifications, not arbitrary app overlays. Impact: Native alarm activity is launched by eligible notification only in locked/non-interactive path.	When notification/full-screen APIs change.
SRC-008	About notifications - Android Developers, accessed 2026-08-05	Android controls notification surfaces including heads-up behavior. Impact: Do not promise precise heads-up height/duration; design compact content and effective channels.	When Material/notification behavior changes.
SRC-009	Notification runtime permission - Android Developers, accessed 2026-08-05	Android 13+ uses POST_NOTIFICATIONS runtime permission for nonexempt notifications. Impact: Request contextually and never start invisible ringing when blocked.	When target SDK/permission behavior changes.
SRC-010	Optimize for Doze and App Standby - Android Developers, accessed 2026-08-05	Doze defers background work and imposes quotas; exact alarm use must be justified. Impact: No polling; explicit exact/limited modes; bounded	When Android background policy changes.

ID	Official source	Baseline fact and design impact	Recheck
		retries.	
SRC-011	Background audio hardening - Android Developers, accessed 2026-08-05	Android 17 restricts background audio unless a visible activity or compliant foreground service exists; exact-alarm permission plus USAGE_ALARM has a documented exemption for the additional target-37 WIU condition. Impact: Use bounded mediaPlayback FGS for ringing, USAGE_ALARM, visible notification and explicit stop lifecycle; test API 37 hardening.	Before targeting API 37 and after QPR changes.
SRC-012	Photo picker - Android Developers, accessed 2026-08-05	Photo Picker gives access to selected images/videos instead of the whole media library and falls back to ACTION_OPEN_DOCUMENT where unavailable. Impact: Avoid broad media permission for visual imports.	When picker/activity dependency changes.
SRC-013	Access documents and other files from shared storage - Android Developers, accessed 2026-08-05	Storage Access Framework supports user-selected documents and create/open flows. Impact: Use for audio/general	When storage policy changes.

ID	Official source	Baseline fact and design impact	Recheck
		import and ZIP export/import.	
SRC-014	Access app-specific files - Android Developers, accessed 2026-08-05	Internal app-specific storage is private, needs no storage permission and is removed on uninstall. Impact: Copy media into internal app storage and disclose uninstall behavior.	When storage architecture changes.
SRC-015	Save data in a local database using Room - Android Developers, accessed 2026-08-05	Room provides SQLite abstraction, compile-time query verification and migration support. Impact: Use Room as native source of truth with migration tests.	When Room major version changes.
SRC-016	Setting up file sharing - Android Developers, accessed 2026-08-05	FileProvider supports secure content URI sharing with temporary grants. Impact: Share completed exports read-only without exposing internal paths.	When provider/share code changes.
SRC-017	Back up user data with Auto Backup - Android Developers, accessed 2026-08-05	Auto Backup can include cloud and device-to-device data; allowBackup=false alone may not disable D2D on every Android 12+ manufacturer implementation. Impact: Provide dataExtractionRul	When target SDK or backup policy changes.

ID	Official source	Baseline fact and design impact	Recheck
		es and legacy exclusions for all app data; manual ZIP is the supported transfer.	
SRC-018	Native Modules: Introduction - React Native, accessed 2026-08-05	Turbo Native Modules and Codegen provide a typed New Architecture bridge to native platform code. Impact: Define coarse-grained typed native use cases and keep alarm path native.	With each RN upgrade.
SRC-019	Principles for improving app accessibility - Android Developers, accessed 2026-08-05	Android accessibility guidance emphasizes labels, accessible actions, non-color cues and accessible media. Impact: TalkBack/alternative action/contrast requirements are P0.	Before accessibility audit.
SRC-020	Test your app accessibility - Android Developers, accessed 2026-08-05	Effective accessibility testing combines approaches and user-perspective testing. Impact: Manual TalkBack/task evidence supplements automated scanning.	Before release.

4 Source quality rules

1. Use Android Developers, React Native and other primary official documentation for platform/library decisions.
2. Community posts may identify an OEM symptom but cannot define platform guarantees.
3. Record access date and exact target/API affected.
4. When official sources disagree or are ambiguous, choose the safer less intrusive behavior and add a physical-device test.
5. Never cite a future scheduled release as stable before its actual official release date.
6. Refresh this document before selecting dependency versions, changing target SDK, submitting to a store, or publishing reliability claims.

5 Known uncertainty

OEM notification ranking, battery policies, lock-screen behavior and audio routing remain partially device-controlled even when the app follows official APIs. The product therefore uses a Health/Test flow and an observed device matrix. This is not treated as a reason to request overlay or unrestricted battery permission.

Android developer verification and sideloading policy is evolving in 2026. Direct APK guidance must be refreshed immediately before public distribution. The technical pack intentionally avoids a promise that any current sideload process remains unchanged indefinitely.

6 Baseline refresh checklist

- Confirm latest stable React Native and compatible Gradle/AGP/JDK matrix.
 - Confirm current Android release/API and target SDK requirement.
 - Re-read exact-alarm and full-screen intent policy plus Play declarations.
 - Re-read foreground service types/background audio behavior.
 - Confirm Photo Picker/SAF and backup/data extraction behavior.
 - Run compatibility tests on current Android/QPR and at least one OEM build.
 - Update ADR-019 if baseline changes; update MR-06, MR-14, MR-20 and traceability.
-

6.1 Governance

This document is part of the **Media Reminder Source-of-Truth Pack v1.0**. In case of conflict, apply this precedence order: (1) explicit platform safety and permission rules, (2) Architecture Decision Records, (3) Android specification, (4) data and backup specifications, (5) PRD and feature specification, (6) UX and visual guidance, (7) implementation notes. Record any intentional deviation in the decision log before release.